Servomotor Python API Documentation

Version 1.5

Generated: 2026-11-11

Latest Firmware Versions

At the time of generating this API reference, the latest released firmware versions for the servomotors are:

• **Model M17:** servomotor_M17_fw0.17.0.0_scc3_hw1.5.firmware

If you are experiencing problems, you can try to set the firmware of your product to this version and try again, and report the problem to us using the feedback page.

Table of Contents

1. Hardware Setup
2. Install the Python Library
3. Controlling the Servomotor From the Command Line
4. Getting Started
5. Know-How, Best Practices, and Gotchas
6. Data Types
7. Command Reference
8. Basic Control
9. Configuration
10. Device Management
11. Motion Control
12. Other
13. Status & Monitoring
14. Unit Conversions
15. Error Handling
16. Error Codes

Hardware Setup

This section covers everything needed to physically connect and power the servomotor before any software is involved.

What you need

• One or more Gearotons servomotors (M17 series: M17-34, M17-40, M17-48, or M17-60).

• A DC power supply providing 12 to 24 V. Budget at least 1.1 A per motor at your chosen voltage (maximum current draw is 1.1 A for the M17-60/48/40 and 1.0 A for the M17-34; rated power 26.4 W for the larger models, 24.0 W for the M17-34).

• A USB-to-RS485 adapter (sold separately; any generic USB-to-RS485 adapter works). The host can be a Mac, PC, Raspberry Pi, Arduino, or ESP32.

• The motor ships with its 6-wire pigtail.

Wiring

Each motor has a 6-pin connector and ships with a matching pigtail wire that plugs into it. The pigtail's six wires, in connector order, are:

• BLACK = GND (power supply negative)

• RED = motor + (power supply positive, +12 to +24 V)

• GREEN = RS485 line A

• BLUE = RS485 line B

• GREEN = RS485 line A (second set)

• BLUE = RS485 line B (second set)

There are two sets of A/B wires so that motors can easily be daisy-chained: one A/B pair comes from the previous device (or the RS485 adapter) and the other A/B pair continues on to the next motor. Electrically the two pairs are the same bus lines.

Connect RED to the supply + terminal and BLACK to the supply - terminal. Connect one GREEN wire to the A terminal and one BLUE wire to the B terminal of the RS485 adapter. Keep each A/B pair twisted if possible.

Grounding: run a common ground between the RS485 adapter's GND terminal and the motors. Because the motors' BLACK power leads are the same ground, tying the adapter's GND to the power supply's negative rail achieves this. A shared ground reference is standard RS485 practice; without one, RS485 communication can be unreliable.

Multiple motors: any number of motors share one bus and one adapter. Either daisy-chain them (adapter A/B into the first motor's first A/B pair, its second A/B pair on to the next motor, and so on) or wire them in a star (all A wires to the adapter's A terminal, all B wires to B) — the daisy chain is what the second A/B pair is for. Power is wired in parallel: all RED leads to supply +, all BLACK leads to supply -. No termination resistors are specified for this product; short benchtop buses typically work without them.

Power supply sizing with many motors: motor inrush and acceleration peaks add up. If many motors may accelerate simultaneously, either size the supply for coincident peaks (about 1.1 A each) or stagger the starts in software (even a random 0-1 s offset per motor spreads the peaks effectively).

Serial link parameters

• Baud rate: 230400 (fixed — there is no autobaud), 8 data bits, no parity, 1 stop bit.

• The bus is half-duplex: only one device transmits at a time. Motion commands execute asynchronously on each motor, so the host can command one motor and immediately talk to others while the first moves.

• Each motor has a factory-programmed 64-bit unique ID and can be assigned a one-byte alias (0-251) for short addressing. Alias 255 is broadcast to all devices.

LED indicators

• GREEN flashing slowly (about once per second): heartbeat — the application firmware is running normally.

• GREEN flashing quickly: the bootloader is running instead of the application (the device is not ready for normal commands; send 'System reset' to relaunch the application).

• GREEN also lights briefly while a packet is being received, so you will see it flicker with communication traffic on the bus.

• RED flashing a repeating count of N blinks with a pause: fatal error number N. Count the blinks or read the code with 'Get status'. An error code of 0 (only possible via a deliberately triggered test) shows the red LED continuously on.

Buttons

The motor has two small buttons:

• Reset: resets the microcontroller. All volatile state (queued moves, zeroed position, limits, enabled state) returns to power-on defaults.

• Test: brief press = spin one way; hold more than 0.3 s and release = spin the other way; hold at least 2 s and release = enter closed loop mode; hold more than 15 s and release = run self-calibration. During calibration the shaft spins and MUST be free to rotate — remove any load first.

Mechanical and environmental

• Standard NEMA 17 mounting: 42.2 x 42.2 mm faceplate with no protrusions. Body heights: M17-60 = 59.7 mm, M17-48 = 48.7 mm, M17-40 = 40.1 mm, M17-34 = 33.5 mm; shaft length 20.6 mm on all models. Weights: 470 / 360 / 285 / 210 g respectively.

• Rated torque: M17-60 = 0.65 N.m, M17-48 = 0.55 N.m, M17-40 = 0.42 N.m, M17-34 = 0.28 N.m. Rated maximum speed: 560 RPM for all M17 models. As of firmware 0.15.8.0, the default maximum commanded velocity is also 560 RPM (9.333 rotations/second). This limit does not guarantee that the motor can attain the commanded speed under its operating conditions. Bench tests on 40 units reported unloaded speeds of 8.59-8.65 rotations/second at both 20 V and 24 V; these observations describe those test conditions, not a universal speed ceiling. In closed-loop operation, commanding a speed the motor cannot attain can accumulate position error and trigger fatal error 45 if the configured position-deviation limit is exceeded.

• Built-in magnetic encoder; closed-loop PID control runs on-board at 31.25 kHz.

• Operating temperature 0 to +80 C; storage -20 to +60 C; humidity 20-80% RH non-condensing; IP20 (indoor use).

• Integrated over-voltage and over-temperature protection. Protection trips are fatal errors: the motor disables itself and latches the error code until reset (over-temperature trips at roughly 80 C internal; over-voltage at a firmware-set threshold of 32 V on the M17). Motor current is limited continuously by the firmware-set current limit rather than by a fatal-error trip.

• Regeneration warning: a rapidly decelerating or externally driven motor pumps energy back into the supply and raises the bus voltage. If the supply cannot absorb it, the overvoltage protection (fatal error 14) trips. Decelerate large inertias gently, and avoid spinning the shaft forcefully by hand while powered.

Install the Python Library

You need to install the servomotor Python library before you can use it in your code. Run this command:

```
pip3 install servomotor
```

If you want to work in a virtual environment, you can create it, activate it, and install the library:

For macOS/Linux:

```
# Create virtual environment
python3 -m venv venv
# Activate virtual environment
source venv/bin/activate
# Install the needed library
pip3 install servomotor
```

For Windows:

```
# Create virtual environment
python -m venv venv
# Activate virtual environment
venv\Scripts\activate
# Install the needed library
pip install servomotor
```

After installation, you can verify the servomotor library is installed correctly by running:

```
python3 -c "import servomotor; print('Servomotor library installed successfully!')"
```

Controlling the Servomotor From the Command Line

You can send commands to the servomotor from the command line using the servomotor_command utility, which gets installed along with the Python library. Make sure to install that library according to the instructions above. After installation, the servomotor_command program should be in the path. You can try running some of the following commands to communicate with the servomotor(s):

```
servomotor_command --help   # get help information about this command line utility
servomotor_command -c       # print out all commands currently available
servomotor_command -P       # select the serial port to be used for communication from a menu
servomotor_command "Detect devices"  # detect connected servomotors

# The following will set the alias for a device. Use the unique ID discovered from the
# "Detect devices" command. The example sets the alias to X. After the alias is set, you
# can send commands to the device and target it with the alias instead of the unique ID.
servomotor_command -a 1122334455667788 "Set device alias" X  # set the alias for a device

servomotor_command -a X "Enable mosfets" # enable the MOSFETS of the servomotor with alias X
servomotor_command -p /dev/ttyUSB0 "Enable mosfets"  # same as above but hard specify the port
servomotor_command -a X "Trapezoid move" 3000000 32000  # move the motor
```

Getting Started

This section provides a complete example showing how to initialize and control a servomotor.

Complete Example Program:

```
#!/usr/bin/env python3
"""
Minimal trapezoid move: rotate 1 turn in 1 second.
Edit ALIAS below if needed. Uses rotations and seconds.
"""
import time, servomotor
from servomotor import communication

# Hard-coded settings for a minimal demo
ALIAS = 'X'                             # Device alias, change if needed
SERIAL_PORT = "/dev/tty.usbserial-110"  # Serial device path; change if needed (e.g., "COM3" on
                                        #  Windows)
DISPLACEMENT_ROTATIONS = 1.0            # 1 rotation
DURATION_SECONDS = 1.0                  # 1 second
DELAY_MARGIN = 0.10                     # +10% wait margin because the motor's clock is not
                                        #  perfectly accurate

communication.serial_port = SERIAL_PORT # if you comment this out then the program
                                        #  should prompt you for the serial port or it will use
                                        #  the last used port from a file
servomotor.open_serial_port()

m = servomotor.M3(ALIAS, time_unit="seconds", position_unit="shaft_rotations", verbose=0)
m.enable_mosfets()
m.trapezoid_move(DISPLACEMENT_ROTATIONS, DURATION_SECONDS)
time.sleep(DURATION_SECONDS * (1.0 + DELAY_MARGIN))
m.disable_mosfets()

servomotor.close_serial_port()
```

Know-How, Best Practices, and Gotchas

Everything in this section was verified against the firmware source code and the project's working test programs. Follow these practices and the motor will behave predictably; ignore them and you will hit fatal errors, lockups, or silent misbehavior.

The golden rules

1. After 'System reset' (and after power-on), wait at least 0.5 seconds before sending ANYTHING on the bus, and keep the bus completely silent during that wait. The device passes through a bootloader window before launching the application. Any valid packet addressed to it (or broadcast) in that window pins it in the bootloader, where normal commands do not work ('Get status' then returns flags 0x0001). The bundled test programs wait 1.5 s for extra margin; 0.5 s is the working minimum with 2x margin. If a device does get pinned in the bootloader, send 'System reset' again and wait the full delay. Evidence: bench-measured boundary — probes 150-200 ms after reset pinned the device in the bootloader 5/5 times; at 250 ms and later, 0/5, so the window ends between 200 and 250 ms; probing early pins RELIABLY, not occasionally.

2. Begin every session with 'System reset' (then the post-reset wait). This guarantees a known-clean state: MOSFETs disabled, default limits and gains, CRC32 enabled, no latched fatal error, position and clock zeroed.

3. After 'Enable MOSFETs' the motor is immediately ready to use, but it may twitch or rotate slightly as the rotor snaps to the nearest commutation step (bench-measured up to ~4 degrees). Only if you are about to zero the position precisely or arm a tight position-deviation limit, let that transient settle first (about 0.3 s) — zeroing or arming tight limits during the twitch gives a corrupted zero or a spurious fatal error 45. For twitch-sensitive mechanisms, enter closed loop directly with go_to_closed_loop() instead: its engagement is dramatically gentler (bench-measured ~0.01 degrees).

4. Every velocity or acceleration move sequence MUST end with the motor back at zero velocity before the queue empties. A velocity move does not stop by itself — the motor holds the last commanded velocity, and when the queue runs dry at nonzero velocity the firmware raises fatal error 18 (ERROR_RUN_OUT_OF_QUEUE_ITEMS) within one control tick. End velocity sequences with move_with_velocity(0, 0.01) queued back-to-back with the moving segments; end acceleration sequences with a mirrored deceleration segment.

5. Fatal errors LATCH. Once any fatal error trips, the motor disables itself, the red LED blinks the error code, and every command except 'Get status' and 'System reset' fails (each failing command's error reply carries the code). Bench-verified exhaustively: 12 probed commands — ping, all position/time/temperature readers, enable/disable MOSFETs, emergency stop, zero position, reset time, detect devices — ALL receive the error reply, and NONE of them un-latches the fault; 'System reset' alone recovers. First find out WHICH error it is — a FatalError exception carries the code (e.args[0] in Python), or read it with 'Get status' — and look it up so you can address the cause. Then clear it with system_reset() followed by the post-reset wait (0.5 s or more of bus silence).

6. NEVER send 'Test mode' with values 10-13 (LED tests): they lock the firmware up after replying, and only a power cycle (or the physical reset button) recovers the device. As of firmware 0.15.4.0, 'Test mode' 0 safely clears all test modes; in OLDER firmware value 0 also locks the device up — 'System reset' always safely clears test modes on any firmware.

7. An empty queue does not always mean motion is finished: if a commanded velocity exceeded what the motor can physically reach under its operating conditions, the commanded position runs ahead of the rotor and, in closed loop, the motor keeps moving after the queue empties until it catches up (bench-measured: 0.35 s of extra travel after a 1 s profile at an unattainable speed). For true motion-complete, require queue empty AND position settled (two consecutive equal hall readings, or commanded-minus-measured within tolerance). Practically: poll 'Get n queued items' to 0, then allow an extra 0.1-0.3 s for mechanical settling before reading exact positions (polling at full rate is harmless; a 0.01-0.1 s sleep between polls just saves host CPU). Alternatively sleep the commanded duration plus ~5% margin (the motor's clock is accurate to well under 1%; +0.41% offset measured on the bench).

8. The motion queue holds at most 32 items, shared by all move commands. A 'Trapezoid move' or 'Go to position' normally occupies 3 slots (accelerate/coast/decelerate — degenerate phases still take a slot, including a zero-displacement dwell), so at most 10 such moves fit. BUT SINCE FIRMWARE 0.15.12.0 IT CAN BE 2, AND THE CONDITION IS COUNTERINTUITIVE: when the acceleration ramp works out to one control tick or less, the planner emits a constant-velocity segment plus a one-tick stop instead of three

segments, and then 16 moves fit instead of 10. The ramp is `maximum velocity / maximum acceleration`, so this happens exactly when you make the speed limit LOW while leaving the acceleration limit high — a slow, precise axis with the 12,000 rot/s^2 factory default. Bench-measured on both a rack motor and the bench motor: at the default speed limit exactly 10 moves were accepted before fatal error 17; with the speed limit at 0.3 rot/s, exactly 16 were. So LOWERING the speed limit INCREASES how many moves fit — the opposite of what most people would guess, and worth knowing if you size a plan against the queue. Do not hard-code either number; poll 'Get n queued items'. 'Move with velocity'/'Move with acceleration' take 1 slot each, and a 'Multimove' entry takes 1 slot per entry; 'Move with velocity'/'Move with acceleration' take 1 slot each. The item currently executing is included in the count. Exceeding the queue is fatal error 17 — and bench-verified, the fault does not just reject the extra item: it ABORTS the entire in-flight choreography (MOSFETs drop, queue cleared). When streaming moves, poll 'Get n queued items' and stay below the cap.

9. Also bench-verified planner algebra: a trapezoid or go-to-position queued after motion that ends at velocity v0 superimposes — it travels an EXTRA v0 x duration and ends at v0, not at rest (measured: MWV(1 rot/s, 1 s) + trapezoid(2 rot, 1 s) landed at exactly 4.0 rot, not 3.0). Only queue them after motion that ends at rest, or account for the superposition.

10. Safety limits live in the CURRENT position frame — 'Zero position' silently moves your fences to new physical locations. Set safety limits AFTER homing and zeroing, and re-send them after any later zeroing if the fence protects real hardware (details under Motion control patterns).

Positioning accuracy: absolute moves converge, and nothing drifts far

A single move delivers up to ONE COUNT LESS than commanded. This is integer division inside the planner, not a mechanical effect, and it is visible with the MOSFETs off: ask for 1,638,400 counts and the commanded position lands on 1,638,399. One count is 0.3 parts per million of a revolution, so it never matters mechanically — but it decides which command to build a repeatable machine on.

• **'Go to position' CONVERGES.** It is absolute, so the second call sees the one-count error and corrects it. Bench-measured, four identical calls to the same target: `1638399, 1638400, 1638400, 1638400`. It is not idempotent on the first call; it is convergent, and idempotent from the second onwards. If you need to land exactly on a coordinate, issue the absolute move twice, or simply re-issue it at each cycle of your loop, which most programs do anyway.

• **Relative 'Trapezoid move' does NOT self-correct**, but it also does not accumulate the way you would fear. Bench-measured, one full rotation split into 10, 50 and 200 relative moves gave a total shortfall of **1 count in every case** — not one count per move. Across every regime measured the total stayed within a few counts: 1-2 counts over a rotation, about 8 counts over 400 rotations, about 5 counts over 5,000 rotations. The internal position keeps sub-count precision, so the error is bounded rather than cumulative.

Practical rule: use relative moves freely; if a machine must return to an exact coordinate, use an absolute 'Go to position' for that step, and expect the first one to be a count short.

A stale limit from a previous program is a real hazard

This is what golden rule 2 is protecting you from, made concrete. Limits are volatile but they survive until the next reset or power cycle, NOT until your program exits. A program that sets a tight limit and exits leaves the motor configured that way for whatever runs next.

Bench-demonstrated: program A sets the speed limit to 0.2 rot/s and exits. Program B starts without a reset and commands an ordinary one-rotation move — and gets fatal error 16. Nothing in program B is wrong; it inherited a limit it never set and cannot see, because there is no getter for it.

Always begin with 'System reset' and the post-reset wait. If you are debugging a program that "works sometimes", check whether something else ran first.

Recommended startup sequence

Almost every problem people hit is a SEQUENCING problem rather than a misunderstanding of an individual command. The command reference tells you what each command does on its own; this is the order to do them in.

1. Set your units. This is host-side only — nothing is sent to the motor, and units survive a device reset because they never lived on the device.

2. 'System reset', then wait at least 0.5 s with a completely silent bus (golden rule 1). This clears any fatal error left over from a previous run and gives you a known state.

3. 'Set maximum motor current' to a working value (150-200 internal units is typical). Check for an error.

4. 'Set maximum velocity' and 'Set maximum acceleration' explicitly, even if the defaults would do. Explicit limits make your program immune to a firmware default changing under it — which has happened (see the firmware version notes below).

5. Choose ONE of the two engagement paths, not both:

• Open loop: 'Enable MOSFETs', wait about 0.3 s for the commutation snap to settle, then 'Zero position'.

• Closed loop (preferred when the mechanism is sensitive): 'Zero position' FIRST, then 'Go to closed loop', which enables the MOSFETs itself and engages far more gently. Zeroing first is what prevents the closed-loop lurch described below.

6. Optional homing: 'Zero position', then 'Homing', then compare the distance actually travelled against the distance you asked for, then 'Zero position' again to set your origin. Homing does not zero anything itself.

7. Set 'Set safety limits' and 'Set max allowable position deviation' now, AFTER homing and zeroing — they are interpreted in the current position frame, so zeroing later silently moves them.

8. Move. To know a move has finished, poll 'Get n queued items' to 0 and then allow 0.1-0.3 s for mechanical settling (golden rule 7).

9. When you are done, 'Disable MOSFETs'.

Two rules that are easy to state and easy to get wrong:

• Enable the MOSFETs ONCE for a burst of work rather than toggling them around every move — but disable them when the machine will be idle for a long time. An enabled idle motor holds with coil current and warms the driver PCB, and the over-temperature fatal error (40, at roughly 80 C) is only monitored while the MOSFETs are enabled, so it is the energised state that carries the thermal risk. How close you get to the cutoff depends on your current limit, the ambient temperature and how the motor is mounted; an unloaded bench M17 at 24 V sits around 38-45 C even after a long working day, so this is about power and margin rather than an imminent trip. Note the trade-off: a later open-loop 'Enable MOSFETs' can twitch the shaft by up to about 4 degrees, so on a twitch-sensitive mechanism prefer to leave it energised, or re-engage via 'Go to closed loop'.

• After ANY reset, everything volatile is gone: motor current, velocity and acceleration limits, safety limits, position deviation limit, PID gains, the zeroed position, the MOSFET state, and closed-loop mode. Re-apply all of it. Only your host-side unit selection survives.

Recommended program skeleton

There is no required program structure — write your program however you like. The skeleton below is just a recommended starting point that handles the common pitfalls for you:

```
import argparse, time, servomotor

parser = argparse.ArgumentParser()
parser.add_argument('-p', '--port', help='serial port device')
parser.add_argument('-P', '--PORT', action='store_true', help='interactive port menu')
parser.add_argument('-a', '--alias', default='X', help='device alias')
args = parser.parse_args()

servomotor.set_serial_port_from_args(args)   # must precede open_serial_port()
servomotor.open_serial_port()
motor = servomotor.M3(args.alias, time_unit='seconds',
                      position_unit='shaft_rotations',
                      velocity_unit='rotations_per_second',
                      acceleration_unit='rotations_per_second_squared',
                      verbose=0)
try:
    motor.system_reset()
    time.sleep(1.5)                          # golden rule 1
    # ... your work here ...
```

```
finally:
    try:
        motor.disable_mosfets()              # best-effort cleanup
    except Exception:
        pass
    servomotor.close_serial_port()
```

Notes on the skeleton:

• You do not have to pass -p every time. The port is resolved in this order, highest priority first: an explicit -p; -P, which jumps straight to the interactive menu (you asked to choose, so nothing pre-empts that); the SERVOMOTOR_PORT environment variable; the saved default from the last successful run; and finally the interactive menu. So setting SERVOMOTOR_PORT once names the port for every program without any file at all.

• Where the saved default lives (library 0.12.2 and later): a per-user config directory — ~/Library/Application Support/servomotor on macOS, ~/.config/servomotor on Linux, the local AppData servomotor folder on Windows, or $XDG_CONFIG_HOME/servomotor if you have set that. A default saved by an older library, which kept it inside the installed package, is still read, so upgrading does not lose it. Saving can never fail a run: if the location is not writable the library warns and carries on, because the port is already open and remembering it is only a convenience. (Library versions before 0.12.1 did fail there — an uncaught PermissionError after the port was already open, which a normal system-wide install hit every time.)

• LINUX PERMISSIONS, the most common first-run failure there: serial devices belong to the dialout group (uucp on some distributions), so a fresh account gets 'Permission denied' opening /dev/ttyUSB0 even though the port plainly exists and the adapter is plugged in. Fix it once with `sudo usermod -a -G dialout $USER`, then log out and back in — group membership is applied only to NEW logins, so the change appears to do nothing until you do. Library 0.12.1 and later prints this advice when it sees a permission error.

• The M3 constructor's verbose parameter defaults to 2 (prints every packet in hex). Pass verbose=0 for production use.

• The library's default read timeout is 1.2 s. A TimeoutError on a normally-addressed command is essentially always a real fault: wrong port, wrong alias, device in bootloader, device locked up, or a CRC-state mismatch. (One historical exception: firmware before 0.15.4.0 stayed silent when reading an empty multipurpose buffer, so that read timed out to mean "buffer empty"; current firmware responds instead.)

• If a program may leave nonstandard device state behind (changed PID gains, tightened limits, changed current limit), finish with a defensive system_reset() so the next user starts clean.

Units — read this before your first move

• Set units explicitly in the M3 constructor to make your programs self-documenting. The defaults are the first unit of each list: seconds, shaft_rotations, rotations_per_second, rotations_per_second_squared, celsius, internal_current_units, millivolts. (Older library versions defaulted the time unit to 'timesteps' — the raw 32-microsecond internal unit — which made trapezoid_move(1.0, 2.0) mean 64 microseconds; if you may be running an old library, pass time_unit='seconds' explicitly.)

• Available unit names (exact strings): time: seconds, milliseconds, minutes, microseconds, timesteps; position: shaft_rotations, degrees, radians, encoder_counts; velocity: rotations_per_second, rpm, degrees_per_second, radians_per_second, counts_per_second, counts_per_timestep; acceleration: the same family with _squared (except rpm, whose acceleration unit is rpm_per_second); current: internal_current_units, milliamps, amps; voltage: millivolts, volts; temperature: celsius, fahrenheit, kelvin. Units can be changed at runtime with set_position_unit() etc.

• Key conversion constants: 1 shaft rotation = 3,276,800 encoder counts (M17/M23; always confirm with 'Get product specs', which returns the update frequency and counts per rotation). The internal time base is 31,250 timesteps per second (one timestep = 32 microseconds, which is also the control-loop period).

• Every converted value is rounded to the nearest integer internal unit before transmission; values smaller than half an internal unit become zero.

• The clock commands ('Get current time', and the masterTime input of 'Time sync') use microseconds on the wire, and the library converts them correctly from/to whatever time unit you configured. The time-error output of 'Time sync' is the exception: it is always returned as raw microseconds with no conversion. (Older library versions mis-scaled these two commands by ~32x in any unit except 'timesteps' — if you may be running an

old library, read the clock with time_unit='timesteps', which then passes raw microseconds through.)

• (Older library versions only: the velocity unit named 'counts_per_timestep' carried a conversion factor ~104.86x too large. It is correct in the current library.)

• Do not hand-roll internal-unit conversions copied from older scripts: legacy demos (ball_throwing_demo.py, ball_juggling_demo.py, magnetic_disk_machine/*) hardcode per-rotation values such as 4,320,000 or 4,752,000 (64*1350*50 or *55) that do not match current motors, and use an obsolete protocol module. Use the library's unit system.

Driving many motors on one bus — measured on a 35-motor rack

Everything else in this document describes one motor. These numbers come from a bus with thirty-five M17s on it, all on firmware 0.15.12.0, with the MOSFETs off so only the commanded position moves.

• **A broadcast move is exact, not approximate.** One 'Trapezoid move' sent to address 255 moved all 35 motors by *identically* 819,199 counts of a commanded 819,200 — spread ZERO across the whole bus. Broadcast is therefore the right primitive for coordinated multi-axis motion: it is one packet, it starts every axis on the same control tick, and every axis performs the same arithmetic.

• **Addressing motors individually costs about 4 ms each.** Queueing the same move on 35 motors one at a time took 142 ms end to end. That is fine for setup, but it means individually-addressed commands cannot start axes together — the last motor begins moving ~140 ms after the first. Use broadcast when simultaneity matters, and individual addressing only for per-axis parameters.

• **A supervisory polling loop tops out around 7 full sweeps per second.** Reading the position of all 35 motors in a loop sustained 262 reads/second, i.e. 7.5 complete sweeps/second, with **zero errors over 11,000 reads**. Budget accordingly: at 35 axes you get roughly 7 Hz of state, and polling harder just queues behind the bus.

• **One motor faulting does NOT disturb the others.** With one motor deliberately faulted, five others were commanded and all five moved correctly. A single broadcast 'System reset' then cleared the fault on every motor at once (35/35 clean). A fault is genuinely per-device, and bus-wide recovery is one packet.

The gotcha that will bite you here

Unit settings are **host-side, per-motor-object, and sticky**. They persist until you change them and are silently applied to every later reading. Nothing errors.

The same device clock, read five ways within a second of each other:

time unit	value reported
seconds	1.8069
milliseconds	1814.5360
microseconds	1818352.0000
timesteps	56585.1562
minutes	0.0304

In a multi-motor script it is very easy to set a unit for one phase of the program and then read something unrelated in a later phase. That exact mistake — setting `timesteps` for a motion section, then reading clocks further down — produced an apparent **4,150-second disagreement** between motors that was really 133 milliseconds of read-order skew. The number looked like a serious hardware fault and was a unit label. **Set the unit immediately before you read, or set it once at startup and never change it.**

Addressing and multi-motor buses

• Aliases are one byte, usable range 0-251. Values 252/253/254 are reserved by the protocol (attempting to assign them latches fatal error 50); 255 means broadcast and also serves as "no alias assigned".

• Broadcast (alias 255) commands execute on every device but produce NO responses (the library returns [] immediately). You cannot read anything via broadcast. The one exception is 'Detect devices', which is designed to be broadcast and collects one response per device.

• GOTCHA: if you omit the alias, some CLI paths default to 255 (broadcast) — query commands then appear to succeed while returning nothing. Always pass a real alias for anything that expects a reply.

• A wrong alias or wrong unique ID produces a TimeoutError, not an error message — timeout is the only symptom of misaddressing. An UNSUPPORTED COMMAND ID is also silently dropped (bench-verified: no response, no error, device unaffected), so a timeout can mean either "nobody home" or "command unknown to this firmware".

• Device discovery: use servomotor.detect_devices_iteratively(n_detections=3). Each round does broadcast reset, 1.5 s wait, flush_receive_buffer(), then 'Detect devices'; results are merged across rounds by unique ID because single rounds can miss devices (responses arrive at random 0-950 ms offsets to avoid collisions). Each entry gives the 64-bit unique ID and the current alias.

• GOTCHA: after 'Detect devices' the device ignores ALL bus traffic for about 1 second (its collision-avoidance window). Wait at least 1.1 s before sending anything else. Queued motion is NOT disturbed — a move in flight continues and completes normally across the window (bench-verified); only hosts that need to keep STREAMING new segments must not run detection mid-path (the queue could starve into fatal error 18). The detect reply itself can take over 1.2 s end to end (random 0-950 ms delay); the library swallows the terminating read timeout for this multiple-response command, so a missed device shows up as an empty or partial result (never a TimeoutError) — retry and merge rounds.

• 'Set device alias' saves to flash and immediately REBOOTS the device. Keep the bus SILENT and wait at least 0.5 s before addressing it at the new alias — exactly like the post-reset rule. Bench-measured: with a silent bus the device answered 0.3 s after the command, but continuously polling it during the reboot stretched recovery to over 1.2 s (and risks the bootloader-pinning trap of golden rule 1). Broadcasting this command sets the SAME alias on every device on the bus — almost never what you want.

• Devices with alias 255 (unassigned) or duplicated aliases must be addressed by their 64-bit unique ID. The M3 constructor accepts it as an int (e.g. 0x0123456789ABCDEF) or as a string of exactly 16 hex digits (e.g. M3("0123456789ABCDEF")); other multi-character strings are parsed as decimal alias numbers. (Older library versions did not parse hex strings in the constructor — there, pass an int or use servomotor.string_to_alias_or_unique_id() first.) The -a command-line flag can carry a 16-hex-digit unique ID two ways: the servomotor_command.py CLI converts it with servomotor.string_to_alias_or_unique_id(), and, with the current library, test programs that pass -a straight to the M3 constructor also accept it — the constructor parses a 16-hex-digit string as a unique ID (M3.py). Only older library versions, whose constructor did not parse hex strings, required the CLI-style conversion first.

• To control several motors, either create one M3 object per motor (different aliases) or create one object and retarget it with motor.use_this_alias_or_unique_id(...). Note that, unlike the M3 constructor, use_this_alias_or_unique_id() stores its argument as-is with no string parsing and no reserved-alias validation. Pass an already-parsed integer — ord('X') for a one-byte alias, or int('0123456789ABCDEF', 16) for a 64-bit unique ID; a single character or hex string stored here is not converted and fails later with a TypeError deep in send_command (the 'X' <= 255 comparison). Keep one extra M3(255) instance for broadcast operations like a bus-wide 'System reset' or emergency 'Disable MOSFETs'.

• Fleet-scale figures (measured on a 39-motor bus): a full-fleet position sweep takes ~150 ms (~6.7 Hz fleet telemetry; per-transaction RTT is uniform across devices at ~3.6 ms regardless of bus position); a single-round 'Detect devices' found all 39 in 13 of 15 tries (the 3-round default is ample); free-running clocks across a 39-unit population spanned ~5000 ppm (so two unsynced motors can drift apart ~5 ms per second — use 'Time sync' or generous margins); round-robin syncing the whole fleet at the bus's full rate held every motor within ~250 microseconds of the master. Phased choreography works by broadcasting 'Reset time' then queueing per-motor [dwell-until-slot, move] pairs with dwells computed against the shared epoch. One faulted motor does not disturb the others (verified: 38/38 executed a broadcast move around a latched neighbor) and rejoins after a solo reset. GOTCHA: a synchronized fleet-wide hard stop can glitch the HOST's receiver (one in-flight reply corrupted in each of two trials, device counters clean) — read telemetry after, not during, mass transients, and wrap such reads in retry+flush.

• The bus stays fully usable while motors execute moves — pinging at 100 Hz during high-speed motion works. You only need bus silence during post-reset windows.

Motion control patterns

• 'Trapezoid move' is RELATIVE (a signed displacement from the end of previously queued motion); 'Go to position' is ABSOLUTE. Successive commands chain correctly because both plan from the predicted end-of-queue position, not the live position.

• Both commands assume the motor starts the move at rest. If the preceding queued motion ends at nonzero velocity, the profiles superimpose and the motor will NOT stop at the expected position. Only queue them

when prior motion ends at rest.

• A move's success response confirms validation and queueing only — it says nothing about motion completion.

• MIGRATION NOTE (firmware 0.15.8.0): the boot-default velocity limit dropped from a buggy 68 rot/s to the datasheet 560 RPM (9.333 rot/s), and the acceleration default became 12,000 rot/s^2. Any pre-0.15.8.0 program that commands faster than 560 RPM now gets fatal error 16/28 at queue time — add an explicit 'Set maximum velocity' call first (the limit raises up to 18.67 rot/s). Three of this project's own long-standing test programs needed exactly this one-line change.

• Set 'Set maximum velocity', 'Set maximum acceleration', and safety limits BEFORE queueing moves — trajectory planning and validation use the values in effect at queue time, and limits are enforced by REJECTING violating moves with a latched fatal error (15/16/26/27/28), not by clamping them. 'Set maximum motor current' is different: it is not a planning input and takes effect immediately, at any time, even mid-move.

• Safety limits are interpreted in the CURRENT position frame: 'Zero position' shifts which physical locations they refer to (bench-demonstrated — after zeroing at 2.5 rotations, a fence set at 3.0 sat at physical 5.5 and the motor happily drove past the old physical fence). Set safety limits AFTER homing and zeroing, and re-send them after any later zeroing if the fence protects real hardware. Do not send an inverted window (lower > upper): firmware 0.15.5.0 and later rejects it cleanly with fatal error 34; firmware up to 0.15.4.0 stored it unvalidated, faulted with error 25 on the next 32-microsecond tick, and the race with the command's own reply could lock the device up completely until a power cycle.

• Speed and acceleration out of the box: firmware 0.15.8.0 and later default to a commanded velocity limit of 560 RPM (9.333 rot/s), matching the rated maximum speed, and an acceleration limit of 12,000 rot/s². Reported unloaded bench speeds were around 8.6 rotations/second, with spin-up acceleration around 11,000 rot/s² at 24 V and 9,300 at 20 V. Those observations were below the configured limits; they do not establish a universal attainable speed. Actual attainable motion depends on operating conditions, and load can reduce it. A command within the configured limits can still exceed what the motor can physically achieve: the rotor can fall behind and continue moving after the queue empties (see golden rule 7). If the position deviation exceeds its configured limit, fatal error 45 stops the motor.

• Changing the limits: they can be raised (silently clamped at 2x/4x the defaults) or lowered with 'Set maximum velocity'/'Set maximum acceleration', with rejection errors 16/28/15 enforcing them immediately.

• Supply voltage: in the reported tests on 40 units, unloaded speeds were 8.59-8.65 rot/s at both 20 V and 24 V. Median unloaded spin-up acceleration was approximately 9,300 rot/s² at 20 V and 11,000 at 24 V. These results describe the tested conditions; they do not establish a voltage-independent speed ceiling for all operating conditions.

• Wire-format ceilings to know: the u32 limit field could carry a request up to ~39.06 rot/s, and MOVE commands carry velocity as a signed i32 (a ~19.5 rot/s wire ceiling), but on firmware 0.15.8.0 neither wire cap is the binding constraint — 'Set maximum velocity' silently clamps the stored limit at 18.67 rot/s (2x the default), and any move faster than the stored limit is rejected with fatal error 16, so 18.67 rot/s is the real ceiling on both the settable limit and the fastest commandable move.

• Choosing how to stop mid-motion (bench-measured from 5 rotations/second, free shaft): a commanded stop — 'Reset time', which clears the queue and forces velocity to zero with MOSFETs still on — halted in ~0.02 rotations, about HALF the travel of 'Emergency stop' or 'Disable MOSFETs' (~0.05 rotations), which remove torque and let the rotor freewheel. With real load inertia the gap grows. For the shortest physical stop keep the MOSFETs on and command zero velocity; use 'Emergency stop' when you want torque removed.

• GOTCHA: do not lower 'Set maximum velocity' while moves are executing — the limit takes effect immediately and the runtime check trips fatal error 16 on the next control tick if the executing velocity now exceeds it. Change limits only while stopped with an empty queue.

• 'Set maximum acceleration' with 0, or any standalone move command with a duration of 0, is rejected with fatal error 34 (ERROR_PARAMETER_OUT_OF_RANGE) as of firmware 0.15.4.0. Exception: a zero-duration entry inside a 'Multimove' is silently dropped and consumes no queue slot (verified in firmware: add_to_queue returns before incrementing the queue count). (Older firmware accepted both: the zero max-acceleration caused a divide-by-zero in later planning, and a zero-duration move was a silent no-op that still returned success.)

• 'Set maximum velocity' with 0 is rejected with fatal error 34 as of firmware 0.15.6.0. (Firmware through 0.15.5.0 accepted it, and then every trapezoid/go-to-position was silently planned as a zero-motion dwell — success response, queue drains over the commanded duration, shaft never moves.) Beware that any limit

below half an internal unit rounds down to 0 during unit conversion, so a tiny requested limit can hit this rejection unexpectedly.

• THE SPEED YOU ASK FOR IS NOT THE SPEED THAT GETS VALIDATED. 'Trapezoid move' and 'Go to position' take a displacement and a duration, so it is natural to compute displacement/duration and check it against your velocity limit. The firmware never looks at that average. It builds a fixed ramp of rampTime = maxVelocity/maxAcceleration (truncated down to a whole 32-microsecond timestep), then sizes the profile so that peakVelocity = displacement / (duration - rampTime) and peakAcceleration = peakVelocity / rampTime, and it validates those two. Because rampTime does not shrink with the move, the peak approaches TWICE the average as the duration approaches 2 x rampTime, and exceeds twice below that. A move whose average is comfortably inside your limit can therefore still be rejected. Check the peak, not the average, whenever you are near a limit.

• Which error you get: for a too-short 'Trapezoid move', 'Go to position' or 'Homing', the rejection is fatal error 15 (ERROR_ACCEL_TOO_HIGH), not 16 or 28. The planner's derived acceleration is checked first and is always reached at or before the velocity limit, so the acceleration check is what fires. Error 28 (ERROR_PREDICTED_VELOCITY_TOO_HIGH) belongs to the acceleration-type commands — 'Move with acceleration' and the acceleration segments of 'Multimove' — where a legal acceleration applied for too long would end above the velocity limit. Error 16 (ERROR_VEL_TOO_HIGH) belongs to the velocity-type commands, and is additionally re-checked every control tick while moves execute.

• Worked example, if the shape is easier to see with numbers: with a 5 rot/s velocity limit and a 1000 rot/s^2 acceleration limit, rampTime is 0.005 s. A move of 0.04 rotations over 0.01 s averages 4 rot/s — apparently fine — but the duration is exactly 2 x rampTime, so the profile degenerates to a pure triangle with no coast phase at all, peaks at 8 rot/s and needs 1600 rot/s^2. It is rejected with fatal error 15. (Below 2 x rampTime there is no coast segment, which is also why such a move occupies 2 of the 32 queue slots rather than the usual 3.)

• Pre-validate trapezoid durations in closed form instead of try/except: the minimum accepted duration is displacement/maxVelocity + maxVelocity/maxAcceleration (bench-verified to 4 decimal places; shorter durations reject with fatal error 15). One refinement to be aware of: the firmware truncates rampTime down to a whole timestep, so the velocity actually reached at the top of the ramp can be slightly below maxVelocity, which makes that formula very slightly optimistic — at the M17 boot defaults the ramp reaches 9.216 rather than 9.333 rot/s, and the formula under-estimates the minimum duration by about 1.3% of its displacement/maxVelocity term. Add a few percent of margin when pre-validating rather than submitting the computed minimum exactly. The velocity and acceleration limits are INCLUSIVE — a move at exactly the limit is accepted; 0.2% over rejects.

• A single 'Go to position' move is limited to a displacement of about +/-655 shaft rotations (a signed 32-bit count range); split longer travels.

• Timed dwell inside a motion sequence: queue trapezoid_move(0, t) as an in-queue pause — the whole choreography (move, dwell, move) then runs from the queue without host timing.

• Streaming motion (continuous paths): keep feeding segments and never let the queue empty mid-path. Two proven patterns: (a) low-latency ramping — keep only ~3 segments buffered, blocking while get_n_queued_items() >= 3; (b) throughput streaming — count segments locally and only query the queue when your local estimate approaches 32. Perform side-channel traffic (time sync, telemetry) only when at least 3 segments are buffered, at most one side-channel transaction per segment queued.

• Multi-axis coordination: send the same segment duration to every motor back-to-back so segment boundaries stay aligned; wait for all queues to empty between phases; sequence potentially colliding axes (e.g. retract Z before long XY traverses).

• For long random/burn-in motion on limited-travel rigs, bias each random move to pull the cumulative displacement back toward zero.

Closed loop, calibration, and homing

• Motors are calibrated at the FACTORY, and the calibration persists in flash. A new motor works in closed loop out of the box — you normally never need to run 'Start calibration'. Recalibrate only if closed-loop control misbehaves or after hardware service.

• If you do calibrate: requirements are a shaft completely free to rotate (remove all loads), empty queue, no test mode active, and the device in open-loop mode (i.e. freshly reset — which also leaves the MOSFETs disabled; calibration enables them itself). The motor spins about 1.5 turns back and forth for roughly 20-60 s (product-dependent; about 20 s measured on an M17).

• Calibration lifecycle: the success response only means calibration STARTED. Keep the bus COMPLETELY QUIET while it runs — polling during calibration can disturb the measurement and reduce its accuracy, and when calibration finishes the firmware saves to flash and AUTOMATICALLY REBOOTS, so a poll landing in the post-reboot bootloader window pins the device in the bootloader. Instead: wait a generous fixed time (60 s covers all products), then send 'System reset', wait the post-reset delay, and verify a clean baseline with 'Get status' (application mode, no fatal error, MOSFETs disabled).

• Entering closed loop: just call go_to_closed_loop() — it enables the MOSFETs by itself (skipping a separate 'Enable MOSFETs' may even give a gentler engagement), optionally after setting PID constants and motor current. Poll 'Get status' until the closed-loop bit (bit 2) sets, with a ~6 s timeout; if it never sets, the motor probably needs calibration.

• ZERO BEFORE ENTERING CLOSED LOOP, and understand why. On M17, M2 and M23, 'Go to closed loop' does NOT synchronise the measured position to the commanded one and does not clear the PID state — it only enables the MOSFETs, reloads the commutation offset, and switches the mode. (Only the M1 code path re-aligns; its source comment even says it is "so that the motor does not move when we go into closed loop mode".) So if a position error is standing when the loop closes, the PID acts on it on the very next 32-microsecond tick and drives it out at full authority. Any error beyond about 52,000 counts (0.016 rotation, 5.8 degrees) at the default gains saturates the controller, which then applies a full 90-electrical-degree commutation lead at the configured maximum motor current until the error is gone.

• CRITICALLY: that correction is NOT speed limited. 'Set maximum velocity' governs the motion planner — it is checked when a move is queued and against the planner's own velocity every tick — but the closed-loop correction has no planned velocity to clamp; in closed loop the commutation angle is slaved to the rotor, so the loop is a torque command, not a speed command. Nothing in the firmware bounds how fast the shaft moves while it closes a standing error. Bench-measured on one M17: a 0.49-rotation standing error present when the loop closed was driven out at about 8 rotations/second average with no acceleration ramp under those test conditions. On a real mechanism that is a safety concern, not just a surprise.

• The fix is simply to call zero_position() BEFORE go_to_closed_loop(). Zeroing atomically sets the commanded position, the measured hall position, the velocity and the PID state (integral term, previous error, filtered error change) all to zero, so there is nothing left to correct. Its only precondition is an empty queue. Note the ordering difference between the two engagement paths: with a bare 'Enable MOSFETs' you enable first, wait ~0.3 s for the commutation snap to settle, and zero after (golden rule 3); with go_to_closed_loop() there is no such snap to wait out, so zero first and enter closed loop second.

• To see the error before you act on it, read 'Get comprehensive position': the difference between the commanded and hall-sensor fields IS what the PID will act on. To make an unexpected lurch fail safe instead of fast, arm 'Set max allowable position deviation' — the deviation check is unconditional on control mode, so a limit tighter than the standing error trips fatal error 45 and removes power instead of allowing the correction. Lowering 'Set maximum motor current' before entering reduces the FORCE of the correction but not its top speed, because nothing in the loop limits speed.

• Nothing observable tells you a correction is in progress: no status bit reflects it, and the queue stays at 0 items throughout, so 'Get n queued items' returning 0 does not mean the shaft is stationary.

• Homing drives the motor until it detects a collision by following-error (threshold: 50,000 counts between commanded and measured position), then shifts the commanded position 50,000 counts back toward the measured position and holds there, MOSFETs still enabled. Requirements: closed-loop mode first (otherwise fatal error 13), empty queue (otherwise fatal error 8).

• ALWAYS CALL 'Zero position' IMMEDIATELY BEFORE 'Homing'. This is the single easiest way to make homing fail silently. The 50,000-count collision threshold is only 0.0153 shaft rotations — about 5.5 degrees on M17/M23 — and it is tested on the very FIRST 32-microsecond control tick after homing starts, before the homing move has advanced at all. So any position error already standing when you call homing is read as an instant hard stop: homing aborts, the success response you already received says nothing, the homing (bit 4) and busy (bit 6) status flags are set and cleared again within one tick so your first status poll sees them already clear, and no error is raised. The shaft performs none of the homing travel. There is no flag and no error code anywhere that distinguishes this from a real homing.

• A standing error that large is easy to arrive at, and the standard homing recipe below actively encourages it: neither 'Enable MOSFETs' nor 'Go to closed loop' re-aligns the commanded position to the measured one on M17/M23, the hall position keeps tracking the shaft even while the MOSFETs are off (so anything that moves the shaft by hand, by gravity or by back-driving leaves an offset), and a move that ended with following error leaves one too. At a deliberately low motor current the unloaded motor can park with a PERMANENT ~0.6-rotation standing error and no error flag — about 39 times the collision threshold. Reducing the current

for a soft approach and then homing is therefore exactly the sequence that walks into this trap. Zero between the two.

• Verify the homing afterwards, because the status flags cannot: compare the distance actually travelled against the maxDistance you asked for. Much less means a hard stop was found; the full amount means none was. Travelled almost nothing (well under 50,000 counts) means the false-collision trap, not a hard stop.

• Homing recipe: REDUCE the motor current first ('Set maximum motor current' to a gentle value, e.g. 50-100 internal units versus the ~200 working default) so the motor presses softly into the hard stop; then call zero_position() so no standing error remains; give a signed maxDistance larger than the full travel (sign = direction) and a generous maxDuration; poll 'Get status' bit 4 (homing) until clear, or poll the queue to empty, budgeting maxDuration plus ~2 s; compare the travelled distance against maxDistance to confirm a stop was actually found. Then restore the working current, and call zero_position() again to establish your origin — homing itself does NOT zero the position or set any limits.

• One boundary worth knowing: the silent-failure window is a standing error between 50,001 counts and the max allowable position deviation (default 2 shaft rotations = 6,553,600 counts). Above that limit you get fatal error 45 instead of a silent no-op, which is at least visible.

• The homing APPROACH SPEED is approximately maxDistance/maxDuration — the firmware paces the internal move across the full maxDuration (bench: 2 rotations with a 5 s budget crawled at ~0.4 rot/s). A generous maxDuration is not just a timeout, it directly makes the approach gentler; conversely a short duration rams the stop fast. During homing both bit 4 (homing) and bit 6 (busy) are set — homing is the one common operation where the busy bit is actually observable. Curiosity: homing with maxDistance 0 is accepted and simply holds the homing/busy state for the full maxDuration without moving; maxDuration 0 rejects with fatal error 34.

• If homing never hits an obstacle it travels the full maxDistance and stops — indistinguishable from a collision by status alone; compare positions to tell.

• For repeatable origins, home 2-3 times and average; back off ("relieve") 10-50 degrees between runs.

• After homing and zeroing, set 'Set safety limits' and/or 'Set max allowable position deviation' so later collisions fault out safely instead of grinding.

• PID constants ('Set PID constants', u32 P, I, D) apply immediately and reset to firmware defaults on reset. Firmware 0.16.0.0 adds 'Get PID constants' (48): no input, three raw u32 outputs in P/I/D order, without changing controller or motion state. The getter returns the last requested values, including the full u32 range; working gains are capped at INT32_MAX, I can be limited further for overflow safety, and D is scaled/quantized for averaging. Known-good M17 values (also the firmware defaults): P=2000, I=5, D=175000. GOTCHA: D values below 32 are quantized to zero derivative action.

• 'Set max allowable position deviation' continuously watches how far the measured position has drifted from the commanded position; if the difference exceeds the limit, fatal error 45 trips (asynchronously — the offending move itself returns success). The default is 2 shaft rotations. The parameter is a signed 64-bit value on the wire and the firmware takes its absolute value, so a negative input acts as its positive equivalent. The deviation check stays armed even with MOSFETs disabled: back-driving a disabled motor more than the limit trips error 45.

• GOTCHA: 'Disable MOSFETs' does NOT stop or clear queued motion — the commanded position keeps advancing invisibly while the rotor stands still, and the deviation check then trips fatal error 45. Stop motion (queue empty or 'Emergency stop') before or together with disabling.

• Temporary torque release in closed loop: 'Disable MOSFETs' keeps the closed-loop mode bit, and a later 'Enable MOSFETs' resumes closed-loop control directly — no second 'Go to closed loop' needed — with a gentle re-engagement (bench-measured ~540 counts ≈ 0.06 degrees, versus ~4 degrees for a cold open-loop enable). CAVEAT: the commanded position is still held from before the disable, so if the shaft moved while unpowered (gravity, hand), the PID yanks it back on re-enable and can trip the deviation limit — re-zero or re-command first if the shaft may have moved.

Reading state

• get_status() returns [statusFlags, fatalErrorCode]. Bits: 0 = in bootloader (if set, all other bits are invalid/zero), 1 = MOSFETs enabled, 2 = closed-loop mode, 3 = calibrating, 4 = homing, 5 = go-to-closed-loop in progress (M1 product only), 6 = busy. A clean post-reset baseline is flags with bits 0 and 1 clear and fatalErrorCode 0. GOTCHA: no bit reflects ordinary queued motion — bit 6 (busy) stays 0 during trapezoid/velocity moves (verified by polling at 20 Hz through a move; it covers long-running tasks like

calibration and homing) — so detect motion via 'Get n queued items' plus position-settled, per golden rule 7. Also note the closed-loop bit (bit 2) SURVIVES 'Disable MOSFETs' (drivers off, mode retained) but is CLEARED by any fatal error. In the fatal-error state the flags all read 0 as of firmware 0.15.4.0 (the MOSFETs are off and nothing is running); in older firmware they were frozen at stale pre-error values, so only fatalErrorCode was trustworthy there.

• Return shapes from M3 methods: single-output commands return a bare scalar; multi-output commands return a flat list; broadcast returns []. Version numbers arrive least-significant-first: [development, patch, minor, major] — reverse for display.

• 'Get position' (commanded) vs 'Get hall sensor position' (measured): compare them to detect stalls or missed motion; at rest expect the hall reading within a few hundred counts of the commanded one. 'Get comprehensive position' returns all three (commanded, hall, external encoder) in one round trip and is safe to poll at 20 Hz. The external encoder field is a raw count from an optional external quadrature encoder — it reads 0 when none is fitted, is never zeroed by 'Zero position', and is NOT in motor position units. GOTCHA: the library nonetheless runs it through the motor's position unit conversion like the other two fields (the default shaft_rotations divides it by 3,276,800); to see the raw count, select the encoder-counts position unit, use the raw variant of the call, or multiply back by the position conversion factor.

• 'Get max PID error' returns [min, max] and RESETS the window on every read. min > max (the sentinels 2147483647 / -2147483648) means "no data since last read" — it only accumulates in closed loop (and in closed loop the window is never empty, since the PID runs every tick). Because the library converts the values into your position unit, the sentinels come out looking like plausible numbers — [+655.36, -655.36] in shaft_rotations — so detect them by min > max, never by magnitude. After entering closed loop, read it once and discard, then read again after your move to measure tracking quality. The values come back converted into your selected POSITION unit (easy to misread as raw counts). Typical unloaded figures: idle holding dither about +/-500 counts (+/-0.05 degrees); worst tracking error during a brisk 2-rotation/1-second trapezoid about 5000 counts (0.55 degrees).

• 'Get debug values' also resets some of its fields on read (profiler max-times, hall-delta stats) — values are per-read-window, not cumulative; the hall-delta average is meaningless when no samples accumulated since the last read.

• 'Get temperature' reads a sensor on the driver PCB; the conversion table covers roughly 33-307 C and out-of-range readings clamp to 0 (so 0 means "below ~33 C", not freezing). A working motor under load should show the value climbing.

• 'Get supply voltage' returns the bus voltage (internally in tenths of a volt); allow ~0.2 s after reset before the first read for the ADC to settle. Expect your PSU voltage within a few percent.

• 'Read multipurpose buffer' has read-once semantics: a successful read CLEARS the buffer. Reading an EMPTY buffer returns a single byte of 0 (the data-type tag for "nothing stored") as of firmware 0.15.4.0; older firmware sent no response at all, so the read timed out and the timeout meant "buffer empty".

Communication robustness

• CRC32 is enabled by default (and after every reset). Leave it on. With CRC enabled, a packet sent without a CRC (or corrupted) is silently DROPPED — the symptom is a TimeoutError, not an error reply. If a device stops answering right after you toggled CRC state, your framing no longer matches its setting; recover by sending 'System reset' first without a CRC and, if that times out, again with one. On a MULTI-DROP bus, never mix CRC modes at all: a device switched to no-CRC treats the 4 CRC bytes of every CRC'd frame it hears — including broadcasts meant for the whole fleet — as excess payload and latches fatal error 51 (bench-verified on a 39-motor bus: one mode-mismatched motor faulted on the first CRC'd broadcast while the other 38 diverged silently). Also never use alias-addressed queries when aliases are duplicated (all holders answer at once — pure collision garbage; bench-verified with 39 same-alias responders); address by unique ID instead.

• The M3 object controls its outgoing CRC32 via motor.set_crc32_enabled(True/False) — after sending 'CRC32 control' 0 to the device, call motor.set_crc32_enabled(False) and the high-level methods keep working. (Older library versions had no such control — the wrapper always appended a CRC32, and CRC-disabled operation required servomotor.communication.execute_command(..., crc32_enabled=False).) One command still needs execute_command instead of the M3 wrapper: 'Multimove' (mixed-unit move list in raw internal units).

• 'Get communication statistics' returns six u32 counters in this order: [crc32_errors, packet_decode_errors, first_bit_errors, framing_errors, overrun_errors, noise_errors]; the input flag (1) resets them after reading.

These count silently-dropped garbage — poll them to monitor bus health (they should stay at 0 on a healthy bus).

• If you suspect you sent a garbled or partial packet, keep the bus idle for at least 100 ms — the firmware's receiver resynchronizes after 100 ms of silence (bench-measured threshold: 95 ms of delay was not enough, 105 ms was — the documented figure is exact). Distinguish the failure modes: a CORRUPTED packet (bad CRC) is dropped and self-clears — the very next command works immediately, no wait needed; only a TRUNCATED/incomplete frame jams the parser and needs the 100 ms silence. Each malformation feeds a specific 'Get communication statistics' counter (LSB-clear first byte -> firstBitErrorCount, bad CRC -> crc32ErrorCount, bad declared size -> packetDecodeErrorCount).

• The protocol's CRC32 is the standard CRC-32 (zlib/PNG polynomial, little-endian on the wire) — verified against Python's zlib.crc32. Useful when implementing the protocol on a new platform.

• servomotor.detect_devices_iteratively() flushes the receive buffer itself (before each round and before returning). If you drive 'Detect devices' at a lower level, or suspect stray bytes in the host receive buffer for any other reason, call servomotor.flush_receive_buffer() before normal traffic resumes.

• Verify the link before trusting motion: ping the device 10-100 times with random 10-byte payloads and require exact echoes. 'Ping' requires exactly 10 bytes.

• Under normal conditions reads do not fail — bench validation ran hundreds of commands with zero communication failures. A few long-running stress tests wrap status reads in a retry loop (their comments call the reads "unreliable" after heavy motion), but a dedicated stress investigation could not reproduce any failure on current firmware/library (49,360 reads across 729 aggressive motion segments, zero failures, all six device-side communication counters at 0) — those comments predate current firmware/library revisions and are not a latent bug. If you ever do see read failures, the first diagnostic is 'Get communication statistics' (then check wiring, grounding, CRC state) rather than routinely retrying them away.

• NEVER transmit a second command before the previous command's reply has fully arrived (or timed out). The bus is half duplex: a device starts replying within microseconds, so a back-to-back second request collides with that reply on the shared wire pair and BOTH are destroyed (bench-verified: two valid packets in one write produced only collision garbage; the same two packets a few milliseconds apart worked perfectly). The library's request/reply methods already enforce this — the rule matters when writing raw bytes or broadcasting rapid-fire sequences.

• Only one process can own the serial port; close it before letting another tool open the same device.

Error handling and recovery

Every code below is a LATCHING fatal error: the motor disables itself, the red LED blinks the code, and every subsequent command except 'Get status' and 'System reset' comes back with that same code. Only 'System reset' clears it — 'Emergency stop' does not (and is itself not a fatal error). After the reset the device is back to its power-on defaults, so every recovery ends with "re-establish your settings". The Error Codes section at the end of this document lists all of them with full causes and solutions; this table is just the short list of the ones customers actually hit, described in terms of what you did wrong rather than what the firmware detected.

Code	What you most likely did	What to do
45 position deviation too large	Disabled the MOSFETs while the queue was still running (the commanded position keeps advancing while the shaft stands still); the shaft was back-driven or hand-turned; the motor current is too low for the load; or you commanded a speed the motor cannot physically reach	Stop motion before disabling the MOSFETs; check the load and raise 'Set maximum motor current'; slow down. The check is armed even while the MOSFETs are off.
18 run out of queue items	A velocity or acceleration move sequence ended with the motor still at nonzero velocity, or a streaming path let the queue run dry. A velocity move does NOT stop by itself	Always end such a sequence with a segment that brings velocity to zero, queued back-to-back with the moving segments; when streaming, keep the queue fed

Code	What you most likely did	What to do
15 accel too high	A 'Trapezoid move', 'Go to position' or 'Homing' duration too short for the distance. Note this is an ACCELERATION error even though it feels like a speed problem — see the peak-versus-average discussion above	Lengthen the duration (minimum is roughly displacement/maxVelocity + maxVelocity/maxAcceleration, plus a few percent) or raise 'Set maximum acceleration'
16 vel too high	A 'Move with velocity' (or a Multimove velocity segment) above 'Set maximum velocity'. Also fires DURING execution if you lower the velocity limit while moves are running	Reduce the velocity or raise the limit; change limits only while stopped with an empty queue
28 predicted velocity too high	A 'Move with acceleration' (or a Multimove acceleration segment) whose acceleration is legal but is applied for long enough to end above the velocity limit	Use a smaller acceleration, a shorter duration, or raise the velocity limit
8 queue not empty	'Zero position', 'Go to closed loop', 'Homing' or 'Start calibration' sent while moves were still queued. These four are the only commands with this requirement	Poll 'Get n queued items' to 0 first, or clear the queue with 'Emergency stop' (also disables the MOSFETs) or 'Reset time' (leaves them on)
17 queue is full	More than 32 queued items. A trapezoid move or go-to-position normally takes 3 slots, so about 10 of them fit; velocity and acceleration moves take 1 each	Pace with 'Get n queued items'. Note the fault aborts the whole in-flight sequence, it does not merely reject the extra item
34 parameter out of range	A duration of 0 on any move, a maxDuration of 0 on homing, a 0 passed to 'Set maximum velocity' or 'Set maximum acceleration', or 'Set safety limits' with lower greater than upper	Validate before sending. Watch for a small duration rounding down to 0 during unit conversion
13 not in closed loop	'Homing' in open loop. Homing is the ONLY command that requires closed loop; everything else works in open loop	'Go to closed loop' first and let it complete, then home
19 motor busy	A move-queueing command, 'Go to closed loop' or 'Start calibration' sent while a calibration or a homing was still running. It does NOT mean "still moving" — ordinary queued motion never sets the busy state	Wait for status bit 6 (busy) to clear. During calibration, do not poll at all: wait a fixed time, then 'System reset'
23 max pwm voltage too high	'Set maximum motor current' given a motorCurrent or regenerationCurrent above the product ceiling — 390 on M17. The same ceiling applies to both parameters	Range-check both values. Remember they are internal current units, not amps
40 overheat	Sustained high torque, poor airflow, or a stall. Monitored only while the MOSFETs are enabled — which is why leaving a machine energised and idle for hours can trip it	Let it cool, then reset. Reduce load, duty cycle or current limit; improve heat sinking; disable the MOSFETs between work bursts
14 overvoltage	Regenerated energy from decelerating an inertial load, or the shaft spun by hand while powered, pushed the supply rail up	Brake more gently, add bulk capacitance or use a supply that can absorb current, do not spin the shaft forcefully while powered
51 command size wrong	A payload that does not match the command's expected size. The realistic field cause is a multi-drop bus with mixed CRC32 settings: a device switched to no-CRC reads the 4 CRC bytes of every CRC'd frame it hears as excess payload	Never mix CRC modes on one bus; keep CRC32 enabled everywhere
50 bad alias	'Set device alias' with 252, 253 or 254, which the protocol reserves	Use 0-251, or 255 for "no alias assigned"

Two codes are internal-consistency faults rather than user errors: 30 (control loop took too long) and 42 (position discrepancy). If you see either, report it to the manufacturer along with your firmware version.

• Exception types from the library: TimeoutError (no response — wrong address/port, device dead, in bootloader, CRC mismatch; on firmware before 0.15.4.0 also an empty multipurpose-buffer read), FatalError (the device answered with a nonzero error code; e.args[0] is the code from the error reference in this document), CommunicationError (malformed/corrupt response), PayloadError (defined and exported by the package but not raised by any current code path), NoAliasOrUniqueIdSet. All of these are the library's own exception classes (plain Exception subclasses) — in particular servomotor.TimeoutError is NOT Python's built-in TimeoutError, so catch servomotor.TimeoutError (or import it), not the builtin; a bare `except TimeoutError:` will not catch it.

• A rejected command can surface two ways depending on timing: the call itself raises FatalError, OR the call returns normally and the error appears shortly afterwards in get_status()[1] (asynchronous checks like position deviation always do the latter). Robust code checks both: catch FatalError, and after risky operations sleep ~0.2 s and poll get_status().

• GOTCHA (firmware up to 0.15.4.0; fixed in 0.15.5.0): a fatal error that fires in the fraction of a millisecond while a command's reply is still transmitting can corrupt that exchange — the reply may be truncated (host sees TimeoutError), or an extra unsolicited error packet may arrive and shift every later reply by one (host sees FatalError raised against the wrong command). If a state-changing command times out or errors unexpectedly, call servomotor.flush_receive_buffer(), then probe with 'Get status'. In the worst timing (observed on the bench with inverted safety limits), the device can lock up completely — silent to every command including 'Get status' and 'System reset' — and only a power cycle recovers it. Firmware 0.15.5.0 closes all of this: an in-flight reply is completed before the fatal-error state takes over, the duplicate error packet is suppressed, and the hardware-verified result is clean error delivery even with a fatal firing during heavy reply traffic.

• Recovery recipe from any fatal error (this is golden rule 5 applied, plus two follow-up steps): read and note the error code ('Get status' or the FatalError exception); system_reset(); wait the post-reset delay (per golden rule 1: 0.5 s or more of bus silence); then verify get_status() == clean baseline; and re-establish your settings (units are host-side and survive, but limits, gains, zeroed position, and enabled state are gone).

• 'Emergency stop' is NOT a fatal error: it disables the MOSFETs and clears the queue, and you can resume with enable_mosfets() without any reset. (Firmware before 0.15.4.0 had a bug here: a mid-motion 'Emergency stop' or 'Reset time' left an internal planner variable stale, making later acceleration-type moves validate incorrectly — on old firmware, send 'System reset' after a mid-motion stop before queueing new moves. Fixed in 0.15.4.0.)

• GOTCHA: 'Reset time' also clears the motion queue and halts motion instantly (it shares the stop path with 'Emergency stop' but leaves MOSFETs enabled). Do not send it mid-motion unless you intend an abrupt stop.

• On KeyboardInterrupt during motion: emergency_stop(), then disable_mosfets().

• GOTCHA: several library input errors (wrong argument count, out-of-range integer for the wire type, unknown command, malformed alias) call exit(1) instead of raising — a long-running application should validate values before calling the library.

• A watchdog for unattended loops: put a hard time limit on any loop that commands motion, so a logic bug cannot run the motor forever.

Upgrading the firmware

Firmware upgrades go over the same RS485 bus as everything else, using the Python tool. The upgrade protocol itself has no Arduino-side implementation — but that does NOT mean a motor wired to an Arduino or ESP32 is unreachable. If the motor's only RS485 connection is to a microcontroller, flash that board with a TRANSPARENT USB-to-RS485 BRIDGE and run the ordinary Python tool through it. A working bridge is in this repository at `Arduino_library/ESP32S3_RS485_Bridge/`; it is about twenty lines and simply relays bytes both ways. This is strongly preferable to reimplementing the upgrade in C++, because the protocol stays in the tested Python and because the same bridge then gives every other host tool access to that motor. Bench-verified: an M17 whose only link was an ESP32-S3 was upgraded 0.15.9.0 -> 0.15.12.0 through such a bridge, and the whole 111-module test suite was then run against it. If you write your own bridge, read the two gotchas at the end of this section first — both cost a full test run to find.

• Get the tool: `pip3 install --upgrade servomotor`. Since library version 0.12.0 this installs an `upgrade_firmware` command directly, along with `servomotor_command`, `detect_and_set_alias_all_devices` and `show_device_information_for_all_devices`. No

repository checkout is needed. Do not install pyserial — a copy is bundled inside the package and is what the library actually imports.

• Use library version 0.12.2 or later. ON WINDOWS, 0.12.0 and every earlier version that bundled pyserial could not open a serial port at all — and the failure is easy to misdiagnose, because `import servomotor` SUCCEEDS. A working import is therefore not evidence that your version is good. The error appears only when you actually open a port, as `ModuleNotFoundError: No module named 'serial'` raised from `from serial import win32` (the bundled pyserial's Windows backend used absolute imports; fixed in 0.12.1). Separately, versions before 0.12.1 aborted with an uncaught PermissionError when they could not save the remembered serial port into the installed-package directory — which a normal system-wide install hit every time. 0.12.2 moves that file to a per-user config directory and adds the SERVOMOTOR_PORT environment variable, so you can name the port once instead of passing -p to every command below.

• The `.firmware` image files are NOT part of the pip package; obtain them separately.

• CHECK COMPATIBILITY FIRST. Run `show_device_information_for_all_devices -p <PORT>` and note the Product Code and Firmware Compatibility Code. The file name encodes both: `servomotor_M17_fw0.15.9.0_scc3_hw1.5.firmware` means model `M17` and compatibility code 3. Both must match your device exactly.

• A MISMATCH IS SILENT. The check happens in the device's bootloader, which simply ignores any page whose codes do not match and says nothing on the bus; the explanatory message goes only to an internal debug port you cannot see. The tool has no way to notice.

• ADDRESS ONE MOTOR: `upgrade_firmware -p <PORT> -a <ALIAS> <file.firmware>`. The tool's DEFAULT is the broadcast address 255, which flashes every matching device on the bus simultaneously but receives no replies at all — so in broadcast mode it prints "Firmware page N written successfully" for every page and exits successfully even when absolutely nothing was written. Use `-a <ALIAS>` so a problem shows up as a timeout rather than as a false success. (If a device still has no alias, assign one first with `detect_and_set_alias_all_devices`.)

• Be aware that the tool's first action is a BROADCAST 'System reset', so every motor on the bus reboots into its bootloader for the duration of the transfer, not just the one you are flashing. Do not upgrade a bus that is mid-motion or safety-critical.

• VERIFY AFTERWARDS — the tool does not. Run `servomotor_command -p <PORT> -a <ALIAS> get_firmware_version` and confirm the new version with inBootloader = 0. (Note: `show_device_information_for_all_devices` fails with an assertion error against a device sitting in the bootloader, so use `servomotor_command` for this check.)

• YOUR CALIBRATION AND ALIAS SURVIVE. An upgrade only rewrites the application flash pages. The device alias, the three hall-sensor midlines, the commutation position offset and the motor-phases-reversed flag live in a separate settings page that the burn routine refuses to write. You do not need to recalibrate after an upgrade.

• AN INTERRUPTED UPGRADE CANNOT BRICK THE MOTOR. The bootloader lives in pages the upgrade also refuses to write, and it verifies the application by CRC32 on every boot. A half-written image simply fails that check, so the device stays in the bootloader, still answering RS485 — recognisable by a fast-blinking green LED (about 10 Hz, versus the brief once-per-second blip of the running application) and by 'Get status' bit 0 being set. Recovery is to run the same `upgrade_firmware` command again.

Two ways to fault a motor with a setting, not a move

Both of these are commands that are perfectly legal, return normally in most circumstances, and nonetheless take the machine down. Neither is documented anywhere else.

Lowering the speed limit while a move is running faults that move. The obvious way to slow a machine down is to reduce 'Set maximum velocity'. Do that mid-move and you get fatal error 16, an abrupt uncommanded stop, and a latched fault — the opposite of the intended effect. The cause is that the limit is enforced in two places with different scopes: a move is validated against the limits when it is QUEUED, but the control loop also re-checks the LIVE velocity against the CURRENT limit on every one of its 31,250 ticks per second. Lowering the ceiling below the speed a move is already travelling at therefore faults a move that was legal when queued. Bench-measured: dropping the ceiling from 5 to 2 rot/s during a move that peaks near 0.5 rot/s is harmless; dropping it to 0.2 rot/s faults immediately with code 16. **To slow a machine down, stop it first, or only ever RAISE the limit mid-flight.**

Setting a safety fence that excludes the current position usually loses its own reply. A fence with lower < upper that simply does not contain where the motor currently is, is accepted and correctly faults with error 25 within one control tick. But the fence goes live before the command's reply is sent, and the fault fires from the control loop, so the reply is frequently never transmitted. Bench-measured over 30 trials: **10 of 30 lost the reply (33%)**, versus 0 of 30 for a fence that includes the current position. The fault itself is reliable — code 25 was raised in 29 of 30.

So a timeout from 'Set safety limits' does NOT mean the motor is gone. Treat it as "check status": read 'Get status' and you will find error 25. Note that the firmware already guards the closely related case — an INVERTED fence (lower > upper) is rejected cleanly with error 34 before the limits are applied, precisely to avoid this race — so only the excludes-current-position case behaves this way.

Diagnosing a faulted motor: you get the error number and nothing else

Golden rule 5 says every command except 'Get status' and 'System reset' fails once a fault latches. The consequence is worth stating separately, because it shapes how you should write monitoring code: **no telemetry survives a fault.** Temperature, supply voltage, position, the device clock, even 'Get product info' and 'Ping' all return the latched error code instead of their data. The single number from 'Get status' is the entire diagnostic surface.

That is a sound fail-safe — a faulted machine should refuse to act — but it means the readings you would want in order to work out WHY a motor stopped (did it overheat? did the supply sag? where was it?) are unavailable at exactly the moment you need them.

Therefore: log telemetry continuously while things are working, not after they break. Polling is cheap and provably harmless — bench-measured, polling any getter continuously through a move leaves the endpoint bit-identical to an undisturbed reference, and reads consume no queue slots.

If you write your own USB-to-RS485 bridge, two traps

Both produce the same symptom: short exchanges work perfectly, sustained traffic fails intermittently, and the failure looks exactly like a firmware fault under load. Both were found the hard way, by a 111-module suite reporting six modules "failing under load" that were nothing of the kind.

1. Yield to the scheduler when idle. A `loop()` that polls both ports and never blocks starves the RTOS idle task, and on an ESP32 the task watchdog eventually resets the board mid-run. The host then sees a plain read timeout. One `delay(1)` when no bytes moved fixes it; without it a 200-cycle soak died at cycle 96.

2. Do not use `readBytes()` to move the data. It inherits Stream's one-second timeout, so if the driver reports N bytes available but returns fewer, the loop blocks for up to a second in one direction and cannot relay the reply travelling the other way. The host times out on a command the motor answered correctly. Use `read()` in a loop bounded by `available()`, so neither direction can ever stall the other. Fixing this took one test module from 22 passed / 18 failed to 40 passed / 0 failed.

Also: enlarge both receive buffers. USB CDC delivers a 2,067-byte firmware page far faster than a 230,400 baud wire can carry it, and the default CDC receive buffer is only 256 bytes.

Behaviour by firmware version

Read your firmware version before anything else — 'Get firmware version' — and check this list before assuming a behaviour described elsewhere in this document applies to you. Version numbers arrive least-significant-first. Everything below is M17; the current release is 0.15.12.0.

Firmware	What changed, and how you would notice
0.15.3.4	PID overhaul. With an integral gain of 10 or more the motor no longer hunts after a move; the derivative term actually acts (it previously had a sticky offset proportional to kD); holding torque no longer collapses when the maximum motor current is set below about 64; changing PID constants mid-operation no longer produces a derivative kick.

Firmware	What changed, and how you would notice
0.15.4.0	Validation and lock-up hardening. A duration of 0 on any move command, and a value of 0 to 'Set maximum acceleration', now raise fatal error 34 instead of silently succeeding. 'Test mode' 0 now clears test modes instead of hanging the device until a power cycle. Reading an EMPTY multipurpose buffer now replies with a single 0x00 byte instead of staying silent — code that treated a read timeout as "buffer empty" must be changed. 'Get status' in the fatal state now reports all flags 0 instead of stale pre-error values. A 'Multimove' with more than 32 moves gives a clean fatal error 24. After a mid-motion 'Emergency stop' or 'Reset time' you can queue new moves directly, with no intervening reset.
0.15.5.0	Fatal-error/reply race fixed. A fatal error firing while a reply is transmitting no longer truncates that reply, no longer emits an extra unsolicited error packet that shifts every later reply by one, and no longer hard-hangs the device (which previously needed a power cycle). 'System reset' now always works in the fatal state. 'Set safety limits' with lower greater than upper is rejected cleanly with fatal error 34 instead of faulting one control tick later.
0.15.6.0	'Set maximum velocity' with 0 is rejected with fatal error 34. Previously a zero limit was accepted and every trapezoid/go-to-position was then silently planned as a zero-motion dwell — success reply, queue occupied for the full duration, shaft never moved.
0.15.7.0	Boot-default motion limits corrected: maximum velocity 68 to 34 rot/s, maximum acceleration 2,000 to 500 rot/s^2. Also new in this release: 'Set maximum velocity' and 'Set maximum acceleration' can RAISE the limit above the boot default at all — earlier firmware clamped every request at the boot default itself.
0.15.8.0	Boot defaults aligned to the product specification: maximum velocity 560 RPM (9.3333 rot/s), maximum acceleration 12,000 rot/s^2. THIS IS THE MOST LIKELY MIGRATION HAZARD. Relative to 0.15.6.0 and earlier the default velocity limit is about 7.3x LOWER (68 to 9.3333 rot/s) while the default acceleration limit is 6x HIGHER (2,000 to 12,000 rot/s^2). Any program written against older firmware that commands faster than 560 RPM now gets a fatal error at queue time and needs one explicit 'Set maximum velocity' call. The maximum SETTABLE velocity also drops, to a firmware clamp of 18.67 rot/s — requests above it are silently clamped and still reply success.
0.15.9.0	'Set max allowable position deviation' with the extreme negative wire value now saturates to the maximum limit instead of storing a negative limit that latched fatal error 45 on the next control tick. That landmine existed in 0.15.4.0 through 0.15.8.0.

Firmware	What changed, and how you would notice
0.15.12.0	Trapezoid-move planner fixes, all three customer-visible. (a) A move is no longer SILENTLY IGNORED when the acceleration limit is very high relative to the speed limit. Setting a speed limit below about 0.384 rotations/second while leaving the 12,000 rotations/second^2 factory-default acceleration — an ordinary slow, precise axis — used to make every 'Trapezoid move' and 'Go to position' return success and never move the shaft. They now move correctly. (b) A move that is far too aggressive is no longer silently WRONG. The planner's acceleration is now saturated rather than truncated into a 32-bit field, so instead of moving an arbitrary distance (measured: 8,191 counts where 3,276,800 were asked for) or not at all, the move is rejected with fatal error 15. (c) When the acceleration ramp works out shorter than one 32-microsecond control tick, the move is queued as a constant-velocity segment plus a stop instead of a three-segment ramp. That removes an intermittent fatal error 18 that struck roughly one ordinary slow move in five. Note the consequences: some very short moves that used to be rejected are now performed (they were always within the speed limit), and an over-commanded short move may now report fatal error 16 rather than 15.

Boot defaults on the current firmware, for reference: maximum velocity 9.3333 rot/s (560 RPM), maximum acceleration 12,000 rot/s^2, maximum motor and regeneration current 200 internal units (of a 390 ceiling), maximum allowable position deviation 2 shaft rotations, PID constants P=2000 I=5 D=175000. All of these are volatile and return on every reset.

Clock synchronization (multi-motor coordinated motion)

• Purpose: 'Time sync' does not set the motor's clock — it disciplines the clock RATE (trims the internal oscillator) so motors converge on the master's timebase. Establish a common epoch first: broadcast 'Reset time' to alias 255 (all clocks zero simultaneously — remember this also stops motion, so do it before motion starts) and record the host's epoch at that instant.

• Then send time_sync(master_time_since_epoch) to EACH motor individually every 0.1 s (10 Hz), in whatever time unit the M3 object is configured with (the library converts to the command's microsecond wire unit; e.g. with the default 'seconds', pass time.time() - epoch directly). (Older library versions converted this command incorrectly in every unit except 'timesteps' — there, pass raw microseconds with time_unit='timesteps'.) GOTCHA: a broadcast 'Time sync' is a complete no-op in current firmware — each motor must be addressed individually. (Verified both in the firmware source, where the sync action sits inside the not-broadcast branch of the handler, and by bench experiment: 30 broadcast syncs left the oscillator trim untouched, while the same 30 syncs sent addressed shifted the clock rate by about 0.7%.)

• Budget about a minute of syncing before trusting tight synchronization (bench-measured: a -3.4 ms initial offset converged to within ±200 microseconds in ~40 s at 10 Hz). Steady-state error should sit within ~5000 microseconds; occasional single-sample spikes are host-side USB jitter, not drift.

• The sync reply returns the motor's clock error in microseconds (positive = motor behind master) — track it as a health metric.

Performance envelope (what the test programs demonstrate works)

• Queued sequence timing is accurate to about +/-0.1 s over multi-second sequences; execution starts essentially immediately when the first move enters an empty queue. Bench-measured device clock rate offset versus the host: +0.41% (+4100 ppm) fast over a 120 s free-run, consistent with the "well under 1%" spec, so a ~5% wait margin on timed moves is ample.

• Commanded (desired) position readback lands within 50-80 counts of the target in the tests (under 0.003% of a rotation; profile discretization); hall-measured position pass/fail tolerances are much looser: ~500 counts at rest, ~20,000 counts tracking an open-loop move. Closed-loop holding tolerance used for pass/fail: ~4000 counts (0.12% of a rotation). Position reads agree across unit systems to within 1 count (commanded) / ~1000 counts (measured, due to jitter between sequential reads).

• End-of-move accuracy for an identical unloaded 2-rotation/1-second trapezoid (bench, M17): open loop landed 504 counts short (~0.055 degrees — commutation-step quantization), closed loop 37 counts (~0.004

degrees) — about 14x tighter. The position-deviation watchdog is hard to trip with attainable commands on a free shaft (even an instantaneous 8 rot/s velocity step stayed within a 0.2-rotation limit); its practical role is catching stalls, overloads, and unattainable commands.

• Decelerating pumps energy back into the supply: a hard 10-to-0 rot/s stop of just the free rotor produced a +0.4 V blip on a 20 V bench supply (no sag was measurable during the matching acceleration). With significant load inertia and a supply that cannot sink current, regeneration is what pushes the rail toward the overvoltage fatal (error 14) — budget for it or brake more gently.

• Move durations from 0.32 ms (10 timesteps) up to minutes work; the theoretical minimum segment is 1 timestep = 32 microseconds.

• The RS485 link at 230400 baud carries roughly 23 KB/s. Measured command round trip ('Get status' via a USB adapter, 200 samples): median 3.4 ms, 95th percentile ~5 ms, worst outlier 15 ms — and statistically identical while the motor executes a move, so ~290 status reads/second are attainable idle or moving. Rapid enable/disable cycling (50 back-to-back) and continuous pinging at up to ~100 Hz pacing throughout a 70 s move sequence are demonstrated reliable, with zero failures.

• The library's read timeout restarts with every received byte, which is why a multi-second 'Capture hall sensor data' stream completes despite the nominal 1.2 s timeout; a fatal error mid-capture kills the stream instead (the read times out — probe 'Get status' afterward).

• Current setting guidance from working programs: ~150-200 internal units as a general working value; up to 390 for demanding moves; ~10-50 for deliberately compliant/soft behavior (catching at 20, soft homing at 50; the main homing test homes at 200 and uses 10 as the deliberately-too-weak value); the current limit doubles as a programmable force limit. The units map directly to PWM drive duty (bench: open-loop holding PWM telemetry reads exactly setting/2). Two measured cautions: at very low current (20) the unloaded motor still runs but parks with a PERMANENT ~0.6-rotation standing error and NO error flag (too weak to overcome its own friction — check |commanded − measured| after moves if you run soft); and raising current does NOT buy top speed (free-shaft top speed measured mildly LOWER at 390 than at 50-100).

• Closed-loop tuning envelope (bench, free shaft, 1-rot/0.6-s test move): tracking error improves from P=500 to P≈4000 and then degrades sharply — in-motion oscillation starts between P=4000 and 8000, and at P=64000 the motor buzzes even at rest; none of this raises a fatal error (the signature is a large 'Get max PID error' span and audible buzz, not a fault). D values below 32 are confirmed identical to zero, and D only becomes visibly effective around 10^5 (default 175000 is mid-range; ~700k damps a P=8000 oscillation best). The PID arithmetic is exact and inspectable live via Test mode 3: P-term = P x error, output = (P+I+D) >> 11. The integral anti-windup clamp works: even I=500 with seconds of forced windup overshoots by a bounded ~23k counts regardless of windup duration.

Data Types

This section describes the various data types used in the Servomotor API commands.

Integer Data Types

Type	Size (bytes)	Range	Description
i16	2	-32,768 to 32,767	16-bit signed integer
i32	4	-2,147,483,648 to 2,147,483,647	32-bit signed integer
i48	6	-549,755,813,888 to 549,755,813,887	48-bit signed integer
i64	8	-9,223,372,036,854,775,808 to 9,223,372,036,854,775,807	64-bit signed integer
i8	1	-128 to 127	8-bit signed integer
u16	2	0 to 65,535	16-bit unsigned integer
u32	4	0 to 4,294,967,295	32-bit unsigned integer
u48	6	0 to 1,099,511,627,775	48-bit unsigned integer
u64	8	0 to 18,446,744,073,709,551,615	64-bit unsigned integer
u8	1	0 to 255	8-bit unsigned integer

Special Data Types

Type	Size (bytes)	Description
buf10	10	10 byte long buffer containing any binary data
crc32	4	32-bit CRC
firmware_page	2058	This is the data to upgrade one page of flash memory. Contents includes the product model code (8 bytes), firmware compatibility code (1 byte), page number (1 byte), and the page data itself (2048 bytes).
general_data	Variable	This is some data. You will need to look elsewhere at some documentation or into the source code to find out what this data is.
list_2d	Variable	A two dimensional list in a Python style format, for example: [[1, 2], [3, 4]]
string8	8	8 byte long string with null termination if it is shorter than 8 bytes

string_null_term	Variable	This is a string with a variable length and must be null terminated
success_response	Variable	Indicates that the command was received successfully and is being executed. The next command can be immediately transmitted without causing a command overflow situation.
u24_version_number	3	3 byte version number. the order is patch, minor, major
u32_version_number	4	4 byte version number. the order is development number, patch, minor, major
u64_unique_id	8	The unique ID of the device (8-bytes long)
u8_alias	1	This can hold an ASCII character where the value is represented as an ASCII character if it is in the range 33 to 126, otherwise it is represented as a number from 0 to 255
unknown_data	Variable	This is an unknown data type (work in progress; will be corrected and documented later)

Command Reference

This section documents all available commands organized by category.

Basic Control

Disable MOSFETs

Immediately disable the motor driver outputs (MOSFETs); the command executes at once (it is not queued) and is idempotent. Note that the MOSFETs are disabled by default after power-on, and any fatal error also force-disables them. Disabling does NOT stop or clear the movement queue: queued moves keep executing virtually with the outputs off, so the commanded position keeps advancing while the rotor stands still, and because the position-deviation check stays armed, disabling mid-move typically ends in fatal error ERROR_POSITION_DEVIATION_TOO_LARGE once the deviation exceeds the limit (default 2 shaft rotations), requiring a 'System reset' to recover. Movement commands continue to be accepted (with success responses) while the MOSFETs are disabled. Takes no parameters; any payload bytes raise fatal error ERROR_COMMAND_SIZE_WRONG.

Example program:

```
#!/usr/bin/env python3
"""
Example: Disable MOSFETs -- de-energize the motor driver outputs.
Enables the driver, then disables it, and shows the change in the 'Get status'
MOSFETs-enabled bit (bit 1). Once disabled, the shaft turns freely by hand.
"""
import time
import servomotor

ALIAS = 'X'                              # Device alias; change if needed
SERIAL_PORT = "/dev/tty.usbserial-110"  # Change to your serial port (e.g. "COM3" on Windows).
                                         #  Set SERIAL_PORT = "MENU" to be prompted interactively.
servomotor.set_serial_port(SERIAL_PORT)
servomotor.open_serial_port()
motor = servomotor.M3(ALIAS, time_unit="seconds", position_unit="shaft_rotations",
                      velocity_unit="rotations_per_second",
                      acceleration_unit="rotations_per_second_squared", verbose=0)

MOSFETS_ENABLED_BIT = 1 << 1            # bit 1 of the 'Get status' flags

try:
    motor.system_reset()
    time.sleep(1.5)                     # mandatory: keep the bus silent after reset

    motor.enable_mosfets()
    time.sleep(0.3)                     # let the rotor's commutation-snap transient settle
    flags, error_code = motor.get_status()
    print(f"After enable:  MOSFETs-enabled bit = {1 if flags & MOSFETS_ENABLED_BIT else 0} "
          f"(fatal error code {error_code})")

    # WARNING: 'Disable MOSFETs' does NOT stop or clear queued motion. Queued moves keep
    # advancing the commanded position with the outputs off, and the position-deviation
    # check then trips fatal error 45. Only disable when the queue is empty (as here) or
    # together with an emergency stop.
    motor.disable_mosfets()
    flags, error_code = motor.get_status()
    print(f"After disable: MOSFETs-enabled bit = {1 if flags & MOSFETS_ENABLED_BIT else 0} "
          f"(fatal error code {error_code})")
    print("The shaft should now turn freely by hand.")
finally:
    try:
        motor.disable_mosfets()
    except Exception:
        pass
    servomotor.close_serial_port()
```

Enable MOSFETs

Enable the motor driver outputs (MOSFETs). Executes immediately (not queued). On M1, M2, and M23, enabling first drives the phases to a zero-volt state, samples the motor current-sensor baseline, raises fatal error ERROR_CURRENT_SENSOR_FAILED if the baseline is outside the expected window, and calibrates the overcurrent watchdog thresholds from that baseline; M17 performs no current-sensor check. Calling while already enabled is a no-op (the baseline check is skipped) and still returns success. Enabling does not re-align the commanded position to the measured hall position and clears no queue or PID state; instead the rotor mechanically snaps to the angle derived from the commanded position, a settling transient lasting roughly 0.3 seconds (on M17 the external stepper driver additionally slews at most 1 microstep per 32 microsecond control tick). The position-deviation check is armed throughout, so applying tight deviation limits before this transient settles can trip fatal error ERROR_POSITION_DEVIATION_TOO_LARGE; the safe sequence is enable, wait to settle, 'Zero position', tighten limits, then move. Takes no parameters; any payload bytes raise fatal error ERROR_COMMAND_SIZE_WRONG.

Example program:

```
#!/usr/bin/env python3
"""
Example: Enable MOSFETs -- energize the motor driver outputs.
Enables the driver, waits out the mechanical settling transient, and verifies
the change via 'Get status' bit 1. You may feel the rotor snap and then hold.
"""
import time
import servomotor

ALIAS = 'X'                               # Device alias; change if needed
SERIAL_PORT = "/dev/tty.usbserial-110"  # Change to your serial port (e.g. "COM3" on Windows).
                                          #  Set SERIAL_PORT = "MENU" to be prompted interactively.
servomotor.set_serial_port(SERIAL_PORT)
servomotor.open_serial_port()
motor = servomotor.M3(ALIAS, time_unit="seconds", position_unit="shaft_rotations",
                      velocity_unit="rotations_per_second",
                      acceleration_unit="rotations_per_second_squared", verbose=0)

MOSFETS_ENABLED_BIT = 1 << 1            # bit 1 of the 'Get status' flags

try:
    motor.system_reset()
    time.sleep(1.5)                     # mandatory: keep the bus silent after reset

    flags, error_code = motor.get_status()
    print(f"Before enable: MOSFETs-enabled bit = {1 if flags & MOSFETS_ENABLED_BIT else 0}")

    motor.enable_mosfets()
    # The motor is ready to use immediately after enabling; it may twitch or rotate
    # slightly as the rotor snaps to a commutation step. The brief settle here is
    # only so the status read below and any precise zeroing happen after the twitch.
    time.sleep(0.3)

    flags, error_code = motor.get_status()
    print(f"After enable:  MOSFETs-enabled bit = {1 if flags & MOSFETS_ENABLED_BIT else 0} "
          f"(fatal error code {error_code})")
    print("The shaft should now resist being turned by hand.")

    motor.disable_mosfets()             # leave the motor de-energized when done
    print("MOSFETs disabled again; shaft is free.")
finally:
    try:
        motor.disable_mosfets()
    except Exception:
        pass
```

```
    servomotor.close_serial_port()
```

Reset time

Reset the device's absolute microsecond clock to zero and, as a major side effect, clears the entire movement queue and stops the motor instantly: all queued moves are silently discarded, the current velocity is forced to zero immediately (no controlled deceleration), the queue's expected end position is snapped to the current position, and the debug profiler timestamps are reset. Unlike 'Emergency stop', the MOSFETs stay enabled and the motor holds position. Do not send this mid-motion unless an abrupt halt is intended. It executes immediately (not queued) and the success response only confirms the reset occurred. On broadcast the reset is still executed by every device and only the response is suppressed; broadcasting is the intended way to zero all motors' clocks simultaneously before using 'Time sync' or 'Multimove'. Call it before issuing timed movement commands so device time starts from a known epoch. The command takes no payload; any nonzero payload raises fatal error 51, ERROR_COMMAND_SIZE_WRONG, which disables the MOSFETs and halts the device until 'System reset'.

Example program:

```
#!/usr/bin/env python3
"""
Example: Reset time -- zero the device's absolute microsecond clock.
Reads the clock, resets it, then reads again: the second reading is near zero.
WARNING: 'Reset time' also clears the entire motion queue and stops any motion
instantly (no deceleration; MOSFETs stay enabled) -- only send it at rest.
"""
import time
import servomotor

ALIAS = 'X'                             # Device alias; change if needed
SERIAL_PORT = "/dev/tty.usbserial-110"  # Change to your serial port (e.g. "COM3" on Windows).
                                        #  Set SERIAL_PORT = "MENU" to be prompted interactively.
servomotor.set_serial_port(SERIAL_PORT)
servomotor.open_serial_port()
motor = servomotor.M3(ALIAS, time_unit="seconds", position_unit="shaft_rotations",
                      velocity_unit="rotations_per_second",
                      acceleration_unit="rotations_per_second_squared", verbose=0)
try:
    motor.system_reset()
    time.sleep(1.5)                     # mandatory: keep the bus silent after reset

    time.sleep(1.0)                     # let the clock accumulate a clearly nonzero count
    before_s = motor.get_current_time()
    print(f"Clock before reset: {before_s:.3f} s since system reset")

    motor.reset_time()                  # clock -> 0 (also empties the motion queue!)
    after_s = motor.get_current_time()
    print(f"Clock after reset:  {after_s:.6f} s")
    print("Second reading is near zero -- just the command round-trip time remains.")
finally:
    try:
        motor.disable_mosfets()
    except Exception:
        pass
    servomotor.close_serial_port()
```

Emergency stop

Stop the motor immediately: disables the MOSFETs, clears the movement queue, zeroes the current velocity, and snaps the internal position target to the current position. This command executes at once (it is not queued). The success response confirms the stop has already been performed. Afterward the motor has no holding torque and a load can backdrive it; send 'Enable MOSFETs' before commanding motion again. The position-deviation watchdog stays armed, so backdriving the shaft more than the max allowable position deviation (default 2 shaft rotations) raises fatal error 45, ERROR_POSITION_DEVIATION_TOO_LARGE, even though the motor is unpowered. In firmware before 0.15.4.0 the motion planner's predicted end-of-queue velocity was not reset by the queue clear, so if the stop interrupted motion, later queued moves could raise spurious fatal errors (28, ERROR_PREDICTED_VELOCITY_TOO_HIGH; 27, ERROR_PREDICTED_POSITION_OUT_OF_SAFETY_ZONE; 26, ERROR_TURN_POINT_OUT_OF_SAFETY_ZONE) or land at the wrong physical position unless 'System reset' was sent first; firmware 0.15.4.0 and later resets it correctly (verified on hardware) and needs no reset before queueing further moves. Broadcasting this command stops all motors at once (each executes silently). Any payload bytes raise fatal error 51, ERROR_COMMAND_SIZE_WRONG.

Example program:

```
#!/usr/bin/env python3
"""
Example: Emergency stop -- halt motion instantly, clear the queue, disable the MOSFETs.
Starts a slow 10-second move, stops it after 1 second, then shows the queue is empty
and the MOSFETs are off. After the stop the shaft freewheels (no holding torque).
"""
import time
import servomotor

ALIAS = 'X'                             # Device alias; change if needed
SERIAL_PORT = "/dev/tty.usbserial-110"  # Change to your serial port (e.g. "COM3" on Windows).
                                        #  Set SERIAL_PORT = "MENU" to be prompted interactively.
servomotor.set_serial_port(SERIAL_PORT)
servomotor.open_serial_port()
motor = servomotor.M3(ALIAS, time_unit="seconds", position_unit="shaft_rotations",
                      velocity_unit="rotations_per_second",
                      acceleration_unit="rotations_per_second_squared", verbose=0)
try:
    motor.system_reset()
    time.sleep(1.5)                     # mandatory: keep the bus silent after reset

    motor.enable_mosfets()
    time.sleep(0.3)                     # let the rotor settle onto its commutation step
    motor.zero_position()

    motor.trapezoid_move(5.0, 10.0)     # slow, long move: 5 rotations over 10 s
    time.sleep(1.0)                     # let the motion get underway
    motor.emergency_stop()              # executes immediately: queue cleared, MOSFETs disabled

    queued = motor.get_n_queued_items()
    status_flags, fatal_error_code = motor.get_status()
    print(f"Queued items after stop:    {queued} (expect 0)")
    print(f"MOSFETs-enabled status bit: {(status_flags >> 1) & 1} (expect 0)")
    print(f"Fatal error code:           {fatal_error_code} (expect 0 -- an emergency stop is NOT a fatal error)
    print(f"Stopped at position:        {motor.get_position():.3f} shaft rotations")

    # Recovery: since this is not a fatal error, enable_mosfets() (+0.3 s settle)
    # is enough to power the motor again. On current firmware (>= 0.15.4.0) the stop also resets the
    # planner's internal state, so enable_mosfets() (+0.3 s settle) is all that is
    # needed before queueing new moves. (Only firmware older than 0.15.4.0 left a
    # stale planner value and required a system reset first.)
    motor.system_reset()
    time.sleep(1.5)                     # bus silent again after reset
    print("Reset complete -- safe to queue new moves now.")
finally:
```

```
    try:
        motor.disable_mosfets()
    except Exception:
        pass
    servomotor.close_serial_port()
```

Zero position

Make the current position the zero position (the origin). Executes immediately and atomically zeroes the commanded position, the end-of-queue target position, the hall-sensor measured position, the current velocity, and the full PID state (integral term and error history), so 'Get position' and 'Get hall sensor position' both read approximately zero afterward and the motor does not move or jerk. Works with MOSFETs enabled or disabled, in open or closed loop, but the movement queue must be empty, otherwise the command raises fatal error 8, ERROR_QUEUE_NOT_EMPTY, and no success response is sent. Warning: safety limits are not rebased; limits set earlier with 'Set safety limits' refer to different physical locations after zeroing, and if the old limit window does not contain zero the next control cycle raises fatal error 25, ERROR_SAFETY_LIMIT_EXCEEDED, so set safety limits only after zeroing (or set them again). Zeroing also discards any accumulated PID wind-up. (In firmware before 0.15.4.0 it did not repair the stale planner velocity left by a mid-motion 'Emergency stop'; 0.15.4.0 fixed that staleness at the source.) Any payload bytes raise fatal error 51, ERROR_COMMAND_SIZE_WRONG.

Example program:

```
#!/usr/bin/env python3
"""
Example: Zero position -- redefine the current shaft position as the origin (position 0).
Zeroes and reads back ~0, makes a small move, zeroes again, and reads back ~0 once more.
The shaft never moves when zeroing; only the coordinate system shifts.
"""
import time
import servomotor

ALIAS = 'X'                               # Device alias; change if needed
SERIAL_PORT = "/dev/tty.usbserial-110"  # Change to your serial port (e.g. "COM3" on Windows).
                                          #  Set SERIAL_PORT = "MENU" to be prompted interactively.
servomotor.set_serial_port(SERIAL_PORT)
servomotor.open_serial_port()
motor = servomotor.M3(ALIAS, time_unit="seconds", position_unit="shaft_rotations",
                      velocity_unit="rotations_per_second",
                      acceleration_unit="rotations_per_second_squared", verbose=0)
try:
    motor.system_reset()
    time.sleep(1.5)                       # mandatory: keep the bus silent after reset

    motor.enable_mosfets()
    time.sleep(0.3)                       # the motor is ready immediately, but it may twitch as
                                          #  the rotor snaps to a commutation step; we settle
                                          #  ONLY because we zero precisely next -- zeroing
                                          #  during the twitch corrupts the origin
    motor.zero_position()                 # the current spot is now position 0
    print(f"After zero:    commanded = {motor.get_position():.4f}, "
          f"measured = {motor.get_hall_sensor_position():.4f} rotations (both ~0)")

    motor.trapezoid_move(0.5, 1.0)      # move +0.5 rotation away from the origin in 1 s
    while motor.get_n_queued_items() > 0:
        time.sleep(0.05)                  # wait for the motion queue to drain
    time.sleep(0.2)                       # brief mechanical settle before reading
    print(f"After move:    commanded = {motor.get_position():.4f} rotations (~0.5)")

    # Zero again: the origin is redefined at the CURRENT spot, without any motion.
    # The queue must be empty when zeroing -- the wait above guarantees that.
    motor.zero_position()
    print(f"After re-zero: commanded = {motor.get_position():.4f}, "
          f"measured = {motor.get_hall_sensor_position():.4f} rotations (both ~0)")
```

```
finally:
    try:
        motor.disable_mosfets()
    except Exception:
```

```
        pass
    servomotor.close_serial_port()
```

System reset

Reset the device. A success response is sent and fully transmitted before the reset occurs, so the host will see the reply; when broadcast, all addressed devices reset and no reply is sent. After the reset the device runs its bootloader for a 250 ms window before automatically launching the application firmware. Any valid-CRC packet addressed to the device (by alias, unique ID, or broadcast) received during that window permanently cancels the launch and keeps the device in the bootloader until another 'System reset' or a power cycle; packets not addressed to the device, or with a bad CRC, do not cancel it. So after resetting, either send nothing for well over 250 ms or deliberately send a command to stay in the bootloader (for example for a firmware upgrade). This is the only software way to clear a latched fatal error; together with 'Get status' it is one of only two commands accepted while the device is in a fatal-error state. The bootloader implements this command with identical semantics, so it can also bounce a device pinned in the bootloader back into the application. If no valid application firmware is present (its CRC check fails), the device stays in the bootloader indefinitely. Sending any payload bytes raises fatal error 51, ERROR_COMMAND_SIZE_WRONG, instead of resetting.

Example program:

```
#!/usr/bin/env python3
"""
Example: System reset -- return the motor to a known-clean power-on state.
Resets the device, honors the mandatory 1.5 s bus-silent window, then reads
'Get status' to confirm a clean baseline (application running, MOSFETs off,
no latched fatal error). Start every session this way.
"""
import time
import servomotor

ALIAS = 'X'                               # Device alias; change if needed
SERIAL_PORT = "/dev/tty.usbserial-110"  # Change to your serial port (e.g. "COM3" on Windows).
                                          #  Set SERIAL_PORT = "MENU" to be prompted interactively.
servomotor.set_serial_port(SERIAL_PORT)
servomotor.open_serial_port()
motor = servomotor.M3(ALIAS, time_unit="seconds", position_unit="shaft_rotations",
                      velocity_unit="rotations_per_second",
                      acceleration_unit="rotations_per_second_squared", verbose=0)
try:
    # 'System reset' restores the power-on state: MOSFETs disabled, default
    # limits and gains, position and clock zeroed. It is also the ONLY software
    # way to clear a latched fatal error, so it is both the first command of
    # every session and the recovery step whenever anything goes wrong.
    motor.system_reset()

    # After reset the device runs its bootloader for a short window (~250 ms)
    # before launching the application. ANY packet it hears in that window pins
    # it in the bootloader, where normal commands stop working. Keep the bus
    # completely silent for a full 1.5 s -- do not talk to ANY device on it.
    time.sleep(1.5)

    # Verify the clean baseline. get_status() returns [statusFlags, fatalErrorCode].
    status_flags, fatal_error_code = motor.get_status()
    in_bootloader = bool(status_flags & (1 << 0))    # bit 0: stuck in bootloader
    mosfets_enabled = bool(status_flags & (1 << 1))  # bit 1: MOSFETs enabled
```

```
    print(f"Status flags: 0b{status_flags:08b}")
    print(f"  In bootloader:    {in_bootloader} (expected: False)")
    print(f"  MOSFETs enabled:  {mosfets_enabled} (expected: False)")
    print(f"Fatal error code:   {fatal_error_code} (expected: 0)")
    if not in_bootloader and not mosfets_enabled and fatal_error_code == 0:
        print("Clean post-reset baseline confirmed.")
    else:
        # A packet leaked onto the bus during the silent window; reset again
        # and wait the full delay.
        print("Unexpected state -- send system_reset() again and wait 1.5 s.")
```

```
finally:
    try:
        motor.disable_mosfets()
    except Exception:
        pass
    servomotor.close_serial_port()
```

Configuration

Set maximum velocity

Set the maximum allowed velocity. This value IS used (the old note claiming otherwise is wrong), in three ways: it sizes the acceleration ramp of every 'Trapezoid move' and 'Go to position' (ramp time equals max velocity divided by max acceleration); it validates every queued move, raising fatal error ERROR_PREDICTED_VELOCITY_TOO_HIGH if the move would exceed it; and it is enforced against the live commanded velocity on every 32 microsecond control tick, so it takes effect immediately, even on moves already queued or in flight. WARNING: lowering the limit below the velocity of an in-flight move raises fatal error ERROR_VEL_TOO_HIGH on the next tick, halting the device until 'System reset'. As of firmware 0.15.6.0, a value of 0 is rejected with fatal error 34, ERROR_PARAMETER_OUT_OF_RANGE (firmware through 0.15.5.0 accepted it without complaint, after which every 'Trapezoid move'/'Go to position' was silently planned as a zero-motion dwell -- success response, queue occupied for the commanded duration, but the shaft never moved and no error was raised); note that any requested limit below half an internal unit rounds down to 0 during unit conversion, so tiny limits can hit this rejection unexpectedly. The limit comparison is inclusive: a move at exactly the limit is accepted. As of firmware 0.15.8.0 the boot default is the datasheet maximum speed: 560 RPM (9.333 rotations/second). This setting limits commanded velocity; it does not guarantee attainable shaft speed under the current operating conditions. The limit may be RAISED for experimentation up to a clamp of twice the default (18.67 rot/s; higher requested values are silently clamped with no error or feedback) or LOWERED freely, and enforcement follows immediately (a move commanding more than the limit is rejected with fatal error 16, or 28 for predicted velocities). History: 0.15.7.0 briefly defaulted to the 34 rot/s electrical rating; firmware before that booted at 68 rot/s due to a constant derivation bug. Executes synchronously, not queued. The payload is a single u32 (4 bytes); any other size raises fatal error ERROR_COMMAND_SIZE_WRONG.

Parameters:
• *maximumVelocity*: u32: Maximum velocity limit. Raw-packet users: the wire value is in internal velocity units, i.e. encoder counts per 32 microsecond timestep multiplied by 2^20; sending literal counts per timestep sets a limit 2^20 times too small. Values above the firmware ceiling MAX_VELOCITY are silently clamped. (Library versions before the 2026-07 fix carried a wrong conversion factor, about 104.86 times too large, for the selectable unit named 'counts_per_timestep'; the current library is correct.)

Example program:

```
#!/usr/bin/env python3
"""
Example: Set maximum velocity -- cap how fast the motor may spin.
Sets a 5 rotations/s ceiling, then runs a move that stays well below it.
A move that would exceed the limit is REJECTED with a latched fatal error,
not slowed down to fit.
"""
import time
import servomotor

ALIAS = 'X'                             # Device alias; change if needed
SERIAL_PORT = "/dev/tty.usbserial-110"  # Change to your serial port (e.g. "COM3" on Windows).
                                        #  Set SERIAL_PORT = "MENU" to be prompted interactively.
servomotor.set_serial_port(SERIAL_PORT)
servomotor.open_serial_port()
motor = servomotor.M3(ALIAS, time_unit="seconds", position_unit="shaft_rotations",
                      velocity_unit="rotations_per_second",
                      acceleration_unit="rotations_per_second_squared", verbose=0)

MAX_VELOCITY = 5.0                      # rotations per second -- the new velocity ceiling
DISPLACEMENT = 1.0                      # shaft rotations for the demo move
DURATION = 2.0                          # seconds; peak velocity ~0.5 rot/s, well under the limit

try:
    motor.system_reset()
    time.sleep(1.5)                     # mandatory: keep the bus silent after reset
```

```
    # Set limits BEFORE queueing moves: planning uses the values in effect at queue time,
    # and a violating move latches a fatal error (it is not clamped to fit). Never LOWER
    # this limit while a move is executing -- it applies immediately and trips fatal
    # error 16 if the in-flight velocity now exceeds it; change it only while stopped
    # with an empty queue (as here, right after reset).
    motor.set_maximum_velocity(MAX_VELOCITY)
    print(f"Maximum velocity set to {MAX_VELOCITY} rotations/s")

    motor.enable_mosfets()
    time.sleep(0.3)                    # settle the commutation-snap transient before zeroing
    motor.zero_position()

    motor.trapezoid_move(DISPLACEMENT, DURATION)   # compliant: stays below the new ceiling
    while motor.get_n_queued_items() > 0:
        time.sleep(0.05)               # always sleep between queue polls
    time.sleep(0.2)                    # brief mechanical settling before reading position

    position = motor.get_position()
```

```
    print(f"Move accepted and completed within the limit; "
          f"final position {position:.4f} shaft rotations")
finally:
    try:
        motor.disable_mosfets()
    except Exception:
        pass
    servomotor.close_serial_port()
```

Set maximum acceleration

Set the maximum acceleration used to plan all trapezoid moves queued afterwards ('Go to position', 'Trapezoid move', calibration moves); items already in the queue are unaffected. The same value is the fatal-error threshold for 'Move with acceleration': a queued item exceeding it raises fatal error 15, ERROR_ACCEL_TOO_HIGH. Values above the firmware clamp are silently clamped to the clamp and success is still returned. The setting is RAM-only and reverts to the default on any reset or power cycle. A value of 0 is rejected with fatal error 34, ERROR_PARAMETER_OUT_OF_RANGE, because trapezoid planning divides by this setting (firmware before 0.15.4.0 accepted 0 and later divided by zero). Raw-packet note: the wire value is encoder counts per timestep squared in Q24 fixed point, that is the counts/timestep^2 value multiplied by 2^24; the library's unit conversion applies this factor automatically. As of firmware 0.15.8.0 the boot default is 12000 rotations/second^2, matching the product spec and slightly above the ~11000 rot/s^2 measured unloaded spin-up, so out of the box the motor itself is the practical limit. The limit may be RAISED up to a clamp of 48000 (higher requested values are silently clamped with no error) or LOWERED freely, and enforcement follows immediately (a move demanding more is rejected with fatal error 15). History: 0.15.7.0 briefly defaulted to 500 with a clamp of 2000; earlier firmware booted at 2000 due to a constant derivation bug.

Parameters:
• *maximumAcceleration*: u32: The maximum acceleration. On the wire this is encoder counts per timestep squared in Q24 fixed point (the counts/timestep^2 value multiplied by 2^24); the library's unit conversion applies this factor. Values above the firmware cap are silently clamped to the cap. A value of 0 is rejected with fatal error 34, ERROR_PARAMETER_OUT_OF_RANGE (firmware before 0.15.4.0 accepted it and later divided by zero during trapezoid planning).

Example program:

```
#!/usr/bin/env python3
"""
Example: Set maximum acceleration -- cap the acceleration used to plan moves.
Sets a 100 rotations/s^2 limit, then runs a move that complies with it.
The firmware REJECTS a move that would exceed the limit with a latched fatal
```

```
error (it does not clamp), so set limits first and plan moves within them.
"""
import time
import servomotor

ALIAS = 'X'                               # Device alias; change if needed
SERIAL_PORT = "/dev/tty.usbserial-110"  # Change to your serial port (e.g. "COM3" on Windows).
                                          #  Set SERIAL_PORT = "MENU" to be prompted interactively.
MAX_ACCELERATION = 100.0                  # rotations/s^2. Do not set 0: firmware >= 0.15.4.0 rejects it
                                          #  with latched fatal error 34 (trapezoid planning divides by this limi
MOVE_ROTATIONS = 1.0                      # shaft rotations (relative move)
MOVE_DURATION = 2.0                       # seconds; generous enough that the planned peak
                                          #  acceleration stays well under MAX_ACCELERATION --
                                          #  a too-short duration trips fatal error 15

servomotor.set_serial_port(SERIAL_PORT)
servomotor.open_serial_port()
motor = servomotor.M3(ALIAS, time_unit="seconds", position_unit="shaft_rotations",
                      velocity_unit="rotations_per_second",
                      acceleration_unit="rotations_per_second_squared", verbose=0)
try:
    motor.system_reset()
    time.sleep(1.5)                       # mandatory: keep the bus silent after reset

    # Set the limit BEFORE queueing moves: planning uses the value in effect at
    # queue time. The setting is RAM-only and reverts to default on any reset.
    motor.set_maximum_acceleration(MAX_ACCELERATION)
    print(f"Maximum acceleration set to {MAX_ACCELERATION} rotations/s^2")

    motor.enable_mosfets()
    time.sleep(0.3)                       # let the rotor settle onto a commutation step
    motor.zero_position()

    motor.trapezoid_move(MOVE_ROTATIONS, MOVE_DURATION)   # complies with the new limit
    while motor.get_n_queued_items() > 0:                 # wait for motion to finish
        time.sleep(0.05)
    time.sleep(0.2)                       # brief mechanical settling before reading position
    print(f"Move complete under the acceleration limit; "
          f"position = {motor.get_position():.4f} rotations (expected {MOVE_ROTATIONS})")
finally:
```

```
    try:
        motor.disable_mosfets()
    except Exception:
        pass
    servomotor.close_serial_port()
```

Start calibration

Run a full motor calibration. The device spins the shaft about 1.5 rotations back and forth for 6 cycles (several seconds per cycle, product dependent: about 3.5 s on M17 and 4 s on M23) while measuring the hall-sensor midlines and determining the motor phase direction and commutation position offset, then writes these settings to flash and automatically reboots the MCU. The shaft must be free to rotate. The command resets the driver IC, enables the MOSFETs itself, and zeroes the current position and commutation offset, discarding any prior 'Zero position'. The success response is sent before calibration runs and acknowledges only that it is starting. Completion is signaled solely by the reboot, which erases all volatile state (zeroed position, maximum velocity and acceleration, enabled MOSFETs); wait about 2 seconds after the reboot before sending any command or the device may be pinned in its bootloader. Preconditions, each raising a fatal error if violated: 7 ERROR_NOT_IN_OPEN_LOOP if not in open-loop position control, 8 ERROR_QUEUE_NOT_EMPTY if the movement queue is not empty, 19 ERROR_MOTOR_BUSY if already calibrating or homing, 41 ERROR_TEST_MODE_ACTIVE if a test mode is active. Any movement command sent during calibration raises fatal error 19, ERROR_MOTOR_BUSY. If a hall channel produces too few minima or maxima the device halts with fatal error 11, ERROR_NOT_ENOUGH_MINIMA_OR_MAXIMA, and never reboots, so treat the absence of a reboot within the expected time as failure. The payload must be empty. Broadcasting starts calibration on every addressed device silently and leaves the whole bus unresponsive until the devices reboot.

Example program:

```
#!/usr/bin/env python3
"""
Example: Start calibration -- measure the hall sensors and commutation offset.
Motors are calibrated at the FACTORY and the results persist in flash, so a new
motor works in closed loop out of the box -- you rarely need this command.
Recalibrate only if closed-loop control misbehaves or after hardware service.
The shaft spins about 1.5 turns back and forth for roughly 20-60 s
(product-dependent; about 20 s measured on an M17), then the device saves the
results to flash and automatically REBOOTS itself. The bus must stay
COMPLETELY QUIET from the start command until well after that reboot.
"""
import time
import servomotor

ALIAS = 'X'                             # Device alias; change if needed
SERIAL_PORT = "/dev/tty.usbserial-110"  # Change to your serial port (e.g. "COM3" on Windows).
                                        #  Set SERIAL_PORT = "MENU" to be prompted interactively.
CALIBRATION_WAIT = 60.0                 # seconds of total bus silence; covers all products
                                        #  (about 20 s measured on an M17)

servomotor.set_serial_port(SERIAL_PORT)
servomotor.open_serial_port()
motor = servomotor.M3(ALIAS, time_unit="seconds", position_unit="shaft_rotations",
                      velocity_unit="rotations_per_second",
                      acceleration_unit="rotations_per_second_squared", verbose=0)
try:
    print("Note: motors are calibrated at the factory and the results persist in")
    print("flash, so running this is rarely needed.")

    # PRECONDITIONS: the shaft must be COMPLETELY FREE to rotate (no load, no
    # coupling). Calibration also requires MOSFETs disabled, an empty queue, no
    # test mode, and open-loop mode -- the fresh reset below guarantees all that.
    motor.system_reset()
    time.sleep(1.5)                     # mandatory: keep the bus silent after reset

    motor.start_calibration()           # the success response only means it STARTED
    print(f"Calibration started; keeping the bus quiet for {CALIBRATION_WAIT:.0f} s ...")

    # Do NOT poll while calibration runs: polling during calibration can disturb
    # the measurement and reduce its accuracy, and when calibration finishes the
    # device saves to flash and automatically REBOOTS -- a poll landing after
    # that reboot pins the device in the bootloader. A generous fixed wait avoids
    # both hazards; 60 s covers all products (about 20 s measured on an M17).
    time.sleep(CALIBRATION_WAIT)
```

```
    # The auto-reboot erased all volatile state (zeroed position, limits, enabled
    # MOSFETs). Reset once more and verify a clean baseline before trusting it.
    motor.system_reset()
    time.sleep(1.5)                     # mandatory: keep the bus silent after reset
    flags, fatal_error = motor.get_status()
    in_bootloader = bool(flags & (1 << 0))
    mosfets_on = bool(flags & (1 << 1))
    print(f"Post-calibration baseline: bootloader={in_bootloader}, "
          f"MOSFETs enabled={mosfets_on}, fatal error code={fatal_error}")
    if in_bootloader or mosfets_on or fatal_error != 0:
        raise RuntimeError("Device did not come back to a clean baseline")
    print("Calibration complete; results are saved in flash and persist across power cycles.")
finally:
    try:
        motor.disable_mosfets()
    except Exception:
        pass
    servomotor.close_serial_port()
```

Set maximum motor current

Set the maximum motor current and maximum regeneration current. Executes immediately (not queued); the success response acknowledges the values are already applied. The values are not saved to non-volatile memory and revert to the product-specific firmware default after a reset (M1: 100, M2/M17/M23: 200). Units are arbitrary: internally the motor current value caps the applied PWM duty ('PWM voltage'), limiting current only indirectly, and it also rescales the PID controller's authority (maximum error, integral limit, and output limit are derived from it); setting it while the motor is moving clamps the integral term and zeroes the derivative-filter state, so expect a brief control transient. Both parameters are validated against a product-specific ceiling (M1/M2: 300, M17: 390, M23: 1024); exceeding it with EITHER parameter raises fatal error 23, ERROR_MAX_PWM_VOLTAGE_TOO_HIGH, which disables the MOSFETs until 'System reset'. Do not choose values above the product ceiling.

Parameters:
• *motorCurrent*: u16: The maximum motor current in arbitrary units (not amps). A value of 150 or 200 is suitable. Internally this caps the applied PWM duty and rescales the PID controller's authority; at speed the firmware additionally allows back-EMF compensation on top of this cap, so it acts as a torque/current cap rather than an absolute duty limit. Must not exceed the product-specific maximum (M1/M2: 300, M17: 390, M23: 1024) or fatal error 23, ERROR_MAX_PWM_VOLTAGE_TOO_HIGH, is raised.
• *regenerationCurrent*: u16: The motor regeneration current (while braking) in the same arbitrary units. It sets the regen-side analog-watchdog threshold, which currently only drives the red LED on M1/M2 and has no functional effect on M17/M23. The value is still range-checked: exceeding the product-specific maximum (M1/M2: 300, M17: 390, M23: 1024) raises fatal error 23, ERROR_MAX_PWM_VOLTAGE_TOO_HIGH, so do not treat this parameter as ignorable.

Example program:

```
#!/usr/bin/env python3
"""
Example: Set maximum motor current -- cap the motor's drive strength.
Sets a working current limit of 200 before enabling the MOSFETs, then performs
a small move under that limit. The current limit doubles as a programmable
force limit: low values make the motor deliberately gentle and compliant.
"""
import time
import servomotor

ALIAS = 'X'                             # Device alias; change if needed
SERIAL_PORT = "/dev/tty.usbserial-110"  # Change to your serial port (e.g. "COM3" on Windows).
                                        #  Set SERIAL_PORT = "MENU" to be prompted interactively.
MOTOR_CURRENT = 200                     # internal units; good general working value
REGEN_CURRENT = 200                     # internal units; braking-side limit, same range rules
DISPLACEMENT = 1.0                      # shaft rotations (small, safe demo move)
DURATION = 2.0                          # seconds

servomotor.set_serial_port(SERIAL_PORT)
servomotor.open_serial_port()
motor = servomotor.M3(ALIAS, time_unit="seconds", position_unit="shaft_rotations",
                      velocity_unit="rotations_per_second",
                      acceleration_unit="rotations_per_second_squared",
                      current_unit="internal_current_units", verbose=0)
try:
    motor.system_reset()
    time.sleep(1.5)                     # mandatory: keep the bus silent after reset

    # Set the current limit BEFORE enabling and moving -- the controller's
    # authority is derived from the value in effect at that moment. 200 is a
    # solid working value; ~390 is the practical maximum on the M17 (exceeding
    # the product ceiling latches fatal error 23). Going LOW (20-100) makes the
    # motor weak and compliant, so the current limit doubles as a programmable
    # force limit -- ideal for gentle homing or pressing softly against objects.
    motor.set_maximum_motor_current(MOTOR_CURRENT, REGEN_CURRENT)
    print(f"Maximum motor current set to {MOTOR_CURRENT} internal units "
          f"(regen: {REGEN_CURRENT}).")

    motor.enable_mosfets()
```

```
    time.sleep(0.3)                     # energizing snaps the rotor; let it settle
    motor.zero_position()               # establish the origin after the settle

    motor.trapezoid_move(DISPLACEMENT, DURATION)
    while motor.get_n_queued_items() > 0:   # wait for the queued move to finish
        time.sleep(0.05)
```

```
    time.sleep(0.2)                     # brief mechanical settling before reading
    position = motor.get_position()
    print(f"Moved to {position:.3f} shaft rotations with the current limit at "
          f"{MOTOR_CURRENT}. Note: the limit is RAM-only and reverts on reset.")
finally:
    try:
        motor.disable_mosfets()
    except Exception:
        pass
    servomotor.close_serial_port()
```

Set safety limits

Set the position safety limits. Takes effect immediately (not queued); the success response acknowledges the limits are already applied. Enforcement is fatal, not clamping: every motor-control tick the actual position is compared against the limits, and a position outside them raises fatal error 25, ERROR_SAFETY_LIMIT_EXCEEDED, disabling the MOSFETs until 'System reset'. Consequently, sending limits that exclude the CURRENT position faults the device essentially instantly. Queued moves are also pre-checked at enqueue time: a predicted final position or velocity-reversal turn point outside the limits raises fatal error 27, ERROR_PREDICTED_POSITION_OUT_OF_SAFETY_ZONE, or fatal error 26, ERROR_TURN_POINT_OUT_OF_SAFETY_ZONE, rather than the move being trimmed. The limits are compared in the firmware's internal position units (encoder counts, the same domain used by 'Get position' and 'Go to position'), not microsteps. As of firmware 0.15.5.0, a lower limit greater than the upper limit is rejected with fatal error 34, ERROR_PARAMETER_OUT_OF_RANGE. WARNING for firmware up to and including 0.15.4.0: inverted limits were stored unvalidated, making every position out of bounds and faulting the device (error 25) on the next control tick -- and because that fatal error fires while this command's own success reply is still transmitting, it can truncate the reply and, depending on the exact timing, deadlock the device completely (no response to any command, not even 'Get status' or 'System reset') until power cycle; on such firmware never send lower > upper. (0.15.5.0 also fixed the underlying race for all fatal errors: a fatal that fires while any reply is transmitting no longer truncates it or hangs the device.) Also note the limits are interpreted in the current position frame: 'Zero position' shifts which physical locations they refer to, so re-send the limits after zeroing whenever the fence protects real hardware. The limits are not saved to non-volatile memory; a reset restores INT64_MIN/INT64_MAX, effectively disabling them.

Parameters:
• *lowerLimit*: i64: The lower position limit in the firmware's internal position units (encoder counts, the same domain returned by 'Get position'), not microsteps.
• *upperLimit*: i64: The upper position limit in the firmware's internal position units (encoder counts), not microsteps. Must be greater than or equal to the lower limit; as of firmware 0.15.5.0 swapped limits are rejected with fatal error 34, ERROR_PARAMETER_OUT_OF_RANGE (firmware through 0.15.4.0 stored them unvalidated and faulted the device immediately with error 25).

Example program:

```
#!/usr/bin/env python3
"""
Example: Set safety limits -- fence the motor into a safe position window.
Zeroes the position, sets limits of -2..+2 rotations, then performs a move that
stays inside the fence. Moves that would leave the fence are REJECTED with a
latched fatal error -- they are never clamped or trimmed.
"""
import time
```

```
import servomotor

ALIAS = 'X'                             # Device alias; change if needed
SERIAL_PORT = "/dev/tty.usbserial-110"  # Change to your serial port (e.g. "COM3" on Windows).
                                        #  Set SERIAL_PORT = "MENU" to be prompted interactively.
LOWER_LIMIT = -2.0                      # shaft rotations
UPPER_LIMIT = 2.0                       # shaft rotations
DISPLACEMENT = 1.0                      # shaft rotations -- comfortably inside the fence
DURATION = 2.0                          # seconds

servomotor.set_serial_port(SERIAL_PORT)
servomotor.open_serial_port()
motor = servomotor.M3(ALIAS, time_unit="seconds", position_unit="shaft_rotations",
                      velocity_unit="rotations_per_second",
                      acceleration_unit="rotations_per_second_squared", verbose=0)
try:
    motor.system_reset()
    time.sleep(1.5)                     # mandatory: keep the bus silent after reset

    # Safe order: enable, settle 0.3 s, zero, THEN tighten limits. Energizing
    # snaps the rotor to a commutation step; zeroing after that transient keeps
    # the fence centered on the true position.
    motor.enable_mosfets()
    time.sleep(0.3)
    motor.zero_position()

    # The limits must bracket the CURRENT position (0 after zeroing): limits
    # that exclude it fault the device essentially instantly (fatal error 25).
    # Limits are RAM-only; a reset restores the +/-infinity defaults.
    motor.set_safety_limits(LOWER_LIMIT, UPPER_LIMIT)
    print(f"Safety limits set: {LOWER_LIMIT:+.1f} to {UPPER_LIMIT:+.1f} shaft rotations.")

    motor.trapezoid_move(DISPLACEMENT, DURATION)    # compliant move: ends at +1.0, inside
    while motor.get_n_queued_items() > 0:           # wait for the queued move to finish
        time.sleep(0.05)
    time.sleep(0.2)                     # brief mechanical settling before reading
    position = motor.get_position()
```

```
    print(f"Move inside the limits succeeded; now at {position:.3f} shaft rotations.")

    # Do NOT queue e.g. motor.trapezoid_move(5.0, 2.0) here: a move whose end
    # position falls outside the limits is rejected with latched fatal error 27
    # (a reversal whose turn-around point falls outside them is rejected with fatal error 26 or 27 -- in practi
    # disables itself and only system_reset() + 1.5 s wait recovers it.
finally:
    try:
        motor.system_reset()            # defensive: clear the tightened limits before exit
        time.sleep(1.5)                 # bus must stay silent after reset
        motor.disable_mosfets()
    except Exception:
        pass
    servomotor.close_serial_port()
```

Test mode

Set or trigger a test mode. Executes immediately (not queued); intended only for developers and production testing. As of firmware 0.15.4.0, value 0 safely clears all test modes (motor test mode, overtemperature test mode, and the overvoltage-threshold override). WARNING for firmware before 0.15.4.0: value 0 hangs the device forever before any reply and only a power cycle recovers it -- on older firmware clear test modes only with 'System reset'. Of motor test modes 1-9, only 2, 3 and 4 do anything: 2 suppresses the fatal error on a failed 'Go to closed loop' (the device drops back to open loop with MOSFETs disabled instead), 3 captures one PID debug snapshot (error, P/I/D terms, output) into the multipurpose buffer (read it with 'Read multipurpose buffer'), and 4 formerly read the GC6609 driver registers into that buffer, but is DEFUNCT on current units (the GC6609 is used only on legacy M17 units with software compatibility code 1; on current hardware the buffer stays empty and the mode stays latched until reset). Any nonzero motor test mode makes 'Start calibration' raise fatal error ERROR_TEST_MODE_ACTIVE. LED test modes 10-13 (10=both LEDs off, 11=green on, 12=red on, 13=both on) send the success response and then hang forever; power cycle required. Values 14-73 deliberately trigger fatal errors 0-59 (value minus 14): no success response is sent -- the device replies with an error packet and stays in the fatal-error state until 'System reset'. Value 14 raises fatal error 0, which shows a solid red LED while the status error code reads 0 and is easily mistaken for no error. Value 74 sets the overvoltage-protection threshold to 22 V (production test; should trip on a 24 V supply), 75 sets it to 26 V (should not trip on 24 V), and 76 raises the overtemperature cutoff to about 90 degrees C (default about 80); these take effect immediately and only a reset restores the defaults. Values above 76 raise fatal error 53, ERROR_INVALID_TEST_MODE. Values 0-9 and 74-76 return a success response, as do LED modes 10-13 (which hang immediately after sending it); values 14-73 and values above 76 reply with an error packet instead. Broadcast executes with no reply; broadcasting 10-13 bricks every device on the bus until power cycled (broadcasting 0 does too, but only on firmware before 0.15.4.0).

Parameters:
• *testMode*: u8: The test mode to use or trigger. See the command description for what each value range does. Never send 10-13: they hang the device until it is power cycled. Value 0 safely clears all test modes on firmware 0.15.4.0 and later (on older firmware value 0 also hangs the device). Values 14-73 trigger fatal errors and values above 76 raise fatal error 53, ERROR_INVALID_TEST_MODE.

Example program:

```
#!/usr/bin/env python3
"""
Example: Test mode -- trigger the one generally useful and safe test mode, value 3.
Test mode 3 captures a single PID debug snapshot (error, P, I, D, output) into the
multipurpose buffer, which we read back and print. Read the DANGER notes below before
experimenting with any other value.
"""
import time
import servomotor

ALIAS = 'X'                             # Device alias; change if needed
SERIAL_PORT = "/dev/tty.usbserial-110"  # Change to your serial port (e.g. "COM3" on Windows).
                                        #  Set SERIAL_PORT = "MENU" to be prompted interactively.
TEST_MODE_PID_SNAPSHOT = 3              # the only test mode demonstrated here
# DANGER -- never send these test mode values:
#   10-13       : the firmware hangs FOREVER; only a power cycle recovers the device
#   0           : safe on firmware >= 0.15.4.0 (clears any motor/overtemperature test mode); hangs forever on O
#   14-73       : each one deliberately latches a fatal error (error code = value - 14)
# To clear an active test mode, system_reset() works on every firmware version;
# test_mode(0) also clears it safely on firmware >= 0.15.4.0 (on older firmware it hangs the device).

servomotor.set_serial_port(SERIAL_PORT)
servomotor.open_serial_port()
motor = servomotor.M3(ALIAS, time_unit="seconds", position_unit="shaft_rotations",
                      velocity_unit="rotations_per_second",
                      acceleration_unit="rotations_per_second_squared", verbose=0)
try:
    motor.system_reset()
    time.sleep(1.5)                     # mandatory: keep the bus silent after reset

    # The PID snapshot is only meaningful in closed-loop mode, so enter it first.
    # go_to_closed_loop() enables the MOSFETs by itself; skipping a separate
```

```
    # 'Enable MOSFETs' may even give a gentler engagement.
    motor.go_to_closed_loop()
    deadline = time.time() + 6.0
    while True:                         # poll until the closed-loop bit (bit 2) sets
        status_flags, fatal_error_code = motor.get_status()
        if status_flags & 0b100:
            break
        if time.time() > deadline:
            raise SystemExit("Never entered closed loop -- has this motor been calibrated?")
        time.sleep(0.1)
    time.sleep(0.3)

    motor.test_mode(TEST_MODE_PID_SNAPSHOT)
```

```
    time.sleep(0.1)                     # give the control loop a moment to store the snapshot

    data = motor.read_multipurpose_buffer()
    if data[0] == 0:                    # a single 0 byte = buffer empty (fw 0.15.4.0+;
        print("Multipurpose buffer was empty (no snapshot stored)")  # older firmware timed out instead)
    else:
        print(f"Data type tag: {data[0]} (4 = PID debug snapshot)")
        for n, name in enumerate(["error", "P term", "I term", "D term", "output"]):
            value = int.from_bytes(data[1 + n * 4:5 + n * 4], "little", signed=True)
            print(f"  {name}: {value} (internal units)")

    # Clear the test mode with a reset. (As of firmware 0.15.4.0, test_mode(0) also
    # clears it safely; on OLDER firmware value 0 hangs the device until power cycle.)
    motor.system_reset()
    time.sleep(1.5)                     # bus must stay silent after reset
finally:
    try:
        motor.disable_mosfets()
    except Exception:
        pass
    servomotor.close_serial_port()
```

Set PID constants

Set the P, I, and D constants of the closed-loop controller that tries to maintain the motion trajectory. Executes immediately, not queued: the new constants are applied atomically (interrupts briefly disabled) before the success response is sent, even mid-move, which can cause a brief control transient. The values are raw fixed-point gains: the controller output is (P*error + integral term + derivative term) right-shifted by a product-specific amount (18 on M1/M2, 11 on M17, 14 on M23), so identical numbers behave very differently across products. The setter accepts and stores raw u32 values from 0 to 4294967295; working coefficients are capped at 2147483647, I may be limited further for overflow safety, and D is scaled for derivative averaging. 'Get PID constants' (48, released in firmware 0.16.0.0) returns the stored raw values unchanged, including values above those effective limits. Setting the constants also rescales internal safety limits: the maximum usable error window is derived as output authority divided by P (a very large P shrinks it), and the integral accumulator is clamped to 25 percent of output authority for anti-windup. The constants are not persisted: a reset or power cycle restores compiled-in per-product defaults (P/I/D: M1 5000/1/5000, M2 20000/1/100000, M17 2000/5/175000, M23 500/1/1000). The command has no preconditions and is accepted in any state; a payload other than exactly 12 bytes raises fatal error 51, ERROR_COMMAND_SIZE_WRONG.

Parameters:
• *kP*: u32: Raw proportional gain (P), 0 to 4294967295. The working coefficient is capped at 2147483647; readback preserves the requested value.
• *kI*: u32: Raw integral gain (I), 0 to 4294967295. The working coefficient is capped at 2147483647 and may be limited further for overflow safety; readback preserves the requested value.
• *kD*: u32: Raw derivative gain (D), 0 to 4294967295. The working coefficient is capped at 2147483647, then right-shifted by 5 bits for derivative averaging in current builds. Values 0 to 31 produce no derivative action; readback preserves the original unshifted value.

Example program:

```
#!/usr/bin/env python3
"""
Example: Set PID constants -- tune the closed-loop controller gains.
Enters closed loop, applies the known-good M17 gains, then performs a small move
so you can observe the motor tracking under the freshly set constants.
"""
import time
import servomotor

ALIAS = 'X'                               # Device alias; change if needed
SERIAL_PORT = "/dev/tty.usbserial-110"  # Change to your serial port (e.g. "COM3" on Windows).
                                          #  Set SERIAL_PORT = "MENU" to be prompted interactively.
PID_P = 2000                              # known-good M17 proportional gain (raw fixed-point)
PID_I = 5                                 # known-good M17 integral gain
PID_D = 175000                            # known-good M17 derivative gain; values below 32 are
                                          #  quantized to ZERO derivative action
servomotor.set_serial_port(SERIAL_PORT)
servomotor.open_serial_port()
motor = servomotor.M3(ALIAS, time_unit="seconds", position_unit="shaft_rotations",
                      velocity_unit="rotations_per_second",
                      acceleration_unit="rotations_per_second_squared", verbose=0)
try:
    motor.system_reset()
    time.sleep(1.5)                       # mandatory: keep the bus silent after reset
    motor.go_to_closed_loop()             # PID gains only act in closed loop. This enables the
                                          #  MOSFETs by itself; skipping a separate
                                          #  'Enable MOSFETs' may even give a gentler engagement.
    deadline = time.time() + 6.0
    while True:
        status_flags, fatal_error_code = motor.get_status()
        if status_flags & (1 << 2):     # bit 2 = closed-loop mode active
            break
        if time.time() > deadline:
            raise TimeoutError("Never entered closed loop -- is the motor calibrated?")
        time.sleep(0.1)
    time.sleep(0.3)                       # settle after the mode change so the zero is clean
    motor.zero_position()
```

```
    # The gains apply immediately (even mid-move); effective coefficients may be
    # limited internally. Firmware 0.16.0.0+ provides get_pid_constants() to read
    # back the raw values. Reset or power cycling restores firmware defaults.
    motor.set_pid_constants(PID_P, PID_I, PID_D)
    print(f"PID constants set: P={PID_P} I={PID_I} D={PID_D}")
    motor.trapezoid_move(1.0, 2.0)      # small safe move to watch it track: 1 rotation in 2 s
    while motor.get_n_queued_items() > 0:
        time.sleep(0.05)                # poll the queue; never busy-poll without a sleep
```

```
    time.sleep(0.2)                     # brief mechanical settling before reading position
    print(f"Move complete under new gains; position = {motor.get_position():.3f} shaft rotations")
finally:
    try:
        motor.disable_mosfets()
    except Exception:
        pass
    servomotor.close_serial_port()
```

Set max allowable position deviation

Set the maximum distance that the actual motor position (as measured by the hall sensors) is allowed to deviate from the commanded position; exceeding it raises fatal error 45 (ERROR_POSITION_DEVIATION_TOO_LARGE). Takes effect immediately (not queued). The firmware stores the absolute value of the input, so negative values behave as positive; as of firmware 0.15.9.0 the value INT64_MIN (which has no positive counterpart) saturates to the maximum limit. (Firmware 0.15.4.0 through 0.15.8.0 had a landmine here: INT64_MIN made the stored limit go negative and fatal error 45 latched on the very next control tick.) The setting is RAM-only and reverts to the default of 2 shaft rotations (6553600 microsteps on M3, M17, and M23; one encoder count equals one internal microstep) at power-up or after 'System reset'. The deviation check runs continuously in the background except during calibration, and it is not gated on the MOSFETs being enabled or a move being in progress: externally turning the shaft past the limit while the motor is idle or disabled also trips fatal error 45. When the error trips, the signed deviation, absolute deviation, commanded position, and hall sensor position are latched into debug values 1 to 4, readable via 'Get debug values'. A payload that is not exactly 8 bytes raises fatal error 51, ERROR_COMMAND_SIZE_WRONG.

Parameters:
• *maxAllowablePositionDeviation*: i64: The new maximum allowable position deviation; the firmware stores the absolute value, so negative values behave as positive

Example program:

```
#!/usr/bin/env python3
"""
Example: Set max allowable position deviation -- a collision/stall watchdog.
Tightens the allowed gap between commanded and measured position to 0.5
rotations, then performs a gentle move. If the shaft ever lags the command by
more than the limit (stall, jam, collision), fatal error 45 latches.
"""
import time
import servomotor

ALIAS = 'X'                             # Device alias; change if needed
SERIAL_PORT = "/dev/tty.usbserial-110"  # Change to your serial port (e.g. "COM3" on Windows).
                                        #  Set SERIAL_PORT = "MENU" to be prompted interactively.
MAX_DEVIATION = 0.5                     # shaft rotations (firmware default is 2)
DISPLACEMENT = 0.5                      # shaft rotations (gentle demo move)
DURATION = 2.0                          # seconds

servomotor.set_serial_port(SERIAL_PORT)
servomotor.open_serial_port()
motor = servomotor.M3(ALIAS, time_unit="seconds", position_unit="shaft_rotations",
```

```
                      velocity_unit="rotations_per_second",
                      acceleration_unit="rotations_per_second_squared", verbose=0)
try:
    motor.system_reset()
    time.sleep(1.5)                    # mandatory: keep the bus silent after reset

    # Order matters: enable + settle + zero FIRST, tighten the guard LAST.
    # The motor is ready immediately after enabling, but it may twitch as the
    # rotor snaps to a commutation step; we settle ONLY because we zero
    # precisely and arm a tight deviation limit right after -- doing that during
    # the twitch gives a corrupted zero or a spurious fatal error 45.
    motor.enable_mosfets()
    time.sleep(0.3)
    motor.zero_position()

    # Tighten the guard from the default 2 shaft rotations down to 0.5. From
    # now on, |commanded - measured| > 0.5 rotations latches fatal error 45
    # ASYNCHRONOUSLY -- the offending move itself still returns success, so
    # check get_status() after risky motion. The check stays armed even with
    # the MOSFETs disabled: back-driving the shaft past the limit trips it too.
    # The setting is RAM-only and reverts to 2 rotations on reset.
    motor.set_max_allowable_position_deviation(MAX_DEVIATION)
    print(f"Max allowable position deviation set to {MAX_DEVIATION} shaft rotations.")

    motor.trapezoid_move(DISPLACEMENT, DURATION)    # gentle move; tracks well within 0.5
```

```
    while motor.get_n_queued_items() > 0:          # wait for the queued move to finish
        time.sleep(0.05)
    time.sleep(0.2)                    # brief mechanical settling before reading
    position = motor.get_position()
    print(f"Move completed with the deviation guard armed; now at "
          f"{position:.3f} shaft rotations.")
    print("If this motor stalls or collides mid-move, fatal error 45 latches; "
          "recover with system_reset() + 1.5 s of bus silence.")
finally:
    try:
        # Defensive: clear the tightened deviation guard before exit -- it stays
        # armed even with the MOSFETs disabled, so hand-turning the shaft past
        # 0.5 rotations after the demo would latch fatal error 45.
        motor.system_reset()
        time.sleep(1.5)                # bus must stay silent after reset
        motor.disable_mosfets()
    except Exception:
        pass
    servomotor.close_serial_port()
```

CRC32 control

Enable or disable the CRC32 layer of the protocol for this device, in both directions. When enabled (the power-up default), every received packet must carry a trailing 4-byte CRC32 and is validated against it, and every response is sent with response-indicator byte 253 followed by a CRC32; when disabled, received packets must not carry a CRC32 and responses use indicator byte 252 with no CRC32. The change takes effect immediately, before the acknowledgment is transmitted, so the success response to this very command already arrives in the new format. The setting is RAM-only and reverts to enabled at every power-up or 'System reset'; a host that disables CRC32 must re-send this command after any reset. While CRC32 is enabled, a received packet that fails validation is dropped silently with no error response: the failure appears only as a timeout and an incremented counter readable via 'Get communication statistics'. Broadcasting applies the change to every device on the bus with no responses, which is the practical way to switch a whole bus at once. A payload that is not exactly 1 byte raises fatal error 51, ERROR_COMMAND_SIZE_WRONG.

Parameters:
• *enableCrc32*: u8: Control value: 0 disables the CRC32 protocol, any nonzero value enables it

Example program:

```
#!/usr/bin/env python3
"""
Example: CRC32 control -- turn the protocol's CRC32 layer off and back on.
Shows that with CRC32 disabled the device accepts CRC-less packets, then
restores the CRC-enabled default. The state changes take effect immediately,
so every command must be framed to match the device's CURRENT CRC state.
"""
import time
import servomotor
from servomotor import communication

ALIAS = 'X'                             # Device alias; change if needed
SYSTEM_RESET_COMMAND_ID = 27            # used by the dual-framing recovery in the finally block
PING_PAYLOAD = b"hello12345"            # 'Ping' requires exactly 10 bytes
SERIAL_PORT = "/dev/tty.usbserial-110"  # Change to your serial port (e.g. "COM3" on Windows).
                                        #  Set SERIAL_PORT = "MENU" to be prompted interactively.
servomotor.set_serial_port(SERIAL_PORT)

servomotor.open_serial_port()
motor = servomotor.M3(ALIAS, time_unit="seconds", position_unit="shaft_rotations",
                      velocity_unit="rotations_per_second",
                      acceleration_unit="rotations_per_second_squared", verbose=0)
try:
    motor.system_reset()
    time.sleep(1.5)                     # mandatory: keep the bus silent after reset
                                        #  (a reset also restores the CRC-enabled default)

    # The motor object's outgoing framing must always match the device's CRC32
    # state: with CRC32 enabled on the device, a CRC-less or corrupted packet is
    # dropped silently (symptom: TimeoutError). The reverse mismatch is worse:
    # while the device has CRC32 disabled, a packet sent WITH a CRC has its 4 CRC
    # bytes counted as payload, which latches fatal error 51
    # (ERROR_COMMAND_SIZE_WRONG), disabling the device until 'System reset'.
    # Keep the two in lockstep: send 'CRC32 control', then set_crc32_enabled().

    # NOTE: the library prints a framing warning after each CRC32-control ack --
    # the device applies the change first, so the ack already arrives in the NEW
    # format; the warnings are expected and harmless.
    print("Disabling CRC32 (this command itself is sent WITH a CRC)...")
    motor.crc32_control(0)
    motor.set_crc32_enabled(False)      # from now on this object sends without a CRC

    print("Pinging WITHOUT a CRC to prove the device now accepts CRC-less packets...")
    echoed = motor.ping(PING_PAYLOAD)
    print(f"  sent {PING_PAYLOAD}, got {echoed}: {'match' if echoed == PING_PAYLOAD else 'MISMATCH'}")
```

```
    print("Re-enabling CRC32 (sent WITHOUT a CRC to match the current device state)...")
    motor.crc32_control(1)
    motor.set_crc32_enabled(True)       # back to the CRC-enabled default

    echoed = motor.ping(PING_PAYLOAD)   # normal M3 call; a CRC is appended again
    print(f"  normal ping echoed {echoed}: {'match' if echoed == PING_PAYLOAD else 'MISMATCH'}")
    print("Device is back at its CRC-enabled default.")
finally:
    # We may abort in either CRC state. A CRC-less reset is silently ignored by a
    # CRC-enabled device (harmless); after the mandatory bus-silent wait, a normal
    # CRC-framed reset covers the other case. Reset also disables the MOSFETs.
    try:
        communication.execute_command(SYSTEM_RESET_COMMAND_ID, [],
                                       alias_or_unique_id=motor.alias_or_unique_id,
                                       crc32_enabled=False, verbose=0)
    except Exception:
        pass
    time.sleep(1.5)
    try:
        motor.system_reset()
        time.sleep(1.5)
    except Exception:
        pass
    servomotor.close_serial_port()
```

Get PID constants

Read a coherent snapshot of the configured raw P, I, and D constants. The representation matches 'Set PID constants' (43). Takes no input and returns exactly three little-endian u32 values (12 payload bytes) in P/I/D order, with no unit conversion. These are the last requested values, not the derived working coefficients: the setter accepts the full u32 range (0 to 4294967295), caps working gains at 2147483647, may further limit I for overflow safety, and scales/quantizes D for derivative averaging. Readback preserves the original D and any raw values above those effective limits. Reading changes no integrator, derivative state, queue, position, or output-enable state, and can be used while moving. Gains are RAM-only; reset or power cycling restores compiled-in product defaults (P/I/D: M1 5000/1/5000, M2 20000/1/100000, M17 2000/5/175000, M23 500/1/1000). Broadcast sends no reply. Any non-empty payload raises fatal error 51, ERROR_COMMAND_SIZE_WRONG; in an existing fatal-error state, the usual error response is returned instead of gains. Released in firmware 0.16.0.0; also present in the validated development build 0.15.12.1. Older firmware does not implement this command.

Returns:
• *kP*: u32: Configured raw proportional gain (P), before working-coefficient limits
• *kI*: u32: Configured raw integral gain (I), before working-coefficient limits
• *kD*: u32: Configured raw derivative gain (D), before working-coefficient limits and derivative-averaging scaling

Example program:

```
#!/usr/bin/env python3
"""Read the configured raw PID gains (requires firmware 0.16.0.0 or later)."""
import servomotor

ALIAS = 'X'                             # Device alias; change if needed.
SERIAL_PORT = "MENU"                   # Or specify your USB-RS485 serial port.

servomotor.set_serial_port(SERIAL_PORT)
servomotor.open_serial_port()
motor = servomotor.M3(ALIAS, verbose=0)
try:
    # Reads the device without enabling outputs or changing controller state.
    # Values are raw user gains; working coefficients may be limited internally.
    p, i, d = motor.get_pid_constants()
    print(f"Configured PID constants: P={p} I={i} D={d}")
finally:
    servomotor.close_serial_port()
```

Device Management

Time sync

Send the master's clock reading to the motor so the motor can discipline its clock rate. The motor never sets or steps its clock to the sent value: it computes the signed error (master minus local, clamped to plus or minus 32768 microseconds per call) and feeds it to a PI controller that only re-trims the internal HSI16 oscillator via the HSITRIM field of the RCC-ICSCR register, clamped to the narrow range 62 to 66 (center 64). Drift therefore converges gradually; a large initial offset is never corrected by this command, so establish a shared epoch first, for example by broadcasting 'Reset time'. Send it regularly, roughly 10 times per second, to each motor. Broadcast is a complete no-op: on a broadcast packet the oscillator is not trimmed and no reply is sent, so each motor on a shared bus must be addressed individually. Executes immediately (not queued). The payload must be exactly 4 bytes or the device raises fatal error 51, ERROR_COMMAND_SIZE_WRONG; a malfunctioning clock can also raise fatal error 1, ERROR_TIME_WENT_BACKWARDS.

Parameters:
• *masterTime*: u32: The low 32 bits of the master's absolute clock, in microseconds (this 32-bit window wraps every ~71.6 minutes). The device truncates its own 64-bit clock to 32 bits and subtracts modularly, so the computed error is only correct while the true offset between master and device is under about 35.8 minutes -- hence the need to sync regularly.

Returns:
• *timeError*: i32: The signed error in the motor's clock, in microseconds, computed as master time minus device time by modular 32-bit subtraction; positive means the device clock is behind the master. Delivered raw, with no unit conversion applied.
• *rccIcscr*: u16: The low 16 bits of the RCC-ICSCR register, holding the oscillator calibration fields HSICAL (bits 7:0) and HSITRIM (bits 14:8).

Example program:

```
#!/usr/bin/env python3
"""
Example: Time sync -- discipline the motor's clock rate to the host's clock.
Zeros the device clock to establish a shared epoch, then sends the host's
elapsed microseconds at 10 Hz for 3 seconds, printing the motor's reported
clock error each time. Used before multi-motor coordinated motion.
"""
import time
import servomotor

ALIAS = 'X'                             # Device alias; change if needed
SERIAL_PORT = "/dev/tty.usbserial-110"  # Change to your serial port (e.g. "COM3" on Windows).
                                        #  Set SERIAL_PORT = "MENU" to be prompted interactively.
SYNC_DURATION = 3.0                     # seconds of syncing to demonstrate
SYNC_INTERVAL = 0.1                     # seconds between syncs (10 Hz is the recommended rate)
servomotor.set_serial_port(SERIAL_PORT)
servomotor.open_serial_port()
motor = servomotor.M3(ALIAS, time_unit="seconds", position_unit="shaft_rotations",
                      velocity_unit="rotations_per_second",
                      acceleration_unit="rotations_per_second_squared", verbose=0)
try:
    motor.system_reset()
    time.sleep(1.5)                     # mandatory: keep the bus silent after reset

    # Establish a shared epoch: zero the device clock and record the host clock
    # at the same instant. (On a multi-motor bus, broadcast 'Reset time' to
    # alias 255 so all clocks zero together -- it also stops motion, so do it
    # before any motion starts.)
    motor.reset_time()
    epoch = time.monotonic()

    # 'Time sync' never sets the clock -- it trims the internal oscillator, i.e.
```

```
    # the clock's RATE, so the error converges gradually; budget ~5 s of regular
    # syncing before trusting tight synchronization. A broadcast time_sync is a
    # complete no-op: each motor on the bus must be addressed individually.
    print("Syncing at 10 Hz (positive error = motor clock behind the host)...")
    while time.monotonic() - epoch < SYNC_DURATION:
        elapsed_s = time.monotonic() - epoch          # master time in seconds
        time_error_us, rcc_icscr = motor.time_sync(elapsed_s)
        print(f"host clock {elapsed_s:>9.6f} s | motor clock error {time_error_us:>6} us (raw microseconds)")
        time.sleep(SYNC_INTERVAL)
finally:
    try:
        motor.disable_mosfets()
    except Exception:
```

```
        pass
    servomotor.close_serial_port()
```

Get product specs

Get two product constants needed for motion math: the update frequency in Hz (the rate of the control loop that runs the hall sensor position, movement, PID, and safety calculations) and the number of position counts per one shaft rotation. One time step, as used by 'Move with acceleration', 'Move with velocity', and the other movement commands, is exactly 1/updateFrequency seconds (31250 Hz, i.e. 32 microseconds, on current firmware). countsPerRotation is product-specific (3276800 for M3/M17/M23, 639744 for M1, 4569600 for M2), so always query it rather than hardcoding a value. Sending any payload bytes raises fatal error 51, ERROR_COMMAND_SIZE_WRONG.

Returns:
• *updateFrequency*: u32: Update frequency in Hz. This is how often the motor executes all calculations for hall sensor position, movement, PID loop, safety, etc. One time step, as used by the movement commands, is exactly 1/updateFrequency seconds (31250 Hz, i.e. 32 microseconds, on current firmware).
• *countsPerRotation*: u32: Counts per rotation. When commanding the motor or when reading back position, this is the number of counts per one shaft rotation. The value is product-specific (3276800 for M3/M17/M23, 639744 for M1, 4569600 for M2), so query it rather than hardcoding.

Example program:

```
#!/usr/bin/env python3
"""
Example: Get product specs -- read the two constants that define the motor's motion math.
Prints the control-loop update frequency (Hz) and the position counts per one shaft
rotation. Expect e.g. 31250 Hz and 3276800 counts/rotation on an M17.
"""
import time
import servomotor

ALIAS = 'X'                             # Device alias; change if needed
SERIAL_PORT = "/dev/tty.usbserial-110"  # Change to your serial port (e.g. "COM3" on Windows).
                                        #  Set SERIAL_PORT = "MENU" to be prompted interactively.
servomotor.set_serial_port(SERIAL_PORT)
servomotor.open_serial_port()
motor = servomotor.M3(ALIAS, time_unit="seconds", position_unit="shaft_rotations",
                      velocity_unit="rotations_per_second",
                      acceleration_unit="rotations_per_second_squared", verbose=0)
try:
    motor.system_reset()
    time.sleep(1.5)                     # mandatory: keep the bus silent after reset

    # Two outputs -> the wrapper returns a flat list [updateFrequency, countsPerRotation].
    update_frequency, counts_per_rotation = motor.get_product_specs()

    # Always query these rather than hardcoding: countsPerRotation is product-specific
    # (3276800 on M3/M17/M23 but 639744 on M1, 4569600 on M2), and one motion "timestep"
    # is exactly 1/updateFrequency seconds.
    print(f"Update frequency:    {update_frequency} Hz")
    print(f"One timestep:        {1.0 / update_frequency * 1e6:.1f} microseconds")
    print(f"Counts per rotation: {counts_per_rotation} counts per shaft rotation")
finally:
    try:
        motor.disable_mosfets()
    except Exception:
        pass
    servomotor.close_serial_port()
```

Detect devices

Detect all devices connected on the RS485 interface. Normally sent to the broadcast address (255); every device replies exactly once with its unique ID and current alias, each after its own pseudo-random delay of 0 to 950 ms, so listen for the full one-second window. A collision between replies is possible but unlikely; repeat the command if devices you expect were not discovered. Even when addressed to one specific device by alias or unique ID, the reply still arrives only after the random delay. Warning: for about 1 second after receiving this command, a device silently discards every incoming packet (no error, no response, nothing queued), so wait at least about 1.1 seconds after sending it before transmitting anything else, including a repeated 'Detect devices'. Devices currently sitting in the bootloader also answer this command, so it discovers devices in either mode.

Returns:
• *uniqueId*: u64_unique_id: A unique ID (unique among all devices manufactured). The response is sent after a pseudo-random delay of between 0 and 950 milliseconds.
• *alias*: u8_alias: The current alias of the device that has this unique ID (255 if no alias is assigned).

Example program:

```
#!/usr/bin/env python3
"""
Example: Detect devices -- discover every servomotor on the RS485 bus.
Broadcast discovery needs no alias. Prints each device's 64-bit unique ID
(hex) and current alias -- expect one summary line per connected motor
(the detection helper also prints its own progress log).
"""
import servomotor

N_DETECTIONS = 3                        # Detection rounds; results are merged across rounds
                                        #  because a single round can miss a device
SERIAL_PORT = "/dev/tty.usbserial-110"  # Change to your serial port (e.g. "COM3" on Windows).
                                        #  Set SERIAL_PORT = "MENU" to be prompted interactively.
servomotor.set_serial_port(SERIAL_PORT)
servomotor.open_serial_port()
try:
    # detect_devices_iteratively() runs the full discovery protocol for us. Each round it
    # broadcasts 'System reset' and keeps the bus silent for 1.5 s (the mandatory
    # post-reset wait), flushes stray bytes, then broadcasts 'Detect devices': every
    # device answers once with its unique ID and alias, each at a random 0-950 ms offset
    # to avoid collisions -- which is why one round can miss a device and several are run.
    devices = servomotor.detect_devices_iteratively(n_detections=N_DETECTIONS)

    # GOTCHA (handled by the helper): for ~1 s after answering 'Detect devices' a device
    # silently IGNORES all bus traffic (its collision-avoidance window). The helper waits
    # out that lockout and finishes with a reset, so the bus is clean when it returns.
    print(f"\nFound {len(devices)} device(s):")
    for dev in devices:
        if dev.alias == 255:
            alias_str = "255 (no alias assigned; address it by unique ID)"
        elif 33 <= dev.alias <= 126:
            alias_str = f"{dev.alias} (ASCII '{chr(dev.alias)}')"
        else:
            alias_str = str(dev.alias)
        print(f"  Unique ID: {dev.unique_id:016X}   Alias: {alias_str}")
finally:
    servomotor.close_serial_port()
```

Set device alias

Set the device's one-byte alias, save it to nonvolatile flash, and then automatically reset the device. Valid aliases are 0 to 251 and 255; setting 255 removes the alias (255 is the broadcast address, so afterwards unique-ID extended addressing is the only way to address the device individually). Values 252 to 254 are reserved and raise fatal error 50, ERROR_BAD_ALIAS: an error packet is sent instead of the success response and the device is left with MOSFETs disabled, answering only 'Get status' and 'System reset' until reset. On success the response is transmitted first, in reply to the old address; then the settings are written to flash and the device reboots itself. It drops off the bus briefly while it saves to flash and reboots (bench-measured: with a SILENT bus it answered in application mode 0.3 s after the command; wait at least 0.5 s of bus silence before addressing it at the new alias -- continuous polling during the reboot delays recovery to over 1.2 s and risks pinning the device in its bootloader window) and comes back under the new alias with all volatile state (motion queue, position zeroing, enabled MOSFETs) lost, so pause before sending the next command. Warning: broadcasting this command assigns the same alias to every device on the bus with no response sent, and a broadcast with a reserved alias silently locks every device in the fatal-error state. The command also works while the device is in the bootloader, with the same save-and-reset behavior.

Parameters:
• *alias*: u8_alias: The alias (which is a one byte ID) ranging from 0 to 251. Values 252 to 254 are reserved and raise fatal error 50, ERROR_BAD_ALIAS. You can set it to 255, which will remove the alias; the device is then addressable individually only by its unique ID.

Example program:

```
#!/usr/bin/env python3
"""
Example: Set device alias -- change a device's one-byte alias from 'X' to 'Y' and back.
The device saves the new alias to flash and then reboots itself; each change is
verified by pinging the device at its new alias.
"""
import os
import time
import servomotor

ALIAS = 'X'                             # Current device alias; change if needed
NEW_ALIAS = 'Y'                         # Temporary alias for this demo. Valid aliases: 0-251
                                        #  (252-254 are reserved; 255 removes the alias)
REBOOT_WAIT = 1.0                       # s: flash write + automatic reboot after the command.
                                        #  Bench measurement showed the device answers at the
                                        #  new alias in application mode after ~0.3 s;
                                        #  0.5-1 s gives margin.
SERIAL_PORT = "/dev/tty.usbserial-110"  # Change to your serial port (e.g. "COM3" on Windows).
                                        #  Set SERIAL_PORT = "MENU" to be prompted interactively.
servomotor.set_serial_port(SERIAL_PORT)
servomotor.open_serial_port()
motor = servomotor.M3(ALIAS, time_unit="seconds", position_unit="shaft_rotations",
                      velocity_unit="rotations_per_second",
                      acceleration_unit="rotations_per_second_squared", verbose=0)

def verify_alias(alias):
    # A ping that echoes our exact bytes proves the device now answers at this alias.
    # use_this_alias_or_unique_id() takes the raw one-byte alias VALUE (it does not
    # convert strings the way the M3 constructor does), hence ord().
    motor.use_this_alias_or_unique_id(ord(alias))
    payload = os.urandom(10)            # 'Ping' requires exactly 10 bytes
    echoed = motor.ping(payload)
    assert bytes(echoed) == payload, "ping echo mismatch"
    print(f"Device responds correctly at alias '{alias}'")

try:
    motor.system_reset()
    time.sleep(1.5)                     # mandatory: keep the bus silent after reset

    print(f"Changing alias '{ALIAS}' -> '{NEW_ALIAS}' ...")
    motor.set_device_alias(NEW_ALIAS)
    # The device sends the success response first, then writes the alias to flash and
    # REBOOTS itself. Keep the bus silent until it is back: bench measurement showed
```

```
    # it answering at the new alias in application mode after ~0.3 s, so 0.5-1 s of
    # waiting gives margin.
```

```
    time.sleep(REBOOT_WAIT)
    verify_alias(NEW_ALIAS)

    print(f"Changing alias back '{NEW_ALIAS}' -> '{ALIAS}' ...")
    motor.set_device_alias(ALIAS)
    time.sleep(REBOOT_WAIT)             # same flash-write + reboot wait as above
    verify_alias(ALIAS)
    print("Done: alias restored to its original value.")
finally:
    try:
        motor.disable_mosfets()
    except Exception:
        pass
    servomotor.close_serial_port()
```

Get product info

Get the product information: product code (model number), firmware compatibility code, hardware version, serial number, and unique ID, returned as a 28-byte response payload. This is an immediate query with no side effects. Before a firmware upgrade, read the product code and firmware compatibility code from the target device: they are exactly the values the bootloader checks against the header of every page sent with 'Firmware upgrade', and they must match the firmware file. The command is also serviced while the device is in the bootloader; the data is stored in bootloader flash, independent of the application image. No response is sent when the command is broadcast. The command takes no payload; sending any payload bytes raises fatal error 51, ERROR_COMMAND_SIZE_WRONG, which disables the MOSFETs and locks the device until 'System reset'.

Returns:
• *productCode*: string8: The product code / model number (when doing a firmware upgrade, this must match between the firmware file and the target device).
• *firmwareCompatibility*: u8: A firmware compatibility code (when doing a firmware upgrade, this must match between the firmware file and the target device).
• *hardwareVersion*: u24_version_number: The hardware version stored as 3 bytes. The first byte is the patch version, followed by the minor and major versions.
• *serialNumber*: u32: The serial number.
• *uniqueId*: u64_unique_id: The unique ID for the product.
• *reserved*: u32: Not currently used.

Example program:

```
#!/usr/bin/env python3
"""
Example: Get product info -- read the device's identity: model, versions, serial, unique ID.
Prints all six fields. Before a firmware upgrade, the product code and firmware
compatibility code shown here must match the firmware file.
"""
import time
import servomotor

ALIAS = 'X'                               # Device alias; change if needed
SERIAL_PORT = "/dev/tty.usbserial-110"  # Change to your serial port (e.g. "COM3" on Windows).
                                          #  Set SERIAL_PORT = "MENU" to be prompted interactively.
servomotor.set_serial_port(SERIAL_PORT)
servomotor.open_serial_port()
motor = servomotor.M3(ALIAS, time_unit="seconds", position_unit="shaft_rotations",
                      velocity_unit="rotations_per_second",
                      acceleration_unit="rotations_per_second_squared", verbose=0)
try:
    motor.system_reset()
    time.sleep(1.5)                       # mandatory: keep the bus silent after reset

    # Six outputs -> a flat list. The hardware version arrives least-significant-first
    # as [patch, minor, major], so reverse it for the usual major.minor.patch display.
    product_code, fw_compat, hw_version, serial_number, unique_id, reserved = \
        motor.get_product_info()

    print(f"Product code:           {product_code.strip()}")   # 8 chars, space-padded
    print(f"Firmware compatibility: {fw_compat}")
    print(f"Hardware version:       {hw_version[2]}.{hw_version[1]}.{hw_version[0]}")
    print(f"Serial number:          {serial_number}")
    print(f"Unique ID:              {unique_id:016X}")
    print(f"Reserved (unused):      {reserved}")
finally:
    try:
        motor.disable_mosfets()
    except Exception:
        pass
    servomotor.close_serial_port()
```

Firmware upgrade

Write one 2048-byte page of application firmware to flash. This command only works while the device is running the bootloader: the main firmware silently discards it with no response, so the host just times out. To upgrade, reset the device ('System reset' or power cycle), then send any valid packet within the bootloader's 250 ms launch window; this cancels the pending application launch and holds the device in the bootloader indefinitely (confirm via 'Get status' bit 0 or 'Get firmware version'), and send 'System reset' when finished to boot the new firmware. Each page's model code and firmware compatibility code must match the device's 'Get product info' values; a mismatch, or a payload shorter than 9 bytes, is dropped with no response at all, indistinguishable from a dead device. A total payload other than 2058 bytes raises fatal error 51, ERROR_COMMAND_SIZE_WRONG; a page number outside 5 to 30 raises fatal error 52, ERROR_INVALID_FLASH_PAGE. The success response is sent only after the page is erased and burned, so pages can be streamed back-to-back, one per acknowledgment. The application image must begin with a u32 count of its 32-bit words, followed by the code, with the CRC32 of those words stored immediately after; the bootloader validates this on every boot and will not launch an image that fails, leaving the device stuck in the bootloader. Broadcasting suppresses the per-page responses but still writes flash, which allows flashing multiple identical devices at once (devices with a non-matching model code silently ignore the pages).

Parameters:
• *firmwarePage*: firmware_page: The data to upgrade one page of flash memory (2058 bytes total). Contents: the product model code (8 bytes), firmware compatibility code (1 byte), absolute flash page number (1 byte), and the page data itself (2048 bytes). Valid page numbers are 5 through 30; page 5 is the start of the application area at 0x8002800 (pages 0 to 4 hold the bootloader and page 31 holds settings). Any other page number raises fatal error 52, ERROR_INVALID_FLASH_PAGE.

Example program:

```
# There is intentionally no minimal example for 'Firmware upgrade'.
# Upgrading firmware requires correct page sequencing, model/compatibility
# checks, and CRC handling; use the supported tool instead. It is installed
# as a command by 'pip install servomotor' (version 0.12.0 and later):
#
#   upgrade_firmware -p <PORT> -a <ALIAS> <firmware_file.firmware>
#
# Address ONE motor with -a <ALIAS>. The tool's default is the broadcast
# address 255, which flashes every matching device on the bus at once but
# receives no replies, so it reports every page as written even when nothing
# was written at all.
#
# The model code and firmware compatibility code are checked by the DEVICE,
# not by the tool: the bootloader silently ignores pages whose codes do not
# match its own. Run 'Get product info' first and compare its productCode and
# firmwareCompatibility fields against the <MODEL> and scc<N> parts of the
# file name. See the firmware upgrade section of this document for the full
# procedure.
```

Get product description

Get a short product description string. The device replies immediately with a fixed, compile-time null-terminated string (in the current main firmware this is 'Servomotor'); the null terminator is included in the response. This is an immediate query with no effect on motion or device state, and it works both in the main firmware and in the bootloader. The command takes no payload; sending any payload bytes raises fatal error 51, ERROR_COMMAND_SIZE_WRONG, which disables the MOSFETs and latches until 'System reset'. When broadcast (address 255) the command produces no response at all, so it is only useful when addressed to a single device.

Returns:
• *productDescription*: string_null_term: A brief description of the product. This is a fixed compile-time string (currently 'Servomotor' in the main firmware) and is transmitted including its null terminator.

Example program:

```
#!/usr/bin/env python3
"""
Example: Get product description -- read the device's short description string.
Prints the fixed compile-time description (currently "Servomotor"). A quick,
side-effect-free way to confirm what kind of device is answering on the bus.
"""
import time
import servomotor

ALIAS = 'X'                             # Device alias; change if needed
SERIAL_PORT = "/dev/tty.usbserial-110"  # Change to your serial port (e.g. "COM3" on Windows).
                                        #  Set SERIAL_PORT = "MENU" to be prompted interactively.
servomotor.set_serial_port(SERIAL_PORT)
servomotor.open_serial_port()
motor = servomotor.M3(ALIAS, time_unit="seconds", position_unit="shaft_rotations",
                      velocity_unit="rotations_per_second",
                      acceleration_unit="rotations_per_second_squared", verbose=0)
try:
    motor.system_reset()
    time.sleep(1.5)                     # mandatory: keep the bus silent after reset

    # Single output -> the wrapper returns the bare string. The device transmits
    # the string INCLUDING its NUL terminator, so strip that before display.
    description = motor.get_product_description().rstrip('\x00')
    print(f"Product description: {description}")
finally:
    try:
        motor.disable_mosfets()
    except Exception:
        pass
    servomotor.close_serial_port()
```

Get firmware version

Get the firmware version, or the bootloader version if the device is currently running the bootloader. The reply payload is exactly five bytes: four version bytes in the order development, patch (bugfix), minor, major, followed by a dedicated one-byte flag that is 1 when the reply comes from the bootloader and 0 when it comes from the main firmware. This flag byte is not the device status bitfield returned by 'Get status'; it is a separate byte in which no other bits are ever set. When the device is in the bootloader, the four version bytes are the bootloader's own version, not the application firmware's. The command takes no payload; sending any payload bytes raises fatal error 51, ERROR_COMMAND_SIZE_WRONG. When broadcast, the command sends no response, in both the firmware and the bootloader.

Returns:
• *firmwareVersion*: u32_version_number: The version stored as 4 bytes. The first byte is the development number, then the patch (bugfix) version, followed by the minor and major versions. When the device is in the bootloader, these are the bootloader's own version numbers, not the application firmware's.
• *inBootloader*: u8: A flag that tells us if we are in the bootloader (=1) or in the regular main firmware (=0). This is a dedicated byte in which no other bits are ever set; it is not the 'Get status' status bitfield.

Example program:

```
#!/usr/bin/env python3
"""
Example: Get firmware version -- read the running firmware (or bootloader) version.
Prints the version as major.minor.patch.development plus the in-bootloader flag.
On a healthy device after reset the flag is 0 (main firmware answering).
"""
import time
import servomotor

ALIAS = 'X'                             # Device alias; change if needed
SERIAL_PORT = "/dev/tty.usbserial-110"  # Change to your serial port (e.g. "COM3" on Windows).
                                        #  Set SERIAL_PORT = "MENU" to be prompted interactively.
servomotor.set_serial_port(SERIAL_PORT)
servomotor.open_serial_port()
motor = servomotor.M3(ALIAS, time_unit="seconds", position_unit="shaft_rotations",
                      velocity_unit="rotations_per_second",
                      acceleration_unit="rotations_per_second_squared", verbose=0)
try:
    motor.system_reset()
    time.sleep(1.5)                     # mandatory: keep the bus silent after reset

    # Two outputs -> [versionBytes, inBootloader]. Version bytes arrive
    # least-significant-first: [development, patch, minor, major] -- reverse
    # them for the conventional major.minor.patch.dev display.
    version, in_bootloader = motor.get_firmware_version()
    dev, patch, minor, major = version
    print(f"Firmware version: {major}.{minor}.{patch}.{dev}")
    print(f"In bootloader:    {in_bootloader}")
    if in_bootloader:
        # The version above is then the BOOTLOADER's own version, not the application's.
        # Recover with another system_reset() followed by the 1.5 s silent wait.
        print("WARNING: device is in the bootloader; reported version is the bootloader's.")
finally:
    try:
        motor.disable_mosfets()
    except Exception:
        pass
    servomotor.close_serial_port()
```

Ping

Send a 10-byte payload and the device immediately responds with the same 10 bytes (executes at once, not queued). The payload must be EXACTLY 10 bytes: any other size raises fatal error 51, ERROR_COMMAND_SIZE_WRONG, which disables the MOSFETs until 'System reset', so a malformed ping halts the device. A broadcast ping is processed but produces no response at all, so it cannot be used for bus discovery; use 'Detect devices' for that.

Parameters:
• *pingData*: buf10: The binary data payload to send to the device. Must be exactly 10 bytes; any other length raises fatal error 51, ERROR_COMMAND_SIZE_WRONG.

Returns:
• *responsePayload*: buf10: The same 10 bytes that were sent to the device will be returned if all went well.

Example program:

```
#!/usr/bin/env python3
"""
Example: Ping -- verify the communication link by echoing 10 random bytes.
The device must return exactly the bytes sent. A correct echo proves the
port, wiring, addressing, and CRC32 framing are all working.
"""
import os
import time
import servomotor

ALIAS = 'X'                             # Device alias; change if needed
SERIAL_PORT = "/dev/tty.usbserial-110"  # Change to your serial port (e.g. "COM3" on Windows).
                                        #  Set SERIAL_PORT = "MENU" to be prompted interactively.
servomotor.set_serial_port(SERIAL_PORT)
servomotor.open_serial_port()
motor = servomotor.M3(ALIAS, time_unit="seconds", position_unit="shaft_rotations",
                      velocity_unit="rotations_per_second",
                      acceleration_unit="rotations_per_second_squared", verbose=0)
try:
    motor.system_reset()
    time.sleep(1.5)                     # mandatory: keep the bus silent after reset

    payload = os.urandom(10)            # 'Ping' requires EXACTLY 10 bytes; any other length
                                        #  trips fatal error 51 and disables the device
    echoed = motor.ping(payload)

    print(f"Sent:     {payload.hex()}")
    print(f"Received: {bytes(echoed).hex()}")
    if bytes(echoed) == payload:
        print("PASS: echo matches exactly -- the communication link is healthy.")
    else:
        print("FAIL: echo mismatch -- check wiring, serial port, alias, and CRC32 state.")
finally:
    try:
        motor.disable_mosfets()
    except Exception:
        pass
    servomotor.close_serial_port()
```

Vibrate

Cause the motor to vibrate (implemented only on the M1 product; a silent no-op on M2, M17, and M23) by rapidly alternating the open-loop PWM voltage between +50 and -50 (a fixed, hard-coded amplitude), or stop vibrating. This is implemented only on the M1 product: on M2, M17, and M23 the command does nothing but still returns a normal success response, so the caller cannot tell that it had no effect. The command executes immediately, not through the motion queue. On M1, sending it while the motion queue is not empty raises fatal error 8, ERROR_QUEUE_NOT_EMPTY; this check runs before the turn-off logic, so even sending value 0 faults if moves are queued. Turning vibration on while the motor is busy raises fatal error 19, ERROR_MOTOR_BUSY, and while vibrating the motor is marked busy, so other motion commands will fault until vibration is turned off with value 0. Turning vibration on implicitly enables the MOSFETs (which can raise fatal error 22, ERROR_CURRENT_SENSOR_FAILED), switches the control mode to open-loop position control, and overwrites the shared buffer read by 'Read multipurpose buffer'. Turning it off leaves the MOSFETs enabled and the control mode unchanged. On all products the payload must be exactly 1 byte or fatal error 51, ERROR_COMMAND_SIZE_WRONG, is raised.

Parameters:

• *vibrationLevel*: u8: Vibration control: 0 turns vibration off; any nonzero value turns it on. The value is only tested for zero versus nonzero, and the vibration amplitude is fixed in firmware, so the number does not scale intensity. Effective only on the M1 product; other products accept the command but do nothing.

Example program:

```
#!/usr/bin/env python3
"""
Example: Vibrate -- turn the motor's vibration mode on for 2 seconds, then off.
IMPORTANT: only the M1 product physically vibrates. On M17, M23, and M2 this
command is a silent no-op that still returns success, so we check the product
code first and print which case applies.
"""
import time
import servomotor

ALIAS = 'X'                             # Device alias; change if needed
SERIAL_PORT = "/dev/tty.usbserial-110"  # Change to your serial port (e.g. "COM3" on Windows).
                                        #  Set SERIAL_PORT = "MENU" to be prompted interactively.
VIBRATE_SECONDS = 2.0                   # How long to leave vibration turned on

servomotor.set_serial_port(SERIAL_PORT)
servomotor.open_serial_port()
motor = servomotor.M3(ALIAS, time_unit="seconds", position_unit="shaft_rotations",
                      velocity_unit="rotations_per_second",
                      acceleration_unit="rotations_per_second_squared", verbose=0)
try:
    motor.system_reset()
    time.sleep(1.5)                     # mandatory: keep the bus silent after reset

    # ONLY THE M1 PRODUCT VIBRATES. Other products (M17, M23, M2) accept this
    # command and reply with success, but nothing physically happens -- the
    # response alone cannot tell you, so identify the product first.
    info = motor.get_product_info()     # [productCode, fwCompat, hwVersion, serialNum, uniqueId, reserved]
    product = info[0].strip("\x00 ")    # productCode arrives as a padded 8-char string
    if product == "M1":
        print(f"Product {product}: the motor will physically vibrate now.")
    else:
        print(f"Product {product}: 'Vibrate' is a silent no-op on this product;")
        print("the commands below will still succeed, but the motor will not vibrate.")

    motor.vibrate(1)                    # nonzero = on; the amplitude is fixed in firmware,
                                        #  so the value does not scale the intensity
    print(f"Vibration ON for {VIBRATE_SECONDS:.0f} s ...")
    time.sleep(VIBRATE_SECONDS)
    motor.vibrate(0)                    # 0 = off (on M1 this leaves the MOSFETs enabled)
    print("Vibration OFF. Done.")
finally:
    try:
        motor.disable_mosfets()
    except Exception:
```

```
        pass
    servomotor.close_serial_port()
```

Identify

Identify a motor by flashing its green LED rapidly: 30 flashes at roughly 90 ms each, about 2.7 seconds total, after which the normal 1-second heartbeat blink resumes. The command executes immediately and is purely cosmetic: it does not touch the motion queue, motion state, or MOSFETs, so it is safe to send mid-move. Sending it again while flashing restarts the 30-flash sequence from the beginning. When unicast, the success response is sent after the flashing has started. Broadcasting to address 255 makes every motor on the bus flash with no replies, which is the intended way to visually identify all connected motors. The command takes no payload and there is no unique-ID parameter; any nonzero payload raises fatal error 51, ERROR_COMMAND_SIZE_WRONG.

Example program:

```
#!/usr/bin/env python3
"""
Example: Identify -- flash the green LED rapidly to visually locate one motor.
The LED flashes for about 2.7 s, then the normal 1 s heartbeat blink resumes.
The command is purely cosmetic, so the device stays fully responsive: this
example proves it by pinging mid-blink.
"""
import os
import time
import servomotor

ALIAS = 'X'                             # Device alias; change if needed
BLINK_TIME = 3.0                        # s: the flash sequence lasts ~2.7 s; watch it finish
SERIAL_PORT = "/dev/tty.usbserial-110"  # Change to your serial port (e.g. "COM3" on Windows).
                                        #  Set SERIAL_PORT = "MENU" to be prompted interactively.
servomotor.set_serial_port(SERIAL_PORT)
servomotor.open_serial_port()
motor = servomotor.M3(ALIAS, time_unit="seconds", position_unit="shaft_rotations",
                      velocity_unit="rotations_per_second",
                      acceleration_unit="rotations_per_second_squared", verbose=0)
try:
    motor.system_reset()
    time.sleep(1.5)                     # mandatory: keep the bus silent after reset

    print("Sending 'Identify' -- watch for the rapid green LED flashing (~2.7 s).")
    motor.identify()

    # Identify does not touch the motion queue, motion state, or MOSFETs, so the
    # device keeps answering commands while the LED flashes -- prove it with a ping:
    payload = os.urandom(10)            # 'Ping' requires exactly 10 bytes
    echoed = motor.ping(payload)
    if bytes(echoed) == payload:
        print("Pinged the device mid-blink: echo matches -- still fully responsive.")
    else:
        print("Pinged the device mid-blink: ECHO MISMATCH -- check the bus.")

    time.sleep(BLINK_TIME)              # let the flash sequence finish before cleanup
    print("Identification finished; the normal 1 s heartbeat blink has resumed.")
finally:
    try:
        motor.disable_mosfets()
    except Exception:
        pass
    servomotor.close_serial_port()
```

Motion Control

Trapezoid move

Travel the given signed displacement over exactly the given duration, relative to the position at the end of previously queued motion (for an absolute target use 'Go to position'). The move is queued, not immediate: it is appended to the 32-item movement queue (normally exactly 3 slots: accelerate/coast/decelerate (a normal-length zero-displacement timed dwell still takes 3 because its accelerate and decelerate segments have nonzero durations even though their acceleration is zero), but the coast item is silently dropped whenever its computed duration works out to exactly zero -- which happens for any move whose duration is even and at most twice max velocity divided by max acceleration (a short even-duration triangular move, or an equally short dwell) -- leaving the move in only 2 slots) and begins only after all previously queued items finish. The success response confirms validation and queueing only, not motion. The speed that gets validated is the profile's PEAK, not the displacement/duration average: the planner sizes a fixed ramp and then checks the peak acceleration and the peak velocity, so a move whose average is inside the limits can still be rejected. A duration too short for the distance raises fatal error 15, ERROR_ACCEL_TOO_HIGH (the derived acceleration is checked first and is always reached at or before the velocity limit); ERROR_PREDICTED_VELOCITY_TOO_HIGH applies to the acceleration-type commands rather than to this one. Other fatal errors: ERROR_QUEUE_IS_FULL, ERROR_PREDICTED_POSITION_OUT_OF_SAFETY_ZONE, ERROR_TURN_POINT_OUT_OF_SAFETY_ZONE, ERROR_MOTOR_BUSY during calibration or homing, and ERROR_COMMAND_SIZE_WRONG for any payload other than 8 bytes. A duration of 0 is rejected with fatal error 34, ERROR_PARAMETER_OUT_OF_RANGE (in firmware before 0.15.4.0 it was a silent no-op that still returned success). The profile assumes zero initial velocity: if the preceding queued motion ends at velocity v0, the actual displacement becomes the commanded value plus v0 times the duration and the move ends at v0, not at rest. Accepted even with MOSFETs disabled, in which case the commanded position advances anyway and typically trips fatal error ERROR_POSITION_DEVIATION_TOO_LARGE.

Parameters:
• *displacement*: i32: The signed relative displacement to travel, measured from the position at the end of the previously queued motion (not an absolute position). Can be positive or negative. Internally in encoder counts; counts per rotation are motor-specific (see the unit conversion file), e.g. 3276800 counts per shaft rotation on M3/M17.
• *duration*: u32: The total time over which to perform the move, including the acceleration and deceleration ramps. Internally counted in 32 microsecond timesteps (31250 per second). A duration of 0 is rejected with fatal error 34, ERROR_PARAMETER_OUT_OF_RANGE (silently ignored in firmware before 0.15.4.0).

Example program:

```
#!/usr/bin/env python3
"""
Example: Trapezoid move -- rotate the shaft +1 rotation over 2 seconds.
The displacement is RELATIVE (a signed offset from the end of previously queued
motion), not an absolute target. Watch the shaft make one full turn, then see
the position read back close to 1.0 rotations.
"""
import time
import servomotor

ALIAS = 'X'                             # Device alias; change if needed
SERIAL_PORT = "/dev/tty.usbserial-110"  # Change to your serial port (e.g. "COM3" on Windows).
                                        #  Set SERIAL_PORT = "MENU" to be prompted interactively.
servomotor.set_serial_port(SERIAL_PORT)
servomotor.open_serial_port()
motor = servomotor.M3(ALIAS, time_unit="seconds", position_unit="shaft_rotations",
                      velocity_unit="rotations_per_second",
                      acceleration_unit="rotations_per_second_squared", verbose=0)

DISPLACEMENT = 1.0                      # shaft rotations (signed, RELATIVE displacement)
DURATION = 2.0                          # seconds for the whole move, ramps included
```

```
try:
    motor.system_reset()
    time.sleep(1.5)                     # mandatory: keep the bus silent after reset

    motor.enable_mosfets()
    time.sleep(0.3)                     # settle the commutation-snap transient before zeroing
    motor.zero_position()               # known origin so the readback below is meaningful

    # Queued, not immediate: the success response only confirms validation + queueing.
    motor.trapezoid_move(DISPLACEMENT, DURATION)
    print(f"Queued a relative move of {DISPLACEMENT:+.1f} rotation(s) over {DURATION} s...")

    # Wait for completion by polling the motion queue (always sleep between polls).
    while motor.get_n_queued_items() > 0:
        time.sleep(0.05)
    time.sleep(0.2)                     # brief mechanical settling before reading position

    position = motor.get_position()
    print(f"Final position: {position:.4f} shaft rotations (expected about {DISPLACEMENT})")
finally:
    try:
        motor.disable_mosfets()
    except Exception:
```

```
        pass
    servomotor.close_serial_port()
```

Go to position

Queue a smooth absolute move. The firmware plans a trapezoid profile from the displacement and duration and adds three items (accelerate, coast, decelerate) to the 32-slot movement queue; the success response only acknowledges queueing and is sent before any motion occurs. Motion begins when earlier queue items finish. The target is planned against the predicted position at the end of everything already queued, not the current measured position, so successive calls chain correctly, but the reply never reflects actual arrival. Planning uses the maximum velocity and acceleration in effect at queue time, so send 'Set maximum velocity' and 'Set maximum acceleration' beforehand. The profile assumes the motor starts at rest: if the preceding queued motion ends at nonzero velocity (for example after 'Move with velocity') the accelerations superimpose and the motor does not stop at the requested position, so only issue this command when prior queued motion ends at rest. A duration of 0 is rejected with fatal error 34, ERROR_PARAMETER_OUT_OF_RANGE (in firmware before 0.15.4.0 it was silently dropped with a success response). Fatal errors: 19 ERROR_MOTOR_BUSY if calibration or homing is running, 17 ERROR_QUEUE_IS_FULL if fewer than 3 slots are free, 15 ERROR_ACCEL_TOO_HIGH if the duration is too short for the distance, 28 ERROR_PREDICTED_VELOCITY_TOO_HIGH, 27 ERROR_PREDICTED_POSITION_OUT_OF_SAFETY_ZONE or 26 ERROR_TURN_POINT_OUT_OF_SAFETY_ZONE if the move or its overshoot point leaves the safety limits, and 46 ERROR_MOVE_TOO_FAR if the required displacement exceeds plus or minus 2^31 counts. Broadcasting executes the move on all devices with no response, which is useful for synchronized multi-axis moves.

Parameters:
• *position*: i32: New absolute position value. The displacement is planned against the predicted position at the end of all items already in the movement queue, not the current measured position, so consecutive queued moves chain correctly.
• *duration*: u32: Time allowed for executing the move. A duration too short for the distance given the current maximum acceleration raises fatal error 15, ERROR_ACCEL_TOO_HIGH; a duration of 0 is rejected with fatal error 34, ERROR_PARAMETER_OUT_OF_RANGE (silently dropped in firmware before 0.15.4.0).

Example program:

```
#!/usr/bin/env python3
"""
Example: Go to position -- queue a smooth ABSOLUTE move to a target position.
Moves the shaft to +0.5 rotations in 2 s, then back to absolute position 0.
You should see the shaft rotate out and return, with both positions printed.
"""
import time
import servomotor

ALIAS = 'X'                             # Device alias; change if needed
SERIAL_PORT = "/dev/tty.usbserial-110"  # Change to your serial port (e.g. "COM3" on Windows).
                                        #  Set SERIAL_PORT = "MENU" to be prompted interactively.
servomotor.set_serial_port(SERIAL_PORT)

TARGET_POSITION = 0.5                   # shaft rotations (ABSOLUTE target, not a displacement)
MOVE_DURATION = 2.0                     # seconds allowed for each move

servomotor.open_serial_port()
motor = servomotor.M3(ALIAS, time_unit="seconds", position_unit="shaft_rotations",
                      velocity_unit="rotations_per_second",
                      acceleration_unit="rotations_per_second_squared", verbose=0)
try:
    motor.system_reset()
    time.sleep(1.5)                     # mandatory: keep the bus silent after reset
    motor.enable_mosfets()
    time.sleep(0.3)                     # energizing snaps the rotor to a commutation step;
                                        #  let it settle before establishing the zero
    motor.zero_position()               # make absolute position 0 mean "right here"

    motor.go_to_position(TARGET_POSITION, MOVE_DURATION)
    # The success response only confirms the move was queued, not that motion
    # finished -- poll the queue until it empties (always sleep between polls).
    while motor.get_n_queued_items() > 0:
        time.sleep(0.05)
    time.sleep(0.2)                     # brief mechanical settling before reading position
    print(f"Position after first move: {motor.get_position():.4f} rotations "
          f"(expected {TARGET_POSITION})")

    motor.go_to_position(0.0, MOVE_DURATION)   # absolute move: back to the origin
    while motor.get_n_queued_items() > 0:
        time.sleep(0.05)
    time.sleep(0.2)
    print(f"Position after second move: {motor.get_position():.4f} rotations (expected 0.0)")
finally:
    try:
```

```
        motor.disable_mosfets()
    except Exception:
        pass
    servomotor.close_serial_port()
```

Homing

Home the motor by moving until it hits a hard stop. The command queues a trapezoid move of maxDistance over maxDuration and sets the homing and motor busy status flags; the success response is sent as soon as homing starts, not when it finishes. Because the move is paced across the FULL maxDuration, the approach speed is approximately maxDistance divided by maxDuration -- a generous maxDuration is not merely a timeout, it directly makes the approach gentler (bench-verified: 2 rotations with a 5 second budget crawled at ~0.4 rotations/second). A maxDistance of 0 is accepted and simply holds the homing/busy state for the full maxDuration without motion; a maxDuration of 0 is rejected with fatal error 34. Poll 'Get status' and wait for the homing flag (bit 4) and motor busy flag (bit 6) to clear. A collision is declared when the commanded and hall-sensor positions differ by more than 50,000 encoder counts (about 0.015 shaft rotations on M17/M23); the queue is then cleared and the commanded position snaps to approximately the stall position, leaving the motor holding there in closed loop with MOSFETs still enabled. If nothing is hit, the motor travels the full maxDistance; status flags cannot distinguish the two outcomes, so compare positions afterward. Preconditions: closed-loop mode (else fatal error 13, ERROR_NOT_IN_CLOSED_LOOP), empty queue (else fatal error 8, ERROR_QUEUE_NOT_EMPTY), not busy (else fatal error 19, ERROR_MOTOR_BUSY). Normal enqueue-time planning checks apply, so a maxDuration too short for maxDistance or a move that would cross safety limits is fatal (errors 15, 28, 27, or 26). Homing does not zero the position and does not set safety limits; follow it with 'Zero position' and, if desired, 'Set safety limits'. If the max allowable position deviation has been set below 50,000 counts, a collision instead raises fatal error 45, ERROR_POSITION_DEVIATION_TOO_LARGE.

Parameters:

• *maxDistance*: i32: The maximum distance to move while searching for a hard stop (if no collision occurs, the motor travels this full distance). This can be positive or negative; the sign determines the direction of movement.

• *maxDuration*: u32: The time allotted for the homing move. Homing is queued as a trapezoid move covering maxDistance in this time, so a duration too short for the distance does not merely move the motor too fast: it raises fatal error 15, ERROR_ACCEL_TOO_HIGH, or 28, ERROR_PREDICTED_VELOCITY_TOO_HIGH, at enqueue time.

Example program:

```
#!/usr/bin/env python3
"""
Example: Homing -- find a mechanical hard stop by moving until the motor stalls against it.
Enters closed loop, lowers the motor current so it only presses gently, homes up to
2 rotations in the negative direction, restores the current, and zeroes at the stop.
"""
import time
import servomotor

ALIAS = 'X'                             # Device alias; change if needed
SERIAL_PORT = "/dev/tty.usbserial-110"  # Change to your serial port (e.g. "COM3" on Windows).
                                        #  Set SERIAL_PORT = "MENU" to be prompted interactively.
HOMING_CURRENT = 50                     # internal current units: a gentle press into the hard stop
WORKING_CURRENT = 200                   # internal current units: normal working torque
MAX_DISPLACEMENT = -2.0                 # shaft rotations to search; the SIGN sets the direction
MAX_DURATION = 5.0                      # seconds allotted for the homing move
servomotor.set_serial_port(SERIAL_PORT)
servomotor.open_serial_port()
motor = servomotor.M3(ALIAS, time_unit="seconds", position_unit="shaft_rotations",
                      velocity_unit="rotations_per_second",
                      acceleration_unit="rotations_per_second_squared",
                      current_unit="internal_current_units", verbose=0)
try:
    motor.system_reset()
    time.sleep(1.5)                     # mandatory: keep the bus silent after reset

    # Homing requires closed-loop mode. go_to_closed_loop() enables the MOSFETs
    # by itself; skipping a separate 'Enable MOSFETs' may even give a gentler
    # engagement.
    motor.go_to_closed_loop()
    deadline = time.time() + 6.0
    while (motor.get_status()[0] >> 2) & 1 == 0:    # wait for the closed-loop status bit
```

```
        if time.time() > deadline:
            raise TimeoutError("Never entered closed loop -- is the motor calibrated?")
        time.sleep(0.1)
    time.sleep(0.3)                    # settle after the closed-loop transition

    # Soft-press homing: with a reduced current limit the motor can only push
    # gently, so hitting the hard stop is harmless. Args: (motor, regeneration) current.
    motor.set_maximum_motor_current(HOMING_CURRENT, HOMING_CURRENT)
    motor.homing(MAX_DISPLACEMENT, MAX_DURATION)
    deadline = time.time() + MAX_DURATION + 2.0
    while True:                        # poll status bit 4 (homing in progress) until it clears
        status_flags, fatal_error_code = motor.get_status()
        if fatal_error_code != 0:
```

```
            motor.system_reset()       # fatal errors latch; reset is the only way out
            time.sleep(1.5)            # bus silent after reset
            raise RuntimeError(f"Fatal error {fatal_error_code} during homing")
        if (status_flags >> 4) & 1 == 0:
            break
        if time.time() > deadline:
            raise TimeoutError("Homing did not finish in time")
        time.sleep(0.1)

    # If no obstacle was hit, the motor traveled the full MAX_DISPLACEMENT --
    # only the traveled distance tells the two outcomes apart.
    print(f"Homing done; traveled {motor.get_position():.3f} rotations from the start")

    motor.set_maximum_motor_current(WORKING_CURRENT, WORKING_CURRENT)
    motor.zero_position()              # homing does NOT zero -- establish the origin ourselves
    print(f"Origin set at the hard stop; position now {motor.get_position():.4f} rotations")
finally:
    try:
        motor.disable_mosfets()
    except Exception:
        pass
    servomotor.close_serial_port()
```

Go to closed loop

Enter closed-loop position control mode. This command executes immediately (it is not queued) but has two preconditions: the movement queue must be empty, otherwise fatal error 8, ERROR_QUEUE_NOT_EMPTY, is raised, and the motor must not be busy, otherwise fatal error 19, ERROR_MOTOR_BUSY, is raised; on either fatal error the device replies with an error packet instead of a success response and stays in the fatal-error state until reset. The command automatically enables the MOSFETs if they are disabled (no separate 'Enable MOSFETs' is needed); on M1, M2, and M23 that step runs a current-sensor baseline check which can raise fatal error 22, ERROR_CURRENT_SENSOR_FAILED, while M17 performs no current-sensor check. It loads the commutation offset from saved settings without verifying that calibration was ever performed, so run 'Start calibration' at least once on a new unit. Completion semantics differ by product: on M2, M17, and M23 the transition is synchronous, so receiving the success response means the motor is already in closed loop (confirm via 'Get status' bit 2); on legacy M1 the success response only means an asynchronous procedure started, completion is indicated by status bit 5 clearing and bit 2 setting, and a failed attempt raises fatal error 39, ERROR_GO_TO_CLOSED_LOOP_FAILED. Calling it while already in closed loop is harmless on non-M1 products (given an empty queue and a non-busy motor). Any payload bytes raise fatal error 51, ERROR_COMMAND_SIZE_WRONG.

Example program:

```
#!/usr/bin/env python3
"""
Example: Go to closed loop -- switch the motor into closed-loop position control.
Enters closed-loop mode (requires a motor that has been calibrated once with
'Start calibration'), then performs a small move. Watch the shaft turn 1 rotation.
"""
import time
import servomotor

ALIAS = 'X'                             # Device alias; change if needed
SERIAL_PORT = "/dev/tty.usbserial-110"  # Change to your serial port (e.g. "COM3" on Windows).
                                        #  Set SERIAL_PORT = "MENU" to be prompted interactively.
servomotor.set_serial_port(SERIAL_PORT)
servomotor.open_serial_port()
motor = servomotor.M3(ALIAS, time_unit="seconds", position_unit="shaft_rotations",
                      velocity_unit="rotations_per_second",
                      acceleration_unit="rotations_per_second_squared", verbose=0)
try:
    motor.system_reset()
    time.sleep(1.5)                     # mandatory: keep the bus silent after reset
    motor.go_to_closed_loop()           # enables the MOSFETs by itself -- no separate
                                        #  'Enable MOSFETs' needed; skipping it may even
                                        #  give a gentler engagement
    deadline = time.time() + 6.0
    while True:
        status_flags, fatal_error_code = motor.get_status()
        if status_flags & (1 << 2):     # bit 2 = closed-loop mode active
            break
        if time.time() > deadline:
            raise TimeoutError("Never entered closed loop -- is the motor calibrated?")
        time.sleep(0.1)
    print("Motor is now in closed-loop mode")
    time.sleep(0.3)                     # settle after the mode change so the zero is clean
    motor.zero_position()
    motor.trapezoid_move(1.0, 2.0)      # small safe demo move: 1 rotation in 2 seconds
    while motor.get_n_queued_items() > 0:
        time.sleep(0.05)                # poll the queue; never busy-poll without a sleep
    time.sleep(0.2)                     # brief mechanical settling before reading position
    print(f"Move complete; position = {motor.get_position():.3f} shaft rotations")
finally:
    try:
        motor.disable_mosfets()
    except Exception:
        pass
    servomotor.close_serial_port()
```

Move with acceleration

Queue a constant-acceleration segment: the acceleration is applied once per time step for the given number of time steps. This is a queued command; the success response only confirms the item entered the 32-item motion queue shared with 'Trapezoid move', 'Go to position', 'Move with velocity', and 'Multimove', and it executes after all previously queued items finish. WARNING: if the queue empties while velocity is non-zero, fatal error 18, ERROR_RUN_OUT_OF_QUEUE_ITEMS, is raised on the very next control tick, so every plan must end at zero net velocity (for example a mirror-image deceleration item) or be continuously fed with follow-up queue items. Enqueue-time validation can raise fatal errors 15 ERROR_ACCEL_TOO_HIGH, 28 ERROR_PREDICTED_VELOCITY_TOO_HIGH, 27 ERROR_PREDICTED_POSITION_OUT_OF_SAFETY_ZONE, 26 ERROR_TURN_POINT_OUT_OF_SAFETY_ZONE, 17 ERROR_QUEUE_IS_FULL, and 19 ERROR_MOTOR_BUSY; an error packet is sent instead of the success response on any fatal error and the device is left with MOSFETs disabled, answering only 'Get status' and 'System reset' until reset. This command does not enable the MOSFETs; with them disabled the profile still executes internally and the commanded position advances with no physical motion, so use 'Enable MOSFETs' or 'Go to closed loop' first. A timeSteps value of 0 is rejected with fatal error 34, ERROR_PARAMETER_OUT_OF_RANGE (silently ignored in firmware before 0.15.4.0). Broadcast queues the move on every listening motor with no response, useful for synchronized multi-axis starts. The payload must be exactly 8 bytes or fatal error 51, ERROR_COMMAND_SIZE_WRONG, is raised.

Parameters:

• *acceleration*: i32: The acceleration. The wire value is the acceleration in counts per time step per time step multiplied by 2^24 (a fixed-point scale factor). It is applied once per time step; one time step is 1/updateFrequency seconds as reported by 'Get product specs'.

• *timeSteps*: u32: The number of time steps to apply this acceleration; query 'Get product specs' for the time step frequency. After this many time steps the acceleration ends, but the reached velocity is NOT held indefinitely: if the movement queue is empty when this item finishes at non-zero velocity, fatal error 18, ERROR_RUN_OUT_OF_QUEUE_ITEMS, is raised on the next control tick. End the plan at zero velocity or keep the queue fed. A value of 0 is rejected with fatal error 34, ERROR_PARAMETER_OUT_OF_RANGE (silently ignored in firmware before 0.15.4.0).

Example program:

```
#!/usr/bin/env python3
"""
Example: Move with acceleration -- queue constant-acceleration segments.
Queues +2 rot/s^2 for 1 s (speed up), 0 for 1 s (coast at 2 rot/s), and
-2 rot/s^2 for 1 s (slow down) back-to-back, waits for the queue to drain,
then prints the final position (expect ~4 shaft rotations).
"""
import time
import servomotor

ALIAS = 'X'                             # Device alias; change if needed
SERIAL_PORT = "/dev/tty.usbserial-110"  # Change to your serial port (e.g. "COM3" on Windows).
                                        #  Set SERIAL_PORT = "MENU" to be prompted interactively.
ACCELERATION = 2.0                      # rotations/second^2
SEGMENT_TIME = 1.0                      # seconds per segment

servomotor.set_serial_port(SERIAL_PORT)
servomotor.open_serial_port()
motor = servomotor.M3(ALIAS, time_unit="seconds", position_unit="shaft_rotations",
                      velocity_unit="rotations_per_second",
                      acceleration_unit="rotations_per_second_squared", verbose=0)
try:
    motor.system_reset()
    time.sleep(1.5)                     # mandatory: keep the bus silent after reset
    motor.enable_mosfets()
    time.sleep(0.3)                     # let the rotor settle on a commutation step
    motor.zero_position()

    # Queue the whole plan back-to-back, BEFORE the first segment finishes.
    # An acceleration segment does not stop by itself: the motor keeps the
    # velocity it reached, and if the queue empties while velocity is nonzero
```

```
    # the firmware raises fatal error 18 (ERROR_RUN_OUT_OF_QUEUE_ITEMS) and
    # disables itself until reset. The mirrored -2 rot/s^2 segment brings the
    # velocity back to exactly zero before the queue runs dry.
    motor.move_with_acceleration(ACCELERATION, SEGMENT_TIME)    # 0 -> 2 rot/s
    motor.move_with_acceleration(0.0, SEGMENT_TIME)             # coast at 2 rot/s
    motor.move_with_acceleration(-ACCELERATION, SEGMENT_TIME)   # 2 rot/s -> rest

    while motor.get_n_queued_items() > 0:   # wait for the whole plan to execute
        time.sleep(0.05)
    time.sleep(0.2)                     # brief mechanical settling before reading

    position = motor.get_position()
    print(f"Final position: {position:.3f} shaft rotations (expected ~4.0)")
finally:
```

```
    try:
        motor.emergency_stop()          # clears queue + disables MOSFETs, safe mid-motion
    except Exception:
        pass
    servomotor.close_serial_port()
```

Move with velocity

Rotate the motor at a constant velocity for a fixed duration. This is a queued command: it is appended to the 32-entry motion queue and executes only after all previously queued items finish; the success response acknowledges enqueueing, not motion, so the next command can be sent immediately. When the item executes, the velocity is applied as an instantaneous step (no ramp) and held constant for exactly the given duration. WARNING: if the queue is empty when the duration expires and the velocity is nonzero, the device raises fatal error 18, ERROR_RUN_OUT_OF_QUEUE_ITEMS, disabling the MOSFETs until 'System reset'; always queue a follow-up item that ends at zero velocity (for example 'Move with velocity' with velocity 0, or a decelerating 'Move with acceleration') before the duration expires. Enqueueing itself can instead raise fatal error 19, ERROR_MOTOR_BUSY (calibration, homing, or go-to-closed-loop in progress), 16, ERROR_VEL_TOO_HIGH, 27, ERROR_PREDICTED_POSITION_OUT_OF_SAFETY_ZONE, or 17, ERROR_QUEUE_IS_FULL. A duration of zero is rejected with fatal error 34, ERROR_PARAMETER_OUT_OF_RANGE (in firmware before 0.15.4.0 it was silently ignored with a success response). The queue is consumed even while the MOSFETs are disabled, so error 18 can fire with the motor unpowered. A commanded velocity within the configured limits can still exceed what the motor can physically reach under its operating conditions. Reported unloaded bench speeds on 40 units were 8.59-8.65 rotations/second at both 20 V and 24 V; these are observations under those test conditions, not a universal speed ceiling. The commanded position can run ahead of the rotor without a position-deviation error while the deviation stays inside the allowed limit. In closed loop the motor can keep moving AFTER the queue empties as it attempts to reach the commanded end position -- an empty queue therefore does not mean motion has finished. Exceeding the configured position-deviation limit triggers fatal error 45. One timestep is 32 microseconds (31250 timesteps per second). The velocity field is a SIGNED 32-bit wire value, which caps the largest commandable velocity at about 19.5 rotations/second in either direction -- values beyond that are rejected by client libraries before transmission.

Parameters:
• *velocity*: i32: The velocity. The raw wire value is the desired velocity in microsteps (counts) per timestep multiplied by 2^20, i.e. 12.20 fixed point; one timestep is 32 microseconds (31250 per second). A magnitude exceeding the configured maximum velocity raises fatal error 16, ERROR_VEL_TOO_HIGH.
• *duration*: u32: The time to maintain this velocity, in timesteps of 32 microseconds (31250 per second). A value of 0 is rejected with fatal error 34, ERROR_PARAMETER_OUT_OF_RANGE (silently ignored in firmware before 0.15.4.0).

Example program:

```
#!/usr/bin/env python3
"""
Example: Move with velocity -- spin at a constant velocity for a fixed time.
Spins the shaft at 1 rotation/second for 2 seconds, stops with a queued
zero-velocity segment, waits for the queue to drain, then prints the final
position (expect ~2 shaft rotations).
"""
import time
import servomotor

ALIAS = 'X'                             # Device alias; change if needed
SERIAL_PORT = "/dev/tty.usbserial-110"  # Change to your serial port (e.g. "COM3" on Windows).
                                        #  Set SERIAL_PORT = "MENU" to be prompted interactively.
VELOCITY = 1.0                          # rotations/second
MOVE_TIME = 2.0                         # seconds
STOP_TIME = 0.1                         # seconds (duration of the zero-velocity stop segment)

servomotor.set_serial_port(SERIAL_PORT)
servomotor.open_serial_port()
motor = servomotor.M3(ALIAS, time_unit="seconds", position_unit="shaft_rotations",
                      velocity_unit="rotations_per_second",
                      acceleration_unit="rotations_per_second_squared", verbose=0)
try:
    motor.system_reset()
    time.sleep(1.5)                     # mandatory: keep the bus silent after reset
    motor.enable_mosfets()
    time.sleep(0.3)                     # let the rotor settle on a commutation step
    motor.zero_position()

    motor.move_with_velocity(VELOCITY, MOVE_TIME)   # 1 rot/s for 2 s
    # ------------------------------------------------------------------
    # MANDATORY STOP SEGMENT: a velocity move does NOT stop on its own.
    # When the 2 s expire the motor keeps the last commanded velocity, and
    # if the queue is empty at that instant the firmware raises fatal
    # error 18 (ERROR_RUN_OUT_OF_QUEUE_ITEMS) and disables itself until
    # reset. Queue the zero-velocity segment back-to-back with the moving
    # one, so it is already waiting in the queue when the move time expires.
    # ------------------------------------------------------------------
    motor.move_with_velocity(0.0, STOP_TIME)

    while motor.get_n_queued_items() > 0:   # wait for both segments to execute
        time.sleep(0.05)
    time.sleep(0.2)                     # brief mechanical settling before reading

    position = motor.get_position()
```

```
    print(f"Final position: {position:.3f} shaft rotations (expected ~2.0)")
finally:
    try:
        motor.emergency_stop()          # clears queue + disables MOSFETs, safe mid-motion
    except Exception:
        pass
    servomotor.close_serial_port()
```

Multimove

Enqueue up to 32 moves in one command; each move is an acceleration segment or an instant velocity change (selected per move by the moveTypes bitmask) executed for its given number of time steps. Two verified gotchas: an entry with 0 time steps is DROPPED SILENTLY and consumes no queue slot (unlike standalone move commands, which reject zero durations with fatal error 34 since firmware 0.15.4.0), and an acceleration-type entry with value 0 MAINTAINS the current velocity rather than stopping -- a trailing stop entry must be a velocity-type 0, or the plan ends at nonzero velocity and raises fatal error 18 when the queue empties. The success response only acknowledges that the moves were validated and enqueued, not that any motion completed. Moves share the single 32-entry motion queue with 'Trapezoid move', 'Go to position', 'Move with velocity', and 'Move with acceleration'; each consumes one queue spot except zero-time-step moves, which are silently discarded. Values use the same internal units as 'Move with acceleration' and 'Move with velocity'. Each move is validated at receive time and can raise fatal error 19 ERROR_MOTOR_BUSY, 15 ERROR_ACCEL_TOO_HIGH, 16 ERROR_VEL_TOO_HIGH, 28 ERROR_PREDICTED_VELOCITY_TOO_HIGH, 27 ERROR_PREDICTED_POSITION_OUT_OF_SAFETY_ZONE, 26 ERROR_TURN_POINT_OUT_OF_SAFETY_ZONE, or 17 ERROR_QUEUE_IS_FULL. If the queue empties while the commanded velocity is nonzero, the device raises fatal error 18, ERROR_RUN_OUT_OF_QUEUE_ITEMS; on emptying it also cross-checks predicted versus actual position and can raise fatal error 42, ERROR_POSITION_DISCREPANCY. A packet claiming more than 32 moves is rejected with fatal error 24, ERROR_MULTIMOVE_MORE_THAN_32_MOVES, before any move data is copied (firmware 0.15.4.0 and later; WARNING: firmware before 0.15.4.0 corrupted device memory before this error could fire).

Parameters:

• *moveCount*: u8: Specify how many moves are being communicated in this one shot. Must be 32 or fewer: a packet claiming more than 32 moves is rejected with fatal error 24, ERROR_MULTIMOVE_MORE_THAN_32_MOVES, before any move data is copied into the device buffer (the count has been validated ahead of the copy since firmware 0.10.0.0; only firmware older than that copied first and could corrupt device memory).

• *moveTypes*: u32: Bitmask selecting the type of each move, LSB first: bit i governs move i, where bit = 0 means a move with acceleration and bit = 1 means a move with velocity (same semantics as 'Move with acceleration' and 'Move with velocity').

• *moveList*: list_2d: A 2D list in Python format (list of lists). Each item is of type [i32, u32]: the first value is the acceleration to move at or the velocity to instantly change to (according to the moveTypes bits), the second is the number of time steps over which that move is executed. For example: '[[100, 30000], [-200, 60000]]'. Values use the same internal scaling as 'Move with acceleration' and 'Move with velocity'. There is a limit of 32 moves per command. Each move takes one spot in the shared 32-entry motion queue, except moves with 0 time steps, which are silently discarded and take no spot; make sure there is enough free queue space to store all of the moves.

Example program:

```
#!/usr/bin/env python3
"""
Example: Multimove -- queue several mixed velocity/acceleration moves in one packet.
Sends 3 moves in one command: jump to 1 rot/s for 1 s, decelerate at
-1 rot/s^2 for 1 s (back to standstill), then hold velocity 0 briefly.
The shaft turns ~1.5 rotations and comes smoothly to rest.
"""
import time
import servomotor
from servomotor import communication

ALIAS = 'X'                             # Device alias; change if needed
SERIAL_PORT = "/dev/tty.usbserial-110"  # Change to your serial port (e.g. "COM3" on Windows).
                                        #  Set SERIAL_PORT = "MENU" to be prompted interactively.

# Multimove takes RAW INTERNAL units, so we convert by hand using the factors
# published in servomotor/unit_conversions_M3.json:
TIMESTEPS_PER_SECOND = 31250            # internal time unit: 1 timestep = 32 us
VELOCITY_FACTOR = 109951162.7776        # internal velocity units per rotation/second
ACCELERATION_FACTOR = 56294.9953421312  # internal accel units per rotation/second^2
```

```
servomotor.set_serial_port(SERIAL_PORT)
servomotor.open_serial_port()
motor = servomotor.M3(ALIAS, time_unit="seconds", position_unit="shaft_rotations",
                      velocity_unit="rotations_per_second",
                      acceleration_unit="rotations_per_second_squared", verbose=0)
try:
    motor.system_reset()
    time.sleep(1.5)                     # mandatory: keep the bus silent after reset
    motor.enable_mosfets()
    time.sleep(0.3)                     # let the rotor settle on a commutation step
    motor.zero_position()

    vel = round(1.0 * VELOCITY_FACTOR)          # 1 rotation/second
    acc = round(-1.0 * ACCELERATION_FACTOR)     # -1 rotation/second^2 (deceleration)
    t_1s = round(1.0 * TIMESTEPS_PER_SECOND)    # 1 second
    t_01s = round(0.1 * TIMESTEPS_PER_SECOND)   # 0.1 seconds

    # moveTypes is a bitmask, LSB = first move: bit=1 -> velocity move,
    # bit=0 -> acceleration move. 0b101 = moves 0 and 2 are velocity moves,
    # move 1 is an acceleration move. The plan MUST end at zero velocity:
    # if the queue empties at nonzero velocity the firmware raises fatal
    # error 18 (ERROR_RUN_OUT_OF_QUEUE_ITEMS) and disables itself until reset.
    move_types = 0b101
    move_list = f"[[{vel}, {t_1s}], [{acc}, {t_1s}], [0, {t_01s}]]"
```

```
    # The high-level M3.multimove() wrapper cannot convert the mixed
    # velocity/acceleration/time list, so drive command 29 through the
    # library's low-level execute_command with internal-unit values.
    communication.execute_command(29, [3, move_types, move_list],
                                  alias_or_unique_id=motor.alias_or_unique_id,
                                  verbose=0)

    while motor.get_n_queued_items() > 0:   # wait for all 3 queued moves to execute
        time.sleep(0.05)
    time.sleep(0.2)                     # brief mechanical settling before reading

    position = motor.get_position()
    print(f"Final position: {position:.3f} shaft rotations (expected ~1.5)")
finally:
    try:
        motor.disable_mosfets()
    except Exception:
        pass
    servomotor.close_serial_port()
```

Set immediate position target

Set and hold a position target without queueing a timed move. Requires an empty queue, enabled MOSFETs and closed-loop position control. Preserves PID history and protections. Invalid or busy requests raise fatal errors. Success confirms application, not arrival. No velocity or acceleration profile. Broadcasts execute silently.

Parameters:
• *targetPosition*: i64: Signed position target relative to the current session zero, in encoder counts on the wire.

Example:

```
targetPosition_value = 0  # Replace with your desired value

motor.set_immediate_position_target(
    targetPosition=targetPosition_value
)
```

Set immediate position target and get position

Set an immediate position target using command 49's requirements and return the latest stored Hall position. No new measurement or control-tick wait. Broadcasts execute silently.

Parameters:
• *targetPosition*: i64: Signed position target relative to the current session zero, in encoder counts on the wire.

Returns:
• *measuredPosition*: i64: Latest stored Hall position, read after applying the target.

Example:

```
targetPosition_value = 0  # Replace with your desired value

measuredPosition = motor.set_immediate_position_target_and_get_position(
    targetPosition=targetPosition_value
)
print(f"measuredPosition: {measuredPosition}")
```

Other

Capture hall sensor data

Capture hall-sensor data and stream it back. This command is fully implemented: it starts a sampling engine in the motor-control interrupt and returns exactly one extended-size response packet whose payload bytes arrive incrementally over the whole capture duration (nPointsToRead times timeStepsPerSample times nSamplesToSum time steps), with a single CRC32 at the end if CRC is enabled, so keep reading one packet until it completes. Warning: the device ignores all incoming packets until the capture finishes; there is no way to abort over the bus and only completion or a power cycle restores command processing, so queue any needed motion before sending this command. A fatal error during the capture (for example error 18 from a queued move that underruns mid-capture) kills the sampling engine and truncates the stream -- the host's read times out; send 'Get status' afterward to discover the latched code. Make sure all queued motion ends at rest before capturing. Unlike other commands it responds even when broadcast, so broadcasting with multiple devices on the bus causes collisions. All numeric parameters must be nonzero; invalid values raise fatal error 49, ERROR_CAPTURE_BAD_PARAMETERS, halting the device, which replies with an error packet (when addressed non-broadcast) and stays in the fatal-error state until 'System reset'. The total payload, nPointsToRead times the number of enabled channels times 2 bytes, must not exceed 65526 bytes or fatal error 20, ERROR_CAPTURE_PAYLOAD_TOO_BIG, is raised. If sampling outruns the 230400-baud transmit drain the device halts with fatal error 21, ERROR_CAPTURE_OVERFLOW, so choose timeStepsPerSample times nSamplesToSum accordingly. For position capture (type 2) also mind ALIASING: each sample is a 16-bit value that wraps, so the position change between consecutive samples must stay below 32768 raw units or the unwrapped trace reads falsely slow -- at high speeds sample fast (e.g. timeStepsPerSample 2, nSamplesToSum 2 = 128 microseconds/sample tracks ~10 rotations/second safely). The raw position unit is not encoder counts; calibrate it against a known move. Sampling is skipped entirely while homing or calibration is active.

Parameters:
• *captureType*: u8: Type of data to capture: 1 = raw hall-sensor ADC readings (3 channels), 2 = fused hall position (a single value placed in channel slot 0 only, so use a channel bitmask of 1), 3 = midline-adjusted hall readings. Any other value, including 0, raises fatal error 49, ERROR_CAPTURE_BAD_PARAMETERS; there is no stop or abort value.
• *nPointsToRead*: u32: Number of points to read back from the device. Must be nonzero, and nPointsToRead times the number of enabled channels times 2 bytes must not exceed 65526 (for example at most 10921 points with 3 channels or 32763 with 1).
• *channelsToCaptureBitmask*: u8: Channels to capture bitmask. Bits 0 to 2 enable hall sensors 1 to 3 respectively (bit value 1 enables the channel). The mask must be nonzero and bits 3 to 7 must be zero, otherwise fatal error 49, ERROR_CAPTURE_BAD_PARAMETERS. For captureType 2 only channel slot 0 receives data, so use a bitmask of 1; enabling higher bits just pads each point with zeros.
• *timeStepsPerSample*: u16: Acquire one sample every this many time steps. Time steps happen at the update frequency, which can be read with the 'Get product specs' command. Must be nonzero.
• *nSamplesToSum*: u16: Number of samples to sum together to make one point to transmit back. Must be nonzero. Together with timeStepsPerSample this sets the per-point period, which must be long enough for each point to transmit at 230400 baud before the next completes, or the device halts with fatal error 21, ERROR_CAPTURE_OVERFLOW.
• *divisionFactor*: u16: Division factor applied to each point's sample sum before transmission. Must be nonzero. The result is silently truncated to 16 bits, so choose it such that the maximum sample value times nSamplesToSum divided by divisionFactor is at most 65535.

Returns:
• *data*: general_data: The captured data: nPointsToRead points, each point containing one u16 per enabled channel in ascending bit order, where each u16 is the sum of nSamplesToSum samples divided by divisionFactor and truncated to 16 bits. Delivered as a single extended-size response packet whose bytes arrive incrementally over the capture duration.

Example program:

```
#!/usr/bin/env python3
"""
Example: Capture hall sensor data -- record the three raw hall-sensor ADC channels during a move.
Starts a slow 1-rotation move, then captures 500 points from all three hall sensors while the
shaft turns. You should see the motor rotate slowly and the first captured points printed as
three ADC values per point.
"""
import time
import servomotor

ALIAS = 'X'                             # Device alias; change if needed
SERIAL_PORT = "/dev/tty.usbserial-110"  # Change to your serial port (e.g. "COM3" on Windows).
                                        #  Set SERIAL_PORT = "MENU" to be prompted interactively.
servomotor.set_serial_port(SERIAL_PORT)

MOVE_ROTATIONS = 1.0                    # shaft rotations (small, safe motion)
MOVE_DURATION = 5.0                     # seconds; slow enough that the capture happens mid-move
CAPTURE_TYPE = 1                        # 1 = raw hall-sensor ADC readings (3 channels)
N_POINTS = 500                          # points to read back
CHANNEL_BITMASK = 7                     # bits 0-2 set = capture all three hall sensors
TIME_STEPS_PER_SAMPLE = 8               # one sample every 8 time steps (1 time step = 32 us)
N_SAMPLES_TO_SUM = 16                   # samples summed into one transmitted point
DIVISION_FACTOR = 8                     # sample sum is divided by this before transmission

servomotor.open_serial_port()
motor = servomotor.M3(ALIAS, time_unit="seconds", position_unit="shaft_rotations",
                      velocity_unit="rotations_per_second",
                      acceleration_unit="rotations_per_second_squared", verbose=0)
try:
    motor.system_reset()
    time.sleep(1.5)                     # mandatory: keep the bus silent after reset
    motor.enable_mosfets()
    time.sleep(0.3)                     # energizing snaps the rotor to a step; let it settle
    motor.zero_position()
    # Generous deviation limit: the device ignores ALL bus traffic until the capture
    # finishes, so nothing must be able to trip a fatal error while we cannot intervene.
    motor.set_max_allowable_position_deviation(100)   # shaft rotations

    # Queue the motion BEFORE capturing: there is no way to send commands (or abort)
    # once the capture starts.
    motor.trapezoid_move(MOVE_ROTATIONS, MOVE_DURATION)
    time.sleep(0.2)                     # let the motor accelerate to a steady velocity

    # This call blocks while the response bytes stream in over the whole capture:
    # 500 points * 8 time steps * 16 sums * 32 us = about 2 s here.
```

```
    data = motor.capture_hall_sensor_data(CAPTURE_TYPE, N_POINTS, CHANNEL_BITMASK,
                                          TIME_STEPS_PER_SAMPLE, N_SAMPLES_TO_SUM,
                                          DIVISION_FACTOR)
    print(f"Received {len(data)} bytes = {len(data) // 6} points x 3 channels x 2 bytes")
    for i in range(0, 10 * 6, 6):       # decode the first 10 points (little-endian u16 triplets)
        h1 = int.from_bytes(data[i + 0:i + 2], "little")
        h2 = int.from_bytes(data[i + 2:i + 4], "little")
        h3 = int.from_bytes(data[i + 4:i + 6], "little")
        print(f"point {i // 6}: hall1={h1}  hall2={h2}  hall3={h3} (ADC units)")

    while motor.get_n_queued_items() > 0:   # let the queued move finish before cleanup
        time.sleep(0.05)
finally:
    try:
        motor.disable_mosfets()
    except Exception:
        pass
    servomotor.close_serial_port()
```

Read multipurpose buffer

Read out and clear the multipurpose data buffer, which holds at most one dataset at a time. On current products (M3, M17, M23) it is filled only via 'Test mode': mode 3 stores a 20-byte PID debug snapshot (five packed i32: error, P term, I term, D term, output). Mode 3 is CONTINUOUS: it stores whenever the buffer is empty AND the PID is running (closed loop), so each read is immediately followed by a fresh snapshot -- back-to-back reads return new data, never the empty tag; in open loop the buffer stays empty. Mode 4 (GC6609 driver register dump) is DEFUNCT on current hardware: the GC6609 chip is used only on legacy M17 units (software compatibility code 1); on current units the code is not compiled in, so mode 4 stores nothing (the buffer stays empty and the test mode stays latched until reset). Also, switching to mode 4 while mode 3 is active can corrupt a pending dataset (tag-only response) -- use one test mode at a time and read the buffer before switching. Go-to-closed-loop and vibrate datasets are produced only on product M1. Contrary to older documentation, calibration never fills this buffer, and 'Capture hall sensor data' streams its data in its own response instead. The first byte of the returned data identifies the contents (2 = go-to-closed-loop, 3 = vibrate, 4 = PID debug, 5 = GC6609 registers), followed by the raw bytes. A successful read clears the buffer, so each dataset can be read exactly once. If the buffer is empty the device replies with a single data-type byte of 0 as of firmware 0.15.4.0; in older firmware it transmitted nothing at all, so the client timed out. A non-empty reply uses the extended-size packet format (first size byte 255, then a u16 total packet size), allowing up to 65535 bytes; the empty-buffer reply (the single 0 byte) arrives as a normal short-format packet. Broadcast produces no response and does not clear the buffer.

Returns:

• *bufferData*: general_data: The buffer contents: a leading data type byte (2 = go-to-closed-loop data, M1 only; 3 = vibrate data, M1 only; 4 = PID debug data, five packed i32: error, P, I, D, output; 5 = GC6609 register dump; 1 is defined for calibration but never produced) followed by the raw data bytes; length depends on the dataset. If the buffer is empty, the device replies with a single data-type byte of 0 as of firmware 0.15.4.0 (older firmware sent no response at all, so the read timed out).

Example program:

```
#!/usr/bin/env python3
"""
Example: Read multipurpose buffer -- read out and clear the device's one-shot data buffer.
Enters closed loop, stores one PID debug snapshot in the buffer with test mode 3, then reads
the buffer back and prints the five PID values. Each dataset can be read exactly ONCE: a
successful read clears the buffer, and reading an empty buffer returns a single 0 byte (firmware 0.15.4.0 and l
"""
import time
import servomotor
from servomotor import communication

ALIAS = 'X'                               # Device alias; change if needed
SERIAL_PORT = "/dev/tty.usbserial-110"  # Change to your serial port (e.g. "COM3" on Windows).
                                          #  Set SERIAL_PORT = "MENU" to be prompted interactively.
servomotor.set_serial_port(SERIAL_PORT)
servomotor.open_serial_port()
motor = servomotor.M3(ALIAS, time_unit="seconds", position_unit="shaft_rotations",
                      velocity_unit="rotations_per_second",
                      acceleration_unit="rotations_per_second_squared", verbose=0)
try:
    motor.system_reset()
    time.sleep(1.5)                       # mandatory: keep the bus silent after reset

    # PID debug data is only meaningful in closed-loop mode, so enter it first.
    # go_to_closed_loop() enables the MOSFETs by itself; skipping a separate
    # 'Enable MOSFETs' may even give a gentler engagement.
    motor.go_to_closed_loop()
    deadline = time.time() + 6.0
    while True:                           # poll until the closed-loop bit (bit 2) sets
        status_flags, fatal_error_code = motor.get_status()
        if status_flags & 0b100:
            break
        if time.time() > deadline:
            raise SystemExit("Never entered closed loop -- has this motor been calibrated?")
        time.sleep(0.1)
```

```
    time.sleep(0.3)

    motor.test_mode(3)                  # store ONE PID debug snapshot into the buffer
    time.sleep(0.1)                     # give the control loop a moment to write it

    data = motor.read_multipurpose_buffer()
    # An empty buffer returns a single byte of 0 (the "nothing stored" tag) as of
    # firmware 0.15.4.0. (Older firmware sent NO response at all -- there, a
    # communication.TimeoutError meant "buffer empty".)
    if data[0] == 0:
```

```
        raise SystemExit("Multipurpose buffer was empty (no dataset stored)")

    print(f"Data type tag: {data[0]} (4 = PID debug snapshot)")
    for n, name in enumerate(["error", "P term", "I term", "D term", "output"]):
        value = int.from_bytes(data[1 + n * 4:5 + n * 4], "little", signed=True)
        print(f"  {name}: {value} (internal units)")

    # Clear the test mode with a reset. (As of firmware 0.15.4.0, test_mode(0)
    # also clears it safely; on OLDER firmware value 0 hangs the device, so a
    # reset is the safe way on any firmware version.)
    motor.system_reset()
    time.sleep(1.5)                     # bus must stay silent after reset
finally:
    try:
        motor.disable_mosfets()
    except Exception:
        pass
    servomotor.close_serial_port()
```

Status & Monitoring

Get current time

Return the device's absolute time as an unsigned 64-bit count of true microseconds elapsed since power-up or since the last 'Reset time'; it is not wall-clock time. Executes immediately (not queued) and has no side effects. The wire value needs no scaling: it is a genuine 1 MHz microsecond count, effectively 48 bits wide, wrapping only after roughly 8.9 years. Silent on broadcast (no reply is sent). Any nonzero payload raises fatal error 51, ERROR_COMMAND_SIZE_WRONG. In rare cases of clock malfunction the read itself can raise fatal error 1, ERROR_TIME_WENT_BACKWARDS, which indicates a firmware or hardware fault rather than a usage error.

Returns:
• *currentTime*: u64: The current absolute time in microseconds elapsed since power-up or since the last 'Reset time'. A true 1 MHz microsecond count on the wire (effectively 48 bits wide; wraps after about 8.9 years).

Example program:

```
#!/usr/bin/env python3
"""
Example: Get current time -- read the device's absolute microsecond clock.
Reads the clock twice, one second apart, and shows it advancing in step with
the host. The clock counts up from power-on or the last 'Reset time'.
"""
import time
import servomotor

ALIAS = 'X'                              # Device alias; change if needed
SERIAL_PORT = "/dev/tty.usbserial-110"  # Change to your serial port (e.g. "COM3" on Windows).
                                         #  Set SERIAL_PORT = "MENU" to be prompted interactively.
servomotor.set_serial_port(SERIAL_PORT)
servomotor.open_serial_port()
motor = servomotor.M3(ALIAS, time_unit="seconds", position_unit="shaft_rotations",
                      velocity_unit="rotations_per_second",
                      acceleration_unit="rotations_per_second_squared", verbose=0)
try:
    motor.system_reset()
    time.sleep(1.5)                      # mandatory: keep the bus silent after reset

    t1_s = motor.get_current_time()     # in the configured time unit (seconds here)
    print(f"Device clock: {t1_s:.6f} s")

    time.sleep(1.0)                      # let both clocks advance by one host second

    t2_s = motor.get_current_time()
    print(f"Device clock: {t2_s:.6f} s")
    print(f"Elapsed on device: {t2_s - t1_s:.3f} s (expect about 1.0 s)")
finally:
    try:
        motor.disable_mosfets()
    except Exception:
        pass
    servomotor.close_serial_port()
```

Get n queued items

Return the number of items currently in the movement queue as a single byte between 0 and 32 (the queue holds at most 32 items). Executes immediately (not queued) and has no side effects; poll it to pace movement commands. This matters because queueing a movement command while the queue already holds 32 items is not a benign rejection: it raises fatal error 17, ERROR_QUEUE_IS_FULL, which disables the MOSFETs and halts the device until 'System reset'. Related pitfalls when reconciling this count with commands sent: queueing a move while the motor is busy (for example calibrating or homing) raises fatal error 19, ERROR_MOTOR_BUSY, and single-move commands with a duration of zero time steps are rejected with fatal error 34 as of firmware 0.15.4.0 (older firmware silently dropped them); zero-duration items inside a 'Multimove' are still silently dropped and never occupy a queue slot. Silent on broadcast (no reply is sent). Any nonzero payload raises fatal error 51, ERROR_COMMAND_SIZE_WRONG.

Returns:

• *queueSize*: u8: The number of items currently in the movement queue, 0 to 32. If it is below 32 you can queue more movement commands so motion continues in order without stopping; queueing when it is already 32 raises fatal error 17, ERROR_QUEUE_IS_FULL. A zero-duration move is rejected with fatal error 34 on firmware 0.15.4.0 and later (older firmware dropped it silently without occupying a slot). A 'Trapezoid move' or 'Go to position' normally occupies 3 slots (only 2 when the planned coast duration is exactly zero, i.e. an even duration at most twice max velocity divided by max acceleration) and 'Move with velocity'/'Move with acceleration' 1 slot; the item currently executing is included in the count.

Example program:

```
#!/usr/bin/env python3
"""
Example: Get n queued items -- read how many moves are waiting in the motion queue.
Queues three short moves, then polls the count as it drains to 0. Polling this
command until it returns 0 is the canonical way to wait for motion to finish.
"""
import time
import servomotor

ALIAS = 'X'                             # Device alias; change if needed
SERIAL_PORT = "/dev/tty.usbserial-110"  # Change to your serial port (e.g. "COM3" on Windows).
                                        #  Set SERIAL_PORT = "MENU" to be prompted interactively.
servomotor.set_serial_port(SERIAL_PORT)
servomotor.open_serial_port()
motor = servomotor.M3(ALIAS, time_unit="seconds", position_unit="shaft_rotations",
                      velocity_unit="rotations_per_second",
                      acceleration_unit="rotations_per_second_squared", verbose=0)
try:
    motor.system_reset()
    time.sleep(1.5)                     # mandatory: keep the bus silent after reset

    motor.enable_mosfets()
    time.sleep(0.3)                     # let the rotor settle onto its commutation step
    motor.zero_position()

    # Queue three short moves back-to-back; they execute in order. Each
    # trapezoid move may occupy up to 3 of the 32 queue slots.
    motor.trapezoid_move(0.5, 1.0)      # +0.5 rotation in 1 s
    motor.trapezoid_move(-0.5, 1.0)     # back again
    motor.trapezoid_move(0.5, 1.0)
    print("Queued 3 moves; watching the queue drain...")

    # Canonical wait-for-idle: poll until the queue reads 0, always sleeping
    # between polls (never busy-poll the bus).
    last_count = None
    while True:
        n = motor.get_n_queued_items()
        if n != last_count:
            print(f"  queued items: {n}")
            last_count = n
        if n == 0:
            break
        time.sleep(0.05)
```

```
    time.sleep(0.2)                    # brief mechanical settle before reading the position
```

```
    print(f"Motion complete. Final position: {motor.get_position():.3f} shaft rotations")
finally:
    try:
        motor.disable_mosfets()
    except Exception:
        pass
    servomotor.close_serial_port()
```

Get hall sensor position

Get the position as measured by the hall sensors, which is the actual shaft position. It is reported in the same frame as the commanded position, so in normal operation it reads about the same as 'Get position'; if the two deviate by more than the max allowable position deviation (default 2 shaft rotations), the firmware raises fatal error 45, ERROR_POSITION_DEVIATION_TOO_LARGE. This is an immediate read-only query that returns exactly one response; the value is zeroed together with the commanded position by 'Zero position'. No response is sent when broadcast. Any payload bytes raise fatal error 51, ERROR_COMMAND_SIZE_WRONG.

Returns:
• *hallSensorPosition*: i64: The current position as determined by the hall sensors

Example program:

```
#!/usr/bin/env python3
"""
Example: Get hall sensor position -- read the measured (actual) shaft position.
Performs a small move, then prints the commanded position, the hall sensor
(measured) position, and their difference. A large or growing difference
means the motor stalled or missed motion -- the basis of stall detection.
"""
import time
import servomotor

ALIAS = 'X'                            # Device alias; change if needed
SERIAL_PORT = "/dev/tty.usbserial-110"  # Change to your serial port (e.g. "COM3" on Windows).
                                       #  Set SERIAL_PORT = "MENU" to be prompted interactively.
MOVE_ROTATIONS = 1.0                   # Small, safe test move (shaft rotations)
MOVE_SECONDS = 2.0                     # Duration of the test move (seconds)

servomotor.set_serial_port(SERIAL_PORT)
servomotor.open_serial_port()
motor = servomotor.M3(ALIAS, time_unit="seconds", position_unit="shaft_rotations",
                      velocity_unit="rotations_per_second",
                      acceleration_unit="rotations_per_second_squared", verbose=0)
try:
    motor.system_reset()
    time.sleep(1.5)                    # mandatory: keep the bus silent after reset

    motor.enable_mosfets()
    time.sleep(0.3)                    # energizing snaps the rotor to a commutation step;
                                       #  zeroing during that transient corrupts the zero
    motor.zero_position()              # zeroes the commanded AND hall sensor positions together

    motor.trapezoid_move(MOVE_ROTATIONS, MOVE_SECONDS)
    while motor.get_n_queued_items() > 0:
        time.sleep(0.05)               # poll with sleeps -- never busy-poll the bus
    time.sleep(0.2)                    # brief mechanical settling before reading positions

    commanded = motor.get_position()             # where the firmware thinks the shaft is
    measured = motor.get_hall_sensor_position()  # where the shaft actually is
```

```
    print(f"Commanded position:       {commanded:+.4f} shaft rotations")
    print(f"Hall sensor position:     {measured:+.4f} shaft rotations")
    print(f"Difference (meas - cmd):  {measured - commanded:+.4f} shaft rotations")
    # At rest the two should agree within a few hundred encoder counts (about
    # 0.0001 rotations). If the difference exceeds the max allowable position
    # deviation (default 2 rotations), the firmware latches fatal error 45.
finally:
    try:
```

```
        motor.disable_mosfets()
    except Exception:
        pass
    servomotor.close_serial_port()
```

Get status

Get the motor's status flags and fatal error code. This is an immediate query and one of only two commands still answered normally after a fatal error (the other is 'System reset'); every other command then receives an error packet until the device is reset. In normal operation the flags are recomputed at the moment of the request. In the fatal-error state the flags all read 0 as of firmware 0.15.4.0 (the MOSFETs are off and nothing is running); in firmware before 0.15.4.0 they were a frozen stale snapshot, so bit 1 could still read as MOSFETs-enabled and only the fatal error code was trustworthy. When broadcast, this command does nothing at all: no reply is sent and the stored snapshot is not even refreshed. No flag reflects ordinary queued motion (the busy flag covers only long-running tasks such as calibration and homing), so to detect motion completion poll 'Get n queued items' and confirm the position has settled rather than watching these flags. Sending any payload bytes raises fatal error 51, ERROR_COMMAND_SIZE_WRONG.

Returns:
• *statusFlags*: u16: A series of flags which are 1 bit each. In normal operation these are recomputed at the moment of the request; in the fatal-error state the flags read 0 as of firmware 0.15.4.0 (older firmware returned a frozen stale snapshot there, where only the fatal error code was trustworthy).
• *fatalErrorCode*: u8: The fatal error code. If 0 then there is no fatal error. Once a fatal error happens, the motor disables its MOSFETs and answers only 'Get status' and 'System reset' until it is reset; press the reset button on the motor or execute the 'System reset' command to get out of the fatal error state. This field is always reliable, on any firmware version.

Example program:

```
#!/usr/bin/env python3
"""
Example: Get status -- read the motor's status flags and fatal error code.
Enables the MOSFETs so one live flag is set, then prints every flag bit by
name plus the fatal error code. Expect bit 1 SET, all others clear, error 0.
"""
import time
import servomotor

ALIAS = 'X'                              # Device alias; change if needed
SERIAL_PORT = "/dev/tty.usbserial-110"  # Change to your serial port (e.g. "COM3" on Windows).
                                         #  Set SERIAL_PORT = "MENU" to be prompted interactively.
FLAG_NAMES = [
    (0, "In bootloader (if set, all other bits read 0)"),
    (1, "MOSFETs enabled"),
    (2, "Closed loop mode"),
    (3, "Calibration in progress"),
    (4, "Homing in progress"),
    (5, "Going to closed loop (only ever set on M1 products)"),
    (6, "Busy with a long-running task"),
]

servomotor.set_serial_port(SERIAL_PORT)
```

```
servomotor.open_serial_port()
motor = servomotor.M3(ALIAS, time_unit="seconds", position_unit="shaft_rotations",
                      velocity_unit="rotations_per_second",
                      acceleration_unit="rotations_per_second_squared", verbose=0)
try:
    motor.system_reset()
    time.sleep(1.5)                    # mandatory: keep the bus silent after reset

    motor.enable_mosfets()             # sets status bit 1 so the report shows a live flag
    time.sleep(0.3)                    # let the rotor settle after energizing

    flags, fatal_error_code = motor.get_status()   # returns [statusFlags, fatalErrorCode]
    print(f"Raw status flags: 0b{flags:07b}")
    for bit, name in FLAG_NAMES:
        state = "SET" if (flags >> bit) & 1 else "clear"
        print(f"  Bit {bit}  {name:<48} {state}")
    print(f"Fatal error code: {fatal_error_code} (0 = no fatal error)")
    # 'Get status' is one of only two commands still answered after a fatal error
    # (the other is 'System reset'). In the fatal-error state the flags are
    # cleared to 0 (only the in-bootloader bit is kept), so use the fatal error code for diagnosis. On any
    # nonzero code, recover with system_reset() followed by a 1.5 s bus-silent wait.
finally:
```

```
    try:
        motor.disable_mosfets()
    except Exception:
        pass
    servomotor.close_serial_port()
```

Control hall sensor statistics

Turn hall sensor statistics gathering on or off. This command executes immediately (it is not queued). Sending 1 atomically resets the statistics (max = 0, min = 65535, sums = 0, count = 0) and turns gathering on; sending 0 turns gathering off, leaving the accumulated statistics frozen and readable via 'Get hall sensor statistics'. There is no way to reset without enabling, or to enable without resetting. Any control value other than 0 or 1 is silently ignored but still returns a success response. The payload must be exactly 1 byte; any other size raises fatal error 51, ERROR_COMMAND_SIZE_WRONG. Broadcast behavior is anomalous: when sent to the broadcast address 255 the command does nothing at all (most commands execute on broadcast and merely suppress the reply).

Parameters:
• *control*: u8: 0 = turn off statistics gathering (accumulated statistics remain frozen and readable), 1 = reset the statistics (max = 0, min = 65535, sums = 0, count = 0) and turn on gathering. Any other value is silently ignored but still returns a success response.

Example program:

```
#!/usr/bin/env python3
"""
Example: Control hall sensor statistics -- start and freeze hall sensor data gathering.
Sends 1 to reset-and-start statistics gathering, samples for 1 second, then sends 0
to freeze the snapshot. A read-back of the measurement count proves data was captured.
"""
import time
import servomotor

ALIAS = 'X'                             # Device alias; change if needed
GATHER_SECONDS = 1.0                    # How long to let statistics accumulate
SERIAL_PORT = "/dev/tty.usbserial-110"  # Change to your serial port (e.g. "COM3" on Windows).
                                        #  Set SERIAL_PORT = "MENU" to be prompted interactively.
servomotor.set_serial_port(SERIAL_PORT)

servomotor.open_serial_port()
motor = servomotor.M3(ALIAS, time_unit="seconds", position_unit="shaft_rotations",
                      velocity_unit="rotations_per_second",
                      acceleration_unit="rotations_per_second_squared", verbose=0)
try:
    motor.system_reset()
    time.sleep(1.5)                     # mandatory: keep the bus silent after reset

    # Value 1 always RESETS the statistics (max=0, min=65535, sums=0, count=0)
    # AND starts gathering -- there is no way to start without resetting.
    # Note: broadcast (alias 255) does nothing for this command; use a real alias.
    motor.control_hall_sensor_statistics(1)
    print(f"Statistics reset and gathering started; sampling for {GATHER_SECONDS} s ...")
    time.sleep(GATHER_SECONDS)

    # Value 0 freezes the accumulated statistics so later reads see a stable
    # snapshot instead of a moving target. The data stays readable until the
    # next reset-and-start (or a system reset).
    motor.control_hall_sensor_statistics(0)
    print("Gathering frozen.")

    stats = motor.get_hall_sensor_statistics()   # last field is the measurement count
    print(f"Measurements captured in the {GATHER_SECONDS} s window: {stats[9]}")
finally:
    try:
        motor.disable_mosfets()
    except Exception:
        pass
    servomotor.close_serial_port()
```

Get hall sensor statistics

Read back the statistics gathered from the three analog hall sensors. This is useful for checking hall sensor health and the noise in the system. Executes immediately and returns an atomic snapshot (taken with the control-loop interrupt masked) of the per-sensor maximum, minimum, and sum, plus the total measurement count; no averages are returned, so compute average = sum / measurementCount yourself. Reading does not stop or reset the statistics. Gathering is off by default after boot or reset and must be enabled with 'Control hall sensor statistics'; if this command is sent before gathering has ever been enabled since power-up, every field reads zero, including the minimums (0, not 65535), which can be misread as a dead sensor. Each recorded value is the sum of 4 oversampled 12-bit ADC readings, so the per-sample range is 0 to 16380, not 0 to 4095. Once measurementCount reaches 4294967295 the count and sums stop accumulating, while max and min continue to update. The payload must be empty; any payload bytes raise fatal error 51, ERROR_COMMAND_SIZE_WRONG. GOTCHA: after boot every field including the maxima reads 0 -- this means the statistics were never started (send 'Control hall sensor statistics' with 1 first), not that the sensors are dead. Healthy M17 baseline: across one full slow rotation each channel sweeps roughly 4000 to 12400 ADC units (peak-to-peak ~8200-8340); accumulation runs at ~31,400 samples/second.

Returns:
• *maxHall1*: u16: The maximum value of hall sensor 1 encountered since the last statistics reset.
• *maxHall2*: u16: The maximum value of hall sensor 2 encountered since the last statistics reset.
• *maxHall3*: u16: The maximum value of hall sensor 3 encountered since the last statistics reset.
• *minHall1*: u16: The minimum value of hall sensor 1 encountered since the last statistics reset (initialized to 65535 by the reset; reads 0 if statistics gathering was never enabled since power-up).
• *minHall2*: u16: The minimum value of hall sensor 2 encountered since the last statistics reset (initialized to 65535 by the reset; reads 0 if statistics gathering was never enabled since power-up).
• *minHall3*: u16: The minimum value of hall sensor 3 encountered since the last statistics reset (initialized to 65535 by the reset; reads 0 if statistics gathering was never enabled since power-up).
• *sumHall1*: u64: The sum of hall sensor 1 values collected since the last statistics reset.
• *sumHall2*: u64: The sum of hall sensor 2 values collected since the last statistics reset.
• *sumHall3*: u64: The sum of hall sensor 3 values collected since the last statistics reset.
• *measurementCount*: u32: The number of times the hall sensors were measured since the last statistics reset. Saturates at 4294967295, after which the count and the sums stop accumulating (max and min continue to update).

Example program:

```
#!/usr/bin/env python3
"""
Example: Get hall sensor statistics -- read per-sensor max/min/sum/count and compute means.
Starts statistics gathering, samples for 1 second, then prints the statistics for each
of the three analog hall sensors. Steady values with modest max-min spread mean a
healthy, low-noise sensor system.
"""
import time
import servomotor

ALIAS = 'X'                               # Device alias; change if needed
SERIAL_PORT = "/dev/tty.usbserial-110"  # Change to your serial port (e.g. "COM3" on Windows).
                                          #  Set SERIAL_PORT = "MENU" to be prompted interactively.
GATHER_SECONDS = 1.0                      # How long to let statistics accumulate

servomotor.set_serial_port(SERIAL_PORT)
servomotor.open_serial_port()
motor = servomotor.M3(ALIAS, time_unit="seconds", position_unit="shaft_rotations",
                      velocity_unit="rotations_per_second",
                      acceleration_unit="rotations_per_second_squared", verbose=0)
try:
    motor.system_reset()
    time.sleep(1.5)                       # mandatory: keep the bus silent after reset

    # Gathering is OFF after reset. Without starting it first, every field
    # (even the minimums) reads 0, which can be misread as dead sensors.
    motor.control_hall_sensor_statistics(1)      # 1 = reset AND start gathering
    time.sleep(GATHER_SECONDS)
```

```
    stats = motor.get_hall_sensor_statistics()   # reading neither stops nor resets
    max1, max2, max3, min1, min2, min3, sum1, sum2, sum3, count = stats
    print(f"Measurement count: {count}")
    # Each recorded value is the sum of 4 oversampled 12-bit ADC readings,
    # so the valid range is 0..16380 (not 0..4095). No averages are returned;
    # compute mean = sum / count yourself.
    for name, mx, mn, sm in (("Hall 1", max1, min1, sum1),
                             ("Hall 2", max2, min2, sum2),
                             ("Hall 3", max3, min3, sum3)):
        mean = sm / count if count else 0.0
        print(f"{name}: max={mx}  min={mn}  sum={sm}  mean={mean:.1f}  (ADC counts, 0..16380)")
finally:
    try:
        motor.disable_mosfets()
    except Exception:
        pass
```

```
    servomotor.close_serial_port()
```

Get position

Get the current commanded (desired) position, i.e. the motion profile's internal target position that advances every control tick. This may differ slightly from the actual position measured by the hall sensors, which is available via 'Get hall sensor position'. Executes immediately (it is not queued), can be sent at any time including during motion, and takes an atomic snapshot with interrupts disabled. Commanded and measured position live in the same internal count space, so the same position unit conversion applies to both. The payload must be empty; any payload bytes raise fatal error 51, ERROR_COMMAND_SIZE_WRONG. Broadcast produces no response and has no side effects.

Returns:
• *position*: i64: The current commanded (desired) position from the motion profile; may differ slightly from the hall-sensor-measured position returned by 'Get hall sensor position'.

Example program:

```
#!/usr/bin/env python3
"""
Example: Get position -- read the current commanded (desired) position.
Zeroes the position, performs a small move, and reads the position before and
after. Expect 0 before the move and about +1.0 shaft rotations after it.
"""
import time
import servomotor

ALIAS = 'X'                             # Device alias; change if needed
SERIAL_PORT = "/dev/tty.usbserial-110"  # Change to your serial port (e.g. "COM3" on Windows).
                                        #  Set SERIAL_PORT = "MENU" to be prompted interactively.
MOVE_ROTATIONS = 1.0                    # Small, safe test move (shaft rotations)
MOVE_SECONDS = 2.0                      # Duration of the test move (seconds)

servomotor.set_serial_port(SERIAL_PORT)
servomotor.open_serial_port()
motor = servomotor.M3(ALIAS, time_unit="seconds", position_unit="shaft_rotations",
                      velocity_unit="rotations_per_second",
                      acceleration_unit="rotations_per_second_squared", verbose=0)
try:
    motor.system_reset()
    time.sleep(1.5)                     # mandatory: keep the bus silent after reset

    motor.enable_mosfets()
    time.sleep(0.3)                     # settle after energizing before zeroing the position
```

```
    motor.zero_position()

    print(f"Position after zeroing:  {motor.get_position():+.4f} shaft rotations")

    motor.trapezoid_move(MOVE_ROTATIONS, MOVE_SECONDS)
    while motor.get_n_queued_items() > 0:
        time.sleep(0.05)               # poll with sleeps -- never busy-poll the bus
    time.sleep(0.2)                    # brief mechanical settling before reading

    print(f"Position after the move: {motor.get_position():+.4f} shaft rotations")
    # NOTE: 'Get position' returns the COMMANDED position -- the motion profile's
    # internal target, not a measurement. It can be read at any time, including
    # during motion. For the measured shaft position use 'Get hall sensor
    # position' and compare the two to detect stalls or missed motion.
finally:
    try:
        motor.disable_mosfets()
    except Exception:
        pass
```

```
    servomotor.close_serial_port()
```

Get comprehensive position

Get the commanded position, the hall sensor (measured) position, and the external encoder count in one shot. Executes immediately (not queued). The first two values are in encoder counts and convert normally with the motor's position units; the third is a raw signed count from an optional external quadrature encoder and must not be unit-converted (see its parameter note). The three values are read back-to-back rather than atomically as a set, so they can be skewed from one another by up to one control-loop tick. The external encoder count is never zeroed by any command, not even 'Zero position'; it accumulates from boot. Silent on broadcast: no response is sent. The payload must be empty or the device raises fatal error ERROR_COMMAND_SIZE_WRONG.

Returns:
• *commandedPosition*: i64: The commanded position (which may differ from actual)
• *hallSensorPosition*: i64: The hall sensor position (or you could say the actual measured position)
• *externalEncoderPosition*: i32: The external encoder position: a raw signed count from an optional external quadrature encoder (incremented or decremented once per rising edge of the encoder A line, with the B line giving direction). This needs special hardware attached to the motor to work. Do not apply the motor's position unit conversion to this value -- counts per revolution depend on the attached encoder, so treat it as a raw dimensionless count. It is never zeroed by any command (not even 'Zero position') and accumulates from boot.

Example program:

```
#!/usr/bin/env python3
"""
Example: Get comprehensive position -- commanded, measured, and external
encoder positions in one round trip. Performs a small move, then prints all
three values from a single query (safe to poll at 20 Hz during motion).
"""
import time
import servomotor

ALIAS = 'X'                            # Device alias; change if needed
SERIAL_PORT = "/dev/tty.usbserial-110"  # Change to your serial port (e.g. "COM3" on Windows).
                                       #  Set SERIAL_PORT = "MENU" to be prompted interactively.
MOVE_ROTATIONS = 1.0                   # Small, safe test move (shaft rotations)
MOVE_SECONDS = 2.0                     # Duration of the test move (seconds)

servomotor.set_serial_port(SERIAL_PORT)
```

```
servomotor.open_serial_port()
motor = servomotor.M3(ALIAS, time_unit="seconds", position_unit="shaft_rotations",
                      velocity_unit="rotations_per_second",
                      acceleration_unit="rotations_per_second_squared", verbose=0)
try:
    motor.system_reset()
    time.sleep(1.5)                    # mandatory: keep the bus silent after reset

    motor.enable_mosfets()
    time.sleep(0.3)                    # settle after energizing before zeroing the position
    motor.zero_position()

    motor.trapezoid_move(MOVE_ROTATIONS, MOVE_SECONDS)
    while motor.get_n_queued_items() > 0:
        time.sleep(0.05)               # poll with sleeps -- never busy-poll the bus
    time.sleep(0.2)                    # brief mechanical settling before reading

    # One command returns all three positions: [commanded, hall, external encoder]
    commanded, measured, external = motor.get_comprehensive_position()
    print(f"Commanded position:     {commanded:+.4f} shaft rotations")
    print(f"Hall sensor position:   {measured:+.4f} shaft rotations")
    # The external encoder value comes from an OPTIONAL add-on quadrature encoder:
    # it reads 0 when none is fitted and is never zeroed by any command -- not
    # even 'Zero position'. The library wrongly applies the motor's position
    # conversion to this raw count (a known wart); multiply back by the
    # shaft_rotations factor to recover the encoder's own counts.
    print(f"External encoder value: {external * 3276800:.0f} raw counts")
finally:
    try:
```

```
        motor.disable_mosfets()
    except Exception:
        pass
    servomotor.close_serial_port()
```

Get supply voltage

Get the measured voltage of the power supply. Executes immediately (not queued). The u16 reply is the supply voltage in decivolts (volts times 10); the firmware averages four ADC samples and applies a per-product calibration constant, so divide the value by 10 to get volts. Silent on broadcast: no response is sent. The payload must be empty or the device raises fatal error ERROR_COMMAND_SIZE_WRONG.

Returns:

• *supplyVoltage*: u16: The voltage. Divide this number by 10 to get the actual voltage in volts.

Example program:

```
#!/usr/bin/env python3
"""
Example: Get supply voltage -- read the motor's power supply voltage.
Reads the bus voltage and prints it in volts. Expect your power supply's set
voltage within a few percent (these motors normally run from 12-24 V).
"""
import time
import servomotor

ALIAS = 'X'                               # Device alias; change if needed
SERIAL_PORT = "/dev/tty.usbserial-110"  # Change to your serial port (e.g. "COM3" on Windows).
                                          #  Set SERIAL_PORT = "MENU" to be prompted interactively.

servomotor.set_serial_port(SERIAL_PORT)
servomotor.open_serial_port()
# voltage_unit="volts" matters: the library's default voltage unit is millivolts.
motor = servomotor.M3(ALIAS, time_unit="seconds", position_unit="shaft_rotations",
                      velocity_unit="rotations_per_second",
                      acceleration_unit="rotations_per_second_squared",
                      voltage_unit="volts", verbose=0)
try:
    motor.system_reset()
    time.sleep(1.5)                       # mandatory: keep the bus silent after reset
    time.sleep(0.2)                       # ~0.2 s ADC settle -- already covered by the 1.5 s wait
                                          #  above; shown for when you read right after other resets

    # The device measures in tenths of a volt (averaging 4 ADC samples);
    # the library converts to the unit chosen above.
    voltage = motor.get_supply_voltage()
    print(f"Supply voltage: {voltage:.1f} V")
    # Sanity check: this should match your PSU setting within a few percent.
    # A wildly different reading points at a wiring or power problem.
finally:
    try:
        motor.disable_mosfets()
    except Exception:
        pass
    servomotor.close_serial_port()
```

Get max PID error

Get the minimum and maximum position error observed by the closed-loop PID controller since the last read, then reset the accumulators. The error is commanded position minus hall sensor position in encoder counts, clamped to plus or minus 2^29 before capture; a positive maximum means the measured position lagged behind the commanded position. Reading is destructive: it resets the accumulators to sentinel values (min = +2147483647, max = -2147483648), so two back-to-back reads return sentinels on the second read unless the PID loop ran in between. The error is only sampled while the PID controller actually runs (closed-loop position or velocity control); if the motor has not been in closed loop since the last read (or since boot), the reply is the sentinel pair itself. Always treat min greater than max as no data, not as a real error excursion (in practice this only appears if the device was never in closed loop during the window, because in closed loop the PID runs every 32-microsecond tick; an idle holding motor shows a dither of roughly plus/minus 500 counts). The wire values are encoder counts; client libraries that perform unit conversion, such as the Python library, return them converted into the currently selected position unit. On broadcast no reply is sent and the accumulators are not reset.

Returns:
• *minPidError*: i32: The minimum PID error observed since the last read, in encoder counts. Equals +2147483647 (the reset sentinel) if the PID loop has not run since the last read; treat min greater than max as no data.
• *maxPidError*: i32: The maximum PID error observed since the last read, in encoder counts. Positive values mean the measured position lagged behind the commanded position. Equals -2147483648 (the reset sentinel) if the PID loop has not run since the last read; treat min greater than max as no data.

Example program:

```
#!/usr/bin/env python3
"""
Example: Get max PID error -- measure closed-loop tracking quality.
Enters closed loop, clears the error window, performs a move, then reads the
[min, max] position error the PID controller saw during that move.
"""
import time
import servomotor

ALIAS = 'X'                             # Device alias; change if needed
SERIAL_PORT = "/dev/tty.usbserial-110"  # Change to your serial port (e.g. "COM3" on Windows).
                                        #  Set SERIAL_PORT = "MENU" to be prompted interactively.
servomotor.set_serial_port(SERIAL_PORT)
servomotor.open_serial_port()
motor = servomotor.M3(ALIAS, time_unit="seconds", position_unit="shaft_rotations",
                      velocity_unit="rotations_per_second",
                      acceleration_unit="rotations_per_second_squared", verbose=0)
try:
    motor.system_reset()
    time.sleep(1.5)                     # mandatory: keep the bus silent after reset
    motor.go_to_closed_loop()           # the error only accumulates while the PID loop runs,
                                        #  i.e. in closed loop. This enables the MOSFETs by
                                        #  itself; skipping a separate 'Enable MOSFETs' may
                                        #  even give a gentler engagement.
    deadline = time.time() + 6.0
    while True:
        status_flags, fatal_error_code = motor.get_status()
        if status_flags & (1 << 2):     # bit 2 = closed-loop mode active
            break
        if time.time() > deadline:
            raise TimeoutError("Never entered closed loop -- is the motor calibrated?")
        time.sleep(0.1)
    time.sleep(0.3)                     # settle after the mode change so the zero is clean
    motor.zero_position()
    # Every read RESETS the min/max window, so read once here and throw the result
    # away -- the next read then covers exactly the move below.
    motor.get_max_pid_error()
    motor.trapezoid_move(1.0, 2.0)      # small safe demo move: 1 rotation in 2 seconds
    while motor.get_n_queued_items() > 0:
        time.sleep(0.05)                # poll the queue; never busy-poll without a sleep
```

```
    time.sleep(0.2)                     # brief mechanical settling
    motor.set_position_unit("encoder_counts")   # the controller's native error unit
    min_err, max_err = motor.get_max_pid_error()
    # min > max means the reset sentinels came back (+2147483647 / -2147483648):
    # no PID samples since the last read -- "no data", not a huge real error.
```

```
    if min_err > max_err:
        print("No PID data accumulated since the last read")
    else:
        print(f"Tracking error during the move: min={min_err:.0f}, max={max_err:.0f} encoder counts")
        print("(1 shaft rotation = 3,276,800 counts; positive max = measured position lagged commanded)")
finally:
    try:
        motor.disable_mosfets()
    except Exception:
        pass
    servomotor.close_serial_port()
```

Get temperature

Get the temperature measured by a dedicated analog sensor on the motor driver PCB (not the microcontroller die and not the motor windings). The command executes immediately. Accuracy is about +/- 3 degrees Celsius, but only within the conversion table's range of roughly 33 to 307 degrees Celsius: any reading outside that range is reported as exactly 0, so a motor at room temperature reads 0 and negative temperatures are never reported despite the signed return type. Treat a value of 0 as an out-of-range sentinel, not a measurement. Independently of this command, the firmware raises fatal error 40, ERROR_OVERHEAT, at about 80 degrees Celsius, but only while the MOSFETs are enabled. The payload must be empty or fatal error 51, ERROR_COMMAND_SIZE_WRONG, is raised. When broadcast, no measurement is taken and no response is sent, so broadcasting this command is useless.

Returns:
• *temperature*: i16: The temperature in degrees Celsius, measured at the motor driver PCB. Accuracy is about +/- 3 degrees Celsius within the sensor's usable range of roughly 33 to 307 degrees Celsius. Readings outside that range are reported as exactly 0 (an out-of-range sentinel, for example a motor at room temperature), and negative values are never returned despite the signed type.

Example program:

```
#!/usr/bin/env python3
"""
Example: Get temperature -- read the temperature of the motor driver PCB.
Reads the dedicated analog temperature sensor and prints degrees Celsius.
At room temperature expect 0 (below-range sentinel); under load the value climbs.
"""
import time
import servomotor

ALIAS = 'X'                              # Device alias; change if needed
SERIAL_PORT = "/dev/tty.usbserial-110"  # Change to your serial port (e.g. "COM3" on Windows).
                                         #  Set SERIAL_PORT = "MENU" to be prompted interactively.

servomotor.set_serial_port(SERIAL_PORT)
servomotor.open_serial_port()
motor = servomotor.M3(ALIAS, time_unit="seconds", position_unit="shaft_rotations",
                      velocity_unit="rotations_per_second",
                      acceleration_unit="rotations_per_second_squared",
                      temperature_unit="celsius", verbose=0)
try:
    motor.system_reset()
    time.sleep(1.5)                      # mandatory: keep the bus silent after reset

    temperature = motor.get_temperature()
    # The sensor's conversion table only covers about 33-307 C. Any reading
    # outside that range is reported as exactly 0 -- so 0 means "below ~33 C"
    # (e.g. a motor at room temperature), NOT freezing.
    if temperature == 0:
        print("Temperature: 0 C -- driver PCB is below ~33 C (out-of-range sentinel).")
    else:
        print(f"Driver PCB temperature: {temperature} C (accuracy about +/- 3 C)")
finally:
    try:
        motor.disable_mosfets()
    except Exception:
        pass
    servomotor.close_serial_port()
```

Get debug values

Get a snapshot of many internal diagnostic values. The snapshot contains: the maximum acceleration, maximum velocity, and current commanded velocity (all in raw internal units; no unit conversion is applied to any output of this command), the measured velocity, the number of time steps remaining in the currently executing queue item, four general-purpose debug values, execution-time profiler counters for the motor control loop, raw hall sensor voltages, the commutation position offset and phase-reversal flag, hall position delta statistics, and the motor PWM voltage. Executes immediately (not queued). Side effect: reading resets the hall position delta statistics, so maxHallPositionDelta, minHallPositionDelta, and averageHallPositionDelta cover only the interval since the previous read; if no samples accumulated in that interval, the max and min return the sentinel initialization values -2000000000 and +2000000000 and the average is meaningless. debugValue1 to debugValue4 are context-dependent diagnostic slots; when fatal error 45, ERROR_POSITION_DEVIATION_TOO_LARGE, trips they are overwritten with position-deviation forensics -- which are, however, UNREACHABLE in practice, because this command is not answered in the fatal-error state (it receives the error reply like every other command; verified on the bench). The profiler times are in microseconds; the motor-control calculations measure 17 us (open-loop idle) to 24-27 us (during a move) of the 32-microsecond control budget. Broadcasting this command does nothing at all: no reply is sent and the statistics are not reset. Any non-empty payload raises fatal error 51, ERROR_COMMAND_SIZE_WRONG.

Returns:

• *maxAcceleration*: i64: Maximum acceleration setting, in raw internal units
• *maxVelocity*: i64: Maximum velocity setting, in raw internal units
• *currentVelocity*: i64: Current commanded velocity, in raw internal units
• *measuredVelocity*: i32: Measured velocity
• *nTimeSteps*: u32: Number of time steps remaining in the currently executing queue item (decremented every control tick)
• *debugValue1*: i64: General-purpose debug value; latches the signed position deviation when fatal error 45, ERROR_POSITION_DEVIATION_TOO_LARGE, trips
• *debugValue2*: i64: General-purpose debug value; latches the absolute position deviation when fatal error 45 trips
• *debugValue3*: i64: General-purpose debug value; latches the commanded position when fatal error 45 trips
• *debugValue4*: i64: General-purpose debug value; latches the hall sensor position when fatal error 45 trips
• *allMotorControlCalculationsProfilerTime*: u16: All motor control calculations profiler time
• *allMotorControlCalculationsProfilerMaxTime*: u16: All motor control calculations profiler maximum time
• *getSensorPositionProfilerTime*: u16: Get sensor position profiler time
• *getSensorPositionProfilerMaxTime*: u16: Get sensor position profiler maximum time
• *computeVelocityProfilerTime*: u16: Compute velocity profiler time
• *computeVelocityProfilerMaxTime*: u16: Compute velocity profiler maximum time
• *motorMovementCalculationsProfilerTime*: u16: Motor movement calculations profiler time
• *motorMovementCalculationsProfilerMaxTime*: u16: Motor movement calculations profiler maximum time
• *motorPhaseCalculationsProfilerTime*: u16: Motor phase calculations profiler time
• *motorPhaseCalculationsProfilerMaxTime*: u16: Motor phase calculations profiler maximum time
• *motorControlLoopPeriodProfilerTime*: u16: Motor control loop period profiler time
• *motorControlLoopPeriodProfilerMaxTime*: u16: Motor control loop period profiler maximum time
• *hallSensor1Voltage*: u16: Hall sensor 1 voltage
• *hallSensor2Voltage*: u16: Hall sensor 2 voltage
• *hallSensor3Voltage*: u16: Hall sensor 3 voltage
• *commutationPositionOffset*: u32: Commutation position offset
• *motorPhasesReversed*: u8: Motor phases reversed flag
• *maxHallPositionDelta*: i32: Maximum hall position delta since the previous read of this command (reading resets it; the sentinel -2000000000 means no samples accumulated since the last read)
• *minHallPositionDelta*: i32: Minimum hall position delta since the previous read of this command (reading resets it; the sentinel +2000000000 means no samples accumulated since the last read)
• *averageHallPositionDelta*: i32: Average hall position delta since the previous read of this command (reading resets it; meaningless if no samples accumulated since the last read)
• *motorPwmVoltage*: u8: Motor PWM voltage

Example program:

```
#!/usr/bin/env python3
"""
Example: Get debug values -- read the motor's 30-field diagnostic snapshot.
Prints every value with its label: velocity/acceleration state, control-loop
profiler times, raw hall sensor voltages, commutation info, and PWM voltage.
"""
import time
import servomotor

ALIAS = 'X'                               # Device alias; change if needed
SERIAL_PORT = "/dev/tty.usbserial-110"  # Change to your serial port (e.g. "COM3" on Windows).
                                          #  Set SERIAL_PORT = "MENU" to be prompted interactively.

# The 30 output fields, in wire order. All values are raw internal units.
LABELS = [
    "maxAcceleration", "maxVelocity", "currentVelocity", "measuredVelocity",
    "nTimeSteps", "debugValue1", "debugValue2", "debugValue3", "debugValue4",
    "allMotorControlCalculationsProfilerTime", "allMotorControlCalculationsProfilerMaxTime",
    "getSensorPositionProfilerTime", "getSensorPositionProfilerMaxTime",
    "computeVelocityProfilerTime", "computeVelocityProfilerMaxTime",
    "motorMovementCalculationsProfilerTime", "motorMovementCalculationsProfilerMaxTime",
    "motorPhaseCalculationsProfilerTime", "motorPhaseCalculationsProfilerMaxTime",
    "motorControlLoopPeriodProfilerTime", "motorControlLoopPeriodProfilerMaxTime",
    "hallSensor1Voltage", "hallSensor2Voltage", "hallSensor3Voltage",
    "commutationPositionOffset", "motorPhasesReversed",
    "maxHallPositionDelta", "minHallPositionDelta", "averageHallPositionDelta",
    "motorPwmVoltage",
]

servomotor.set_serial_port(SERIAL_PORT)
servomotor.open_serial_port()
motor = servomotor.M3(ALIAS, time_unit="seconds", position_unit="shaft_rotations",
                      velocity_unit="rotations_per_second",
                      acceleration_unit="rotations_per_second_squared", verbose=0)
try:
    motor.system_reset()
    time.sleep(1.5)                     # mandatory: keep the bus silent after reset

    # CAUTION: reading resets some fields (the profiler max-times and the hall
    # position delta min/max/average), so those cover only the window since the
    # previous read -- they are NOT cumulative or monotonic. Hall-delta sentinels
    # -2000000000 / +2000000000 mean "no samples accumulated since the last read".
    values = motor.get_debug_values()   # multi-output command -> flat list of 30 values
    print(f"Received {len(values)} debug values (raw internal units):")
    for label, value in zip(LABELS, values):
```

```
        print(f"  {label:44s} = {value}")
finally:
    try:
        motor.disable_mosfets()
    except Exception:
        pass
    servomotor.close_serial_port()
```

Get communication statistics

Get six communication error counters and optionally reset them: CRC32 errors, packet decode errors (inconsistent packet size), first-bit errors (first byte of a packet lacked the required 1 in the least significant bit), and framing, overrun, and noise errors from the RS485 receiver. A nonzero reset flag clears all six counters, not just the CRC32 counter; the counters are snapshotted and cleared atomically in one call, so no events are lost between read and reset. Executes immediately (not queued). Counters are RAM-only and start at zero at every power-up or 'System reset'. Framing, overrun, and noise conditions are counted but are never fatal. This command is the only way to observe silently dropped packets: with CRC32 checking enabled (see 'CRC32 control'), a packet with a bad CRC32 is discarded with no error response and crc32ErrorCount increments. Broadcast gotcha: broadcasting this command does nothing at all, no reply and no reset, so counters cannot be bulk-reset over the bus. A payload that is not exactly 1 byte raises fatal error 51, ERROR_COMMAND_SIZE_WRONG. Verified counter-to-cause mapping: a first byte with its LSB clear increments firstBitErrorCount; a corrupted CRC32 increments crc32ErrorCount and is fully self-clearing (the very next command works, no resync wait needed); a TRUNCATED frame increments nothing immediately but jams the receiver until the 100 ms silence resync (measured threshold 95-105 ms), eating the next packet if sent too soon; an impossible declared size increments packetDecodeErrorCount.

Parameters:
• *resetCounter*: u8: Reset flag: 0 to just read; any nonzero value resets all six counters after they are read

Returns:
• *crc32ErrorCount*: u32: Number of received packets dropped due to CRC32 validation failure (each is discarded silently with no error response)
• *packetDecodeErrorCount*: u32: Number of packet decode errors detected
• *firstBitErrorCount*: u32: Number of times that the first bit in the first byte of a packet was not 1 as expected
• *framingErrorCount*: u32: Number of framing errors detected during reception from the RS485 interface
• *overrunErrorCount*: u32: Number of overrun errors detected during reception from the RS485 interface
• *noiseErrorCount*: u32: Number of noise errors detected during reception from the RS485 interface

Example program:

```
#!/usr/bin/env python3
"""
Example: Get communication statistics -- read the RS485 error counters.
Reads the six error counters without resetting them (flag 0) and prints each
one with its label. On a healthy bus every counter should be 0.
"""
import time
import servomotor

ALIAS = 'X'                              # Device alias; change if needed
SERIAL_PORT = "/dev/tty.usbserial-110"  # Change to your serial port (e.g. "COM3" on Windows).
                                         #  Set SERIAL_PORT = "MENU" to be prompted interactively.
RESET_FLAG = 0                           # 0 = just read; nonzero = clear all six counters after reading

# The six u32 counters, in wire order.
LABELS = [
    "crc32ErrorCount        (packets dropped by CRC32 validation)",
    "packetDecodeErrorCount (inconsistent packet size)",
    "firstBitErrorCount     (first byte's LSB was not 1)",
    "framingErrorCount      (RS485 receiver framing errors)",
    "overrunErrorCount      (RS485 receiver overrun errors)",
    "noiseErrorCount        (RS485 receiver noise errors)",
]

servomotor.set_serial_port(SERIAL_PORT)
servomotor.open_serial_port()
motor = servomotor.M3(ALIAS, time_unit="seconds", position_unit="shaft_rotations",
                      velocity_unit="rotations_per_second",
                      acceleration_unit="rotations_per_second_squared", verbose=0)
try:
    motor.system_reset()
    time.sleep(1.5)                      # mandatory: keep the bus silent after reset

    # These counters are the only way to observe silently-dropped packets: with
```

```
    # CRC32 enabled, a corrupted packet gets NO error reply -- it just increments
    # crc32ErrorCount and the sender sees a timeout. Counters are RAM-only and
    # restart at zero on every reset/power-up.
    stats = motor.get_communication_statistics(RESET_FLAG)  # -> flat list of 6 counters
    print("Communication error counters since the last reset:")
    for label, count in zip(LABELS, stats):
        print(f"  {label} = {count}")
    if all(c == 0 for c in stats):
        print("All counters are 0 -- the bus is healthy.")
    else:
        print("Nonzero counters mean garbled/dropped packets -- check wiring,")
```

```
        print("termination, baud rate, and that both ends agree on the CRC32 setting.")
finally:
    try:
        motor.disable_mosfets()
    except Exception:
        pass
    servomotor.close_serial_port()
```

Unit Conversions

The servomotor library supports multiple unit systems for convenience.

Time Units:
• seconds - time in seconds (default)
• milliseconds - time in milliseconds
• minutes - time in minutes
• microseconds - time in microseconds
• timesteps - raw internal time unit of 32 microseconds (31,250 per second)

Position Units:
• shaft_rotations - rotations of the motor shaft (default)
• degrees - degrees of rotation
• radians - radians of rotation
• encoder_counts - raw encoder counts (3,276,800 per shaft rotation)

Velocity Units:
• rotations_per_second - rotations per second (default)
• rpm - revolutions per minute
• degrees_per_second - degrees per second
• radians_per_second - radians per second
• counts_per_second - encoder counts per second
• counts_per_timestep - encoder counts per 32-microsecond timestep

Acceleration Units:
• rotations_per_second_squared - rotations per second squared (default)
• rpm_per_second - RPM per second
• degrees_per_second_squared - degrees per second squared
• radians_per_second_squared - radians per second squared
• counts_per_second_squared - encoder counts per second squared
• counts_per_timestep_squared - encoder counts per timestep squared

Current Units:
• internal_current_units - raw internal current units (about 150-200 is a typical working value) (default)
• milliamps - milliamperes
• amps - amperes

Voltage Units:
• volts - volts (default)
• millivolts - millivolts

Temperature Units:
• celsius - degrees Celsius (default)
• fahrenheit - degrees Fahrenheit
• kelvin - kelvin

Setting Units:

You can set the units for a motor instance during initialization or at runtime:

```
# During initialization (the alias is a single character, an integer
# 0-251, or a 64-bit unique ID passed as an int)
motor = servomotor.M3(
    'X',
    time_unit='seconds',
    position_unit='degrees',
    velocity_unit='rpm',
    acceleration_unit='rpm_per_second'
```

```
)

# At runtime
motor.set_position_unit('radians')
motor.set_velocity_unit('rotations_per_second')
```

Error Handling

The servomotor has comprehensive error detection and handling. If an error condition is detected then a fatal error condition will result. When this happens, the motor will immediately disable its power stage (MOSFETs) and stop driving the motor, the green LED will turn off, and the red LED will flash. The red LED flashes the error code: it blinks a number of times equal to the error code, then pauses, and repeats (an error code of 0, which can only be triggered artificially, keeps the red LED on continuously). You can count the pulses to read the error code visually, or retrieve it with the "Get status" command. Once you know the error code, you can look it up in the section below to understand the root cause. While in the fatal error state, the servomotor responds only to "Get status" and "System reset"; all other commands receive an error response. You will need to send "System reset" (or power cycle) to get the device back into a functional state. Note that a reset returns the device to its power-on state: the position is zeroed and volatile settings such as safety limits and motion limits return to their defaults.

Not every problem is a fatal error. Communication line problems (framing errors, noise, receive overruns, invalid first bytes, CRC32 mismatches, undecodable packets) do not halt the device; the affected packets are discarded and the events are counted. You can read and optionally reset these counters with the "Get communication statistics" command, which is useful for monitoring the health of the RS485 bus. To protect against corrupted data being accepted as valid, keep CRC32 enabled.

For testing your error handling, the "Test mode" command with values 14 to 73 deliberately triggers fatal error codes 0 to 59 (the triggered code is the value minus 14). Avoid other test mode values during normal operation. In particular, values 10 to 13 (LED test) halt the device permanently after replying: it stops responding to all commands, including "System reset", and only a power cycle recovers it.

With careful programming, a fatal error should not be triggered. In nearly all cases, if a fatal error occurs, it is for a good reason and most likely you will need to improve the way you use the motor.

Error Codes

This section lists all possible error codes that can be returned by the servomotor.

Error 1: ERROR_TIME_WENT_BACKWARDS

Short Description: time went backwards

Description: The internal 64-bit microsecond clock produced a timestamp earlier than the previous one. This is an internal consistency check of the device's time base and should never occur during normal operation.

Possible Causes:

- Internal firmware or timer malfunction (this indicates a bug, not a usage error)
- Severe electrical disturbance corrupting the timer state

Solutions:

- Send the 'System reset' command to restart the device
- Check for electrical noise and power supply stability if it recurs
- Report the problem to the manufacturer along with the firmware version and what the device was doing, if it is reproducible

Error 2: ERROR_FLASH_UNLOCK_FAIL

Short Description: flash unlock fail

Description: The firmware failed to unlock the microcontroller's flash memory for writing. Flash is written when settings are saved (for example after 'Set device alias') and when firmware pages are written during a firmware upgrade.

Possible Causes:

- The flash controller was left in a locked or error state by a previous operation
- Microcontroller hardware fault

Solutions:

- Send the 'System reset' command and retry the operation
- Power cycle the device if the error persists after a reset
- If it still persists, the device likely has a hardware problem; contact the manufacturer

Error 3: ERROR_FLASH_WRITE_FAIL

Short Description: flash write fail

Description: A write to the microcontroller's flash memory did not complete successfully (the flash controller did not confirm end of programming). This can occur while saving settings (for example after 'Set device alias') or while writing a firmware page during a firmware upgrade.

Possible Causes:

- Unstable or interrupted power supply during the flash write
- Flash memory wear or damage
- The flash controller was in an error state from a previous operation

Solutions:

- Send the 'System reset' command and retry the operation
- Ensure the power supply is stable before saving settings or upgrading firmware
- If a firmware upgrade fails repeatedly, power cycle and retry; if it still fails, contact the manufacturer

Error 4: ERROR_TOO_MANY_BYTES

Short Description: too many bytes

Description: Reserved error code. It is defined in the firmware but no condition in the current firmware raises it.

Possible Causes:

- No code path raises this error in the current firmware
- It can only appear if triggered artificially with the 'Test mode' command (values 14 to 73 deliberately raise fatal error codes for testing)

Solutions:

- Send the 'System reset' command to clear the fatal error state

Error 5: ERROR_COMMAND_OVERFLOW

Short Description: command overflow

Description: A byte arrived on the RS485 bus while both of the device's receive buffers were still occupied by unprocessed packets. The device could not keep up with the incoming data stream.

Possible Causes:

• Commands were sent back-to-back without waiting for the response to the previous command
• Many broadcast commands (which produce no response to wait for) were sent in a rapid burst
• Bus contention: more than one master transmitting, or electrical echo on the RS485 line

Solutions:

• Wait for each command's response before sending the next command to the same device
• Insert a small delay between broadcast commands
• Check the bus wiring for reflections or multiple simultaneous transmitters
• Send the 'System reset' command to clear the fatal error state

Error 6: ERROR_COMMAND_TOO_LONG

Short Description: command too long

Description: Reserved error code. It is defined in the firmware but no condition in the current firmware raises it. A received packet that is too long for the receive buffer is silently discarded instead.

Possible Causes:

• No code path raises this error in the current firmware
• It can only appear if triggered artificially with the 'Test mode' command (values 14 to 73 deliberately raise fatal error codes for testing)

Solutions:

• Send the 'System reset' command to clear the fatal error state

Error 7: ERROR_NOT_IN_OPEN_LOOP

Short Description: not in open loop

Description: The 'Start calibration' command was received while the motor was not in open loop control mode. Calibration must be started from open loop mode, which is the mode the device is in after power-up or after a 'System reset'. It cannot be run after the motor has been switched to closed loop mode.

Possible Causes:

• 'Start calibration' was sent after the motor had already transitioned to closed loop mode (for example via 'Go to closed loop')

Solutions:

• Send the 'System reset' command first (the device boots into open loop mode), then send 'Start calibration'

Error 8: ERROR_QUEUE_NOT_EMPTY

Short Description: queue not empty

Description: A command that requires an empty motion queue was received while queued movements were still pending. This is raised by 'Start calibration', 'Go to closed loop', 'Homing', and 'Zero position' if the motion queue is not empty when they are received (and by 'Vibrate' on the M1 product only; on other products 'Vibrate' does nothing). Both immediate-target commands also raise this error if any active or queued trajectory remains; they reject the new target without cancelling that trajectory.

Possible Causes:

• The command was sent while previously queued moves were still executing or still waiting in the queue

Solutions:

• Wait until 'Get n queued items' returns 0 before sending the command
• Alternatively, send 'Emergency stop' to discard all queued moves, then send the command
• Send the 'System reset' command to clear the fatal error state

Error 9: ERROR_HALL_SENSOR

Short Description: hall sensor error

Description: Not raised by the current firmware (the check that used this code is disabled). Historically it indicated hall sensor readings outside the valid range. Hall sensor problems in the current firmware typically surface as error 47 (hall position delta too large), error 10 (calibration overflow), or error 11 (not enough minima or maxima) instead.

Possible Causes:

• No code path raises this error in the current firmware
• It can only appear if triggered artificially with the 'Test mode' command (values 14 to 73 deliberately raise fatal error codes for testing)

Solutions:

• Send the 'System reset' command to clear the fatal error state
• If you suspect a hall sensor problem, run 'Start calibration' and check for errors 10 or 11, or use 'Get hall sensor statistics' to inspect the sensor signals

Error 10: ERROR_CALIBRATION_OVERFLOW

Short Description: calibration overflow

Description: During calibration the firmware records the positions of the hall sensor signal minima and maxima. More signal extremes were detected than the calibration buffer can hold, so calibration was aborted. This usually means the hall sensor signals contained many spurious peaks.

Possible Causes:

• Noisy or glitchy hall sensor signals producing spurious minima and maxima
• Electrical interference during calibration
• A hardware problem with the hall sensors

Solutions:

• Send the 'System reset' command, then retry 'Start calibration'
• Reduce sources of electrical noise near the motor during calibration
• If calibration fails repeatedly with this error, the device may have a hall sensor hardware problem; contact the manufacturer

Error 11: ERROR_NOT_ENOUGH_MINIMA_OR_MAXIMA

Short Description: not enough minima or maxima

Description: Calibration completed its movement, but at least one hall sensor channel produced fewer signal peaks than expected for a full calibration rotation. This usually means the motor shaft did not actually rotate as commanded, or a hall sensor is not producing a usable signal.

Possible Causes:

• The motor shaft was blocked or under load during calibration (calibration must be done with the shaft free to rotate)
• The motor did not rotate (mechanical binding, or motor windings not driven correctly)
• A faulty or disconnected hall sensor

Solutions:

• Remove any load or obstruction from the motor shaft and retry calibration ('System reset', then 'Start calibration')
• Verify that the shaft actually turns during calibration
• If the shaft turns freely and the error persists, the device may have a hall sensor hardware problem; contact the manufacturer

Error 12: ERROR_VIBRATION_FOUR_STEP

Short Description: vibration four step

Description: Reserved error code. It is defined in the firmware but no condition in the current firmware raises it.

Possible Causes:

• No code path raises this error in the current firmware
• It can only appear if triggered artificially with the 'Test mode' command (values 14 to 73 deliberately raise fatal error codes for testing)

Solutions:

• Send the 'System reset' command to clear the fatal error state

Error 13: ERROR_NOT_IN_CLOSED_LOOP

Short Description: not in closed loop

Description: The 'Homing' command was received while the motor was not in closed loop control mode. Homing works by detecting position deviation when the motor hits an obstacle, which requires closed loop control. Both immediate-target commands also raise this error unless the mode is closed-loop position control. Output enablement is checked separately (error 55).

Possible Causes:

• 'Homing' was sent before switching the motor to closed loop mode

Solutions:

• Send 'Go to closed loop' first (the motor must have been calibrated at some point before that), wait for it to complete, then send 'Homing'
• Send the 'System reset' command to clear the fatal error state

Error 14: ERROR_OVERVOLTAGE

Short Description: overvoltage

Description: The motor supply voltage exceeded the overvoltage threshold. Detection is done by a hardware comparator that triggers immediately via an interrupt. The comparator's reference is set by the firmware to a per-product value (26 V on M1/M2, 32 V on M17, 38 V on M23); there is no user command to change it. The most common cause is regenerated energy: a decelerating or externally driven motor acts as a generator and pumps energy back into the supply rail, raising its voltage.

Possible Causes:

- Regenerative energy from rapidly decelerating a high-inertia load raised the supply voltage
- The motor shaft was spun by an external force while the device was powered
- The power supply voltage is above the specified operating range

Solutions:

- Verify the power supply voltage is within the specified operating range
- Decelerate more gently when moving high-inertia loads
- Use a power supply that can absorb regenerated energy, or add capacitance across the supply
- Avoid forcefully spinning the motor shaft while powered
- Send the 'System reset' command to clear the fatal error state

Error 15: ERROR_ACCEL_TOO_HIGH

Short Description: accel too high

Description: A move was requested with an acceleration whose magnitude exceeds the configured maximum acceleration. This is checked when the move is added to the motion queue, for example by 'Move with acceleration' or by acceleration segments of 'Multimove'. The limit is set with 'Set maximum acceleration'.

Possible Causes:

- The requested acceleration is larger than the configured maximum acceleration
- A unit conversion mistake made the acceleration value much larger than intended

Solutions:

- Reduce the requested acceleration
- Increase the limit with 'Set maximum acceleration' if your application genuinely needs higher acceleration, but note that the firmware silently clamps the maximum acceleration to four times the boot default (48000 rot/s^2 on M17 as of firmware 0.15.8.0); a request above that ceiling returns success yet does not take effect, so this error keeps firing until you lower the requested acceleration
- Double-check the units of the acceleration value you are sending
- Send the 'System reset' command to clear the fatal error state

Error 16: ERROR_VEL_TOO_HIGH

Short Description: vel too high

Description: A velocity exceeded the configured maximum velocity. This is checked when a move is added to the motion queue (for example by 'Move with velocity' or velocity segments of 'Multimove') and is also enforced continuously while moves execute: if the commanded velocity ever exceeds the maximum during execution, this error is raised as well. The limit is set with 'Set maximum velocity'.

Possible Causes:

• The requested velocity is larger than the configured maximum velocity
• The limit was lowered with 'Set maximum velocity' while moves were executing: the new limit takes effect immediately (there is no guard against changing it mid-motion), so if it is below the currently executing velocity the runtime check trips at the next control cycle
• In firmware before 0.15.4.0 only: an 'Emergency stop' or 'Reset time' sent mid-motion left an internal planner variable stale, so a later acceleration-based move could exceed the limit during execution even though it passed validation when queued (fixed in 0.15.4.0)
• A unit conversion mistake made the velocity value much larger than intended

Solutions:

• Reduce the requested velocity or acceleration
• Increase the limit with 'Set maximum velocity' if your application genuinely needs higher speed; note that 'Set maximum velocity' silently clamps any request above twice the boot default (18.67 rot/s on M17 as of firmware 0.15.8.0) to that ceiling and still replies success, so if this error persists after you raised the limit, your request exceeded the ceiling and the limit is capped there
• Only change the maximum velocity while the motor is stopped and the queue is empty
• On firmware before 0.15.4.0 only: after using 'Emergency stop' during motion, send 'System reset' before queueing new moves to restore a consistent internal state (fixed in 0.15.4.0; no reset is needed on current firmware)
• Double-check the units of the velocity value you are sending
• Send the 'System reset' command to clear the fatal error state

Error 17: ERROR_QUEUE_IS_FULL

Short Description: queue is full

Description: An attempt was made to add a move to the motion queue while the queue already held the maximum number of items (32). Note that a 'Multimove' command adds each of its moves to the same queue, and that 'Trapezoid move', 'Go to position', and 'Homing' each add up to 3 items (acceleration, coast, and deceleration segments), so about 10 such moves fill the queue.

Possible Causes:

• Moves were queued faster than they were being executed, without checking the queue level

Solutions:

• Poll 'Get n queued items' and only add moves when there is room in the queue
• Send the 'System reset' command to clear the fatal error state

Error 18: ERROR_RUN_OUT_OF_QUEUE_ITEMS

Short Description: run out of queue items

Description: The motion queue became empty while the motor still had a nonzero commanded velocity. Every motion sequence must end with the motor brought back to zero velocity. If the last queued item finishes with the motor still moving and no further item has been queued, this error is raised.

Possible Causes:

• A move sequence (for example 'Move with velocity' or a 'Multimove') ended at a nonzero velocity with nothing else in the queue
• When streaming moves continuously, the host did not queue the next move in time (queue underrun)

Solutions:

• Always end motion sequences with a move that brings the velocity to zero (for example a final 'Move with velocity' with velocity 0, or a decelerating segment)
• When streaming moves, keep the queue topped up ahead of execution; monitor it with 'Get n queued items'
• Send the 'System reset' command to clear the fatal error state

Error 19: ERROR_MOTOR_BUSY

Short Description: motor busy

Description: A command was received while the motor was busy with an exclusive operation. The operations that make the motor busy are calibration and homing (on the M1 product, also the go-to-closed-loop procedure and vibration). Raised by 'Start calibration', 'Go to closed loop', and all move-queueing commands ('Trapezoid move', 'Go to position', 'Move with velocity', 'Move with acceleration', 'Multimove', 'Homing') if they arrive while such an operation is in progress. Immediate-target commands additionally reject capture/test modes, closed-loop entry and nonzero residual commanded/planned velocity with this code.

Possible Causes:

• A new operation or move was requested before a previous exclusive operation (calibration or homing) finished

Solutions:

• Wait for the current operation to complete before sending more commands; you can poll 'Get status' to see when the device is no longer busy
• Note that 'Start calibration' finishes with an automatic reboot of the device; wait for the device to come back before sending further commands
• Send the 'System reset' command to clear the fatal error state

Error 20: ERROR_CAPTURE_PAYLOAD_TOO_BIG

Short Description: too much capture data

Description: The 'Capture hall sensor data' command was asked to return more data than fits in a single RS485 response packet. The response payload is limited to just under 65536 bytes (about 65526 bytes of data). The requested payload size is the number of points to capture multiplied by 2 bytes for each channel selected in the channel bitmask.

Possible Causes:

• The requested number of points multiplied by 2 bytes per selected channel exceeds the maximum response payload size (about 65526 bytes)

Solutions:

• Request fewer points to capture
• Select fewer channels in the channel bitmask (for example, bitmask 1 captures only the first hall sensor channel at 2 bytes per point)
• Send the 'System reset' command to clear the fatal error state

Error 21: ERROR_CAPTURE_OVERFLOW

Short Description: capture overflow

Description: During a 'Capture hall sensor data' operation, a new data point became ready before the previous one had been transmitted over RS485. The capture was producing data faster than the serial link could carry it.

Possible Causes:

• The capture parameters produce data faster than the RS485 link (230400 baud) can transmit it

Solutions:

• Capture fewer channels by clearing bits in the channel bitmask
• Capture less often by increasing the 'time steps per sample' parameter
• Average more data into each transmitted point by increasing the 'number of samples to sum' parameter
• Send the 'System reset' command to clear the fatal error state

Error 22: ERROR_CURRENT_SENSOR_FAILED

Short Description: current sensor failed

Description: Whenever the MOSFETs are enabled ('Enable MOSFETs', or automatically at the start of calibration, go to closed loop, or vibrate), the firmware measures the motor current baseline while applying zero effective voltage to the motor. The measured baseline was outside the valid window, which indicates the current sensing circuitry is not reading correctly. This check exists on the M1, M2, and M23 products; the M17 firmware performs no such check, so this error does not occur on M17.

Possible Causes:

• A hardware fault in the current sensing circuit
• A power supply problem distorting the baseline measurement

Solutions:

• Send the 'System reset' command (or power cycle) and try enabling the MOSFETs again
• Verify the power supply voltage is within specification and stable
• If the error persists, the device has a hardware defect; contact the manufacturer

Error 23: ERROR_MAX_PWM_VOLTAGE_TOO_HIGH

Short Description: max pwm voltage too high

Description: The 'Set maximum motor current' command requested a motor current or regeneration current setting that maps to a PWM voltage above the absolute maximum the firmware allows.

Possible Causes:

• The requested maximum motor current or maximum regeneration current value is too large
• A unit conversion mistake made the value much larger than intended (the command takes internal current units)

Solutions:

• Request a smaller maximum motor current and/or regeneration current
• Double-check the units of the values you are sending
• Send the 'System reset' command to clear the fatal error state

Error 24: ERROR_MULTIMOVE_MORE_THAN_32_MOVES

Short Description: multi-move more than 32 moves

Description: A 'Multimove' command specified more than the maximum of 32 moves in a single command. As of firmware 0.15.4.0 the count is validated before the move list is copied; in older firmware an oversized but size-consistent packet could corrupt memory before this error was raised.

Possible Causes:

• The 'number of moves' parameter of 'Multimove' was greater than 32

Solutions:

• Send at most 32 moves per 'Multimove' command; split longer sequences into multiple commands
• Send the 'System reset' command to clear the fatal error state

Error 25: ERROR_SAFETY_LIMIT_EXCEEDED

Short Description: safety limit exceeded

Description: The commanded position moved outside the configured safety limits (set with 'Set safety limits'). This is the real-time enforcement of the limits, checked continuously while the motor runs. Note that it tests the commanded (motion profile) position, not the measured position: an external force pushing the rotor does not trigger this error (that situation raises error 45 instead). Separate predictive checks at move-queueing time raise errors 26, 27, and 28.

Possible Causes:

• 'Set safety limits' was sent with limits that exclude the current commanded position (the limits take effect immediately and are not validated against the current position)
• During 'Homing', the collision-detection logic adjusted the commanded position to a point outside the configured limits
• The safety limits were configured too close to the intended operating range

Solutions:

• Set the safety limits with some margin beyond the intended range of motion, and only while the commanded position is inside the new limits
• When homing, make sure the expected contact point lies within the safety limits
• Send the 'System reset' command to clear the fatal error state; note that after a reset the position is re-zeroed and safety limits return to their power-on defaults, so re-establish your reference position and limits

Error 26: ERROR_TURN_POINT_OUT_OF_SAFETY_ZONE

Short Description: turn point out of safety zone

Description: When an acceleration-type move is queued, the firmware predicts the point where the motor would momentarily reach zero velocity and reverse direction (the turn point). The predicted turn point lies outside the configured safety limits (set with 'Set safety limits'), so the move was rejected with this fatal error before executing.

Possible Causes:

• The requested move would carry the motor beyond a safety limit before turning around
• The safety limits are configured too close to the intended range of motion

Solutions:

• Adjust the move so its trajectory, including the turnaround point, stays within the safety limits
• Widen the safety limits with 'Set safety limits' if appropriate
• Send the 'System reset' command to clear the fatal error state

Error 27: ERROR_PREDICTED_POSITION_OUT_OF_SAFETY_ZONE

Short Description: predicted position out of safety zone

Description: When a move is queued, the firmware predicts the position at the end of the move. The predicted end position lies outside the configured safety limits (set with 'Set safety limits'), so the move was rejected with this fatal error before executing. Immediate-target commands use this code to reject a target outside the same configured safety limits before installing it.

Possible Causes:

- The requested move ends outside the safety limits
- The safety limits are configured too close to the intended range of motion
- As of firmware 0.15.4.0, also raised when an extreme acceleration combined with a long move duration would overflow the internal 64-bit position prediction (such a move is invalid in any case)

Solutions:

- Command moves that end within the safety limits
- Widen the safety limits with 'Set safety limits' if appropriate
- Send the 'System reset' command to clear the fatal error state

Error 28: ERROR_PREDICTED_VELOCITY_TOO_HIGH

Short Description: predicted velocity too high

Description: When an acceleration-type move is queued, the firmware predicts the velocity at the end of the move. The predicted velocity exceeds the configured maximum velocity (set with 'Set maximum velocity'), so the move was rejected with this fatal error before executing.

Possible Causes:

- The requested acceleration applied for the requested duration would take the motor past the maximum velocity
- As of firmware 0.15.4.0, also raised when an extreme acceleration-times-duration product would overflow the internal 64-bit velocity prediction (such a move is invalid in any case)

Solutions:

- Use a smaller acceleration or a shorter duration
- Increase the limit with 'Set maximum velocity' if your application genuinely needs higher speed; note that 'Set maximum velocity' silently clamps any request above twice the boot default (18.67 rot/s on M17 as of firmware 0.15.8.0) to that ceiling and still replies success, so if this error persists after you raised the limit, your request exceeded the ceiling and the limit is capped there
- Send the 'System reset' command to clear the fatal error state

Error 29: ERROR_DEBUG1

Short Description: debug1

Description: Internal debug error code. In the current firmware it is used only by internal sanity checks that cannot trigger in a correctly built firmware. If observed, it was most likely triggered artificially via the 'Test mode' command (value 43).

Possible Causes:

- Triggered artificially via 'Test mode' (value 43)
- An internal firmware sanity check failed (indicates a firmware build problem, not a usage error)

Solutions:

- Send the 'System reset' command to clear the fatal error state
- Report the problem to the manufacturer if it occurred without using 'Test mode'

Error 30: ERROR_CONTROL_LOOP_TOOK_TOO_LONG

Short Description: control loop took too long

Description: The periodic motor control interrupt took longer than its allowed execution time budget. Real-time control could no longer be guaranteed, so the device shut down. This indicates a firmware performance problem rather than a usage error.

Possible Causes:

• The motor control calculations exceeded their time budget (firmware performance issue)
• An unusual combination of simultaneously active features increased the control loop's workload

Solutions:

• Send the 'System reset' command to clear the fatal error state
• If reproducible, note which features were active (capture, statistics, streaming, etc.) and report it to the manufacturer with the firmware version

Error 31: ERROR_INDEX_OUT_OF_RANGE

Short Description: index out of range

Description: Reserved error code. It is defined in the firmware but no condition in the current firmware raises it.

Possible Causes:

• No code path raises this error in the current firmware
• It can only appear if triggered artificially with the 'Test mode' command (values 14 to 73 deliberately raise fatal error codes for testing)

Solutions:

• Send the 'System reset' command to clear the fatal error state

Error 32: ERROR_CANT_PULSE_WHEN_INTERVALS_ACTIVE

Short Description: can't pulse when intervals are active

Description: Reserved error code. It is defined in the firmware but no condition in the current firmware raises it.

Possible Causes:

• No code path raises this error in the current firmware
• It can only appear if triggered artificially with the 'Test mode' command (values 14 to 73 deliberately raise fatal error codes for testing)

Solutions:

• Send the 'System reset' command to clear the fatal error state

Error 33: ERROR_INVALID_RUN_MODE

Short Description: invalid run mode

Description: Reserved error code. It is defined in the firmware but no condition in the current firmware raises it.

Possible Causes:

• No code path raises this error in the current firmware
• It can only appear if triggered artificially with the 'Test mode' command (values 14 to 73 deliberately raise fatal error codes for testing)

Solutions:

• Send the 'System reset' command to clear the fatal error state

Error 34: ERROR_PARAMETER_OUT_OF_RANGE

Short Description: parameter out of range

Description: A command parameter had a value that cannot be executed. As of firmware 0.15.4.0 this is raised for: a move duration of 0 in 'Trapezoid move', 'Go to position', 'Move with velocity', or 'Move with acceleration' (and a maximum homing time of 0 in 'Homing', which plans its move the same way) (in older firmware a zero-duration move was a silent no-op that still returned success), and for 'Set maximum acceleration' with the value 0 (which the motion planner divides by). As of firmware 0.15.5.0 it is also raised for 'Set safety limits' with the lower limit greater than the upper limit (in older firmware inverted limits were stored unvalidated and faulted with error 25 one control tick later, racing the command's own reply). As of firmware 0.15.6.0 it is also raised for 'Set maximum velocity' with the value 0 (older firmware accepted 0 and then silently planned every trapezoid/go-to-position as a zero-motion dwell). Other out-of-range parameters raise more specific errors (for example 15, 16, 23, 49, 50, or 51).

Possible Causes:

- A move command was sent with a duration of 0
- 'Set maximum acceleration' was sent with the value 0
- 'Set safety limits' was sent with the lower limit greater than the upper limit (firmware 0.15.5.0 and later)
- 'Set maximum velocity' was sent with the value 0 (firmware 0.15.6.0 and later)
- A unit conversion mistake rounded a small duration down to 0 internal timesteps

Solutions:

- Use a duration of at least one timestep (32 microseconds); durations shorter than half a timestep round to 0
- Use a nonzero maximum acceleration
- Use a nonzero maximum velocity
- Send 'Set safety limits' with the lower limit less than or equal to the upper limit
- Send the 'System reset' command to clear the fatal error state

Error 35: ERROR_DISABLE_MOSFETS_FIRST

Short Description: disable MOSFETs first

Description: Reserved error code. It is defined in the firmware but no condition in the current firmware raises it.

Possible Causes:

- No code path raises this error in the current firmware
- It can only appear if triggered artificially with the 'Test mode' command (values 14 to 73 deliberately raise fatal error codes for testing)

Solutions:

- Send the 'System reset' command to clear the fatal error state

Error 36: ERROR_FRAMING

Short Description: framing error

Description: Not raised as a fatal error by the current firmware. UART framing errors on the RS485 line are detected and counted; the count can be read (and optionally reset) with the 'Get communication statistics' command. Note that the affected byte is still processed, so corruption is only caught by the packet-level checks (first byte validation and CRC32, when enabled) — another reason to keep CRC32 enabled.

Possible Causes:

- Baud rate mismatch between the host and the device (the device uses 230400 baud)
- Poor signal quality on the RS485 line: wiring, termination, or grounding problems
- Electrical interference

Solutions:

- Check the framing error counter with 'Get communication statistics' to monitor line quality
- Verify the host is using 230400 baud
- Check RS485 wiring, termination, and grounding

Error 37: ERROR_OVERRUN

Short Description: overrun error

Description: Not raised as a fatal error by the current firmware. UART receive overruns (bytes arriving faster than the firmware could read them) are detected, counted, and the affected data is discarded; the count can be read (and optionally reset) with the 'Get communication statistics' command.

Possible Causes:

- The device's receive interrupt was delayed while data kept arriving
- Extremely heavy bus traffic

Solutions:

- Check the overrun error counter with 'Get communication statistics' to monitor for this condition
- Reduce bus traffic or add small delays between packets

Error 38: ERROR_NOISE

Short Description: noise error

Description: Not raised as a fatal error by the current firmware. UART noise detections on the RS485 line are counted; the count can be read (and optionally reset) with the 'Get communication statistics' command. Note that the affected byte is still processed, so corruption is only caught by the packet-level checks (first byte validation and CRC32, when enabled) — another reason to keep CRC32 enabled.

Possible Causes:

- Electrical interference on the RS485 line
- Poor wiring, missing termination, or ground potential differences

Solutions:

- Check the noise error counter with 'Get communication statistics' to monitor line quality
- Improve cable shielding, routing, termination, and grounding

Error 39: ERROR_GO_TO_CLOSED_LOOP_FAILED

Short Description: go to closed loop failed

Description: The 'Go to closed loop' procedure could not determine the rotor position with enough confidence (the measured signal quality ratio was below the acceptance threshold), so the transition to closed loop control was aborted. This procedure exists only on the M1 product; on M17, M2, and M23 the 'Go to closed loop' command switches to closed loop mode immediately using the stored calibration and this error can only appear if triggered artificially via 'Test mode' value 53.

Possible Causes:

- The motor has not been calibrated, or the stored calibration does not match the motor
- A load, friction, or an obstruction disturbed the motor during the procedure
- Weak or noisy hall sensor signals

Solutions:

- Run 'Start calibration' (with the shaft free to rotate) and then retry 'Go to closed loop'
- Reduce the load on the shaft during the go-to-closed-loop procedure
- Send the 'System reset' command to clear the fatal error state and retry

Error 40: ERROR_OVERHEAT

Short Description: overheat

Description: The device's internal temperature exceeded the overheat threshold (approximately 80 degrees Celsius) while the MOSFETs were enabled. The temperature is monitored continuously in the background whenever the motor is energized.

Possible Causes:

• Sustained operation at high torque/current
• High ambient temperature or insufficient airflow around the motor
• A stall or mechanical binding causing continuous high current

Solutions:

• Let the motor cool down, then send the 'System reset' command to clear the fatal error state
• Reduce the load, duty cycle, or the maximum motor current ('Set maximum motor current')
• Improve ventilation or reduce the ambient temperature
• Improve heat dissipation: add a heat sink to the motor or mount it against a metallic part of the machine so the structure carries the heat away
• Check for mechanical binding that forces the motor to work excessively hard

Error 41: ERROR_TEST_MODE_ACTIVE

Short Description: test mode active

Description: The 'Start calibration' command was received while a test mode was active. Calibration cannot run while the device is in a test mode.

Possible Causes:

• A 'Test mode' command was sent earlier and the test mode is still active

Solutions:

• Send the 'System reset' command to clear the test mode (and the fatal error state), then start calibration
• As of firmware 0.15.4.0, 'Test mode' with value 0 also safely clears the test modes; in OLDER firmware value 0 locks up the device until it is power cycled — on older firmware always use 'System reset' instead

Error 42: ERROR_POSITION_DISCREPANCY

Short Description: position discrepancy

Description: An internal consistency check failed: when the motion queue emptied, the position accumulated during execution did not match the position that was predicted when the moves were queued. This indicates an internal calculation error in the firmware, not a usage error.

Possible Causes:

• A firmware bug in the motion calculations (this check exists to catch such bugs)

Solutions:

• Send the 'System reset' command to clear the fatal error state
• If reproducible, record the exact sequence of move commands that leads to it and report it to the manufacturer with the firmware version

Error 43: ERROR_OVERCURRENT

Short Description: overcurrent

Description: Reserved error code. It is defined in the firmware but no condition in the current firmware raises it. Motor current is limited by the 'Set maximum motor current' setting rather than by a separate overcurrent shutdown; sustained excessive current will eventually trigger error 40 (overheat) instead.

Possible Causes:

• No code path raises this error in the current firmware
• It can only appear if triggered artificially with the 'Test mode' command (values 14 to 73 deliberately raise fatal error codes for testing)

Solutions:

• Send the 'System reset' command to clear the fatal error state

Error 44: ERROR_PWM_TOO_HIGH

Short Description: PWM too high

Description: The motor control loop computed a PWM duty cycle outside the valid range after compensating for the measured supply voltage. This most commonly means the supply voltage sagged too low for the motor voltage being requested, so the firmware could not produce the required output. In the current firmware this check exists only in the M23 build; on other products this error can only appear if triggered artificially via 'Test mode' value 58.

Possible Causes:

• The power supply voltage dropped (sagged) under load
• The maximum motor current / PWM voltage setting is too high for the available supply voltage
• The supply voltage is below the specified operating range

Solutions:

• Use a power supply that holds its voltage under load (adequate current rating, shorter or thicker cables)
• Reduce the maximum motor current with 'Set maximum motor current'
• Verify the supply voltage stays within the specified operating range while the motor is working; 'Get supply voltage' can help monitor it
• Send the 'System reset' command to clear the fatal error state

Error 45: ERROR_POSITION_DEVIATION_TOO_LARGE

Short Description: position deviation too large

Description: The difference between the commanded position and the actual position measured by the hall sensors exceeded the configured limit (2 shaft rotations by default; changeable with 'Set max allowable position deviation'). The motor could not follow the commanded trajectory. This check runs continuously in the background. Immediate-target commands also check the proposed target against the measured position before installing it, and reject reference steps beyond the configured maximum deviation with this code.

Possible Causes:

• The motor stalled or hit an obstruction
• The load is too heavy for the configured maximum motor current
• The commanded velocity/acceleration exceeds what the motor can deliver at this load
• The deviation limit is configured very tight and normal tracking error exceeded it
• Motion was commanded immediately after enabling the MOSFETs, before the rotor settled (allow roughly 0.3 seconds after 'Enable MOSFETs' before moving when using tight deviation limits)
• Moves were commanded while the MOSFETs were disabled: the commanded position advances but the shaft does not, so the deviation grows with the commanded distance until it crosses the limit
• 'Set max allowable position deviation' was set to a value the tracking check cannot satisfy: a limit of zero trips on the first tick of any deviation (ordinary negative values are converted to their absolute value by the command, so they behave normally; on firmware 0.15.4.0 through 0.15.8.0 the wire value INT64_MIN was an exception that latched fatal 45 immediately (signed-absolute-value edge case, BUG-24); firmware 0.15.9.0 saturates it to the maximum limit instead)

Solutions:

• Remove the obstruction or reduce the load
• Increase the maximum motor current with 'Set maximum motor current' to give the motor more torque
• Use gentler velocity and acceleration
• Increase the limit with 'Set max allowable position deviation' if it is unnecessarily tight
• Send the 'System reset' command to clear the fatal error state

Error 46: ERROR_MOVE_TOO_FAR

Short Description: move too far

Description: A 'Go to position' command requested a target whose distance from the predicted end-of-queue position does not fit in a signed 32-bit displacement. Positions are tracked and reported internally as 64-bit values, and the position can accumulate beyond the 32-bit range through velocity moves, but the displacement of any single 'Go to position' move must fit in a signed 32-bit count value: about plus or minus 655.36 shaft rotations on M17, M23, and M3 (about 3357 on M1, about 470 on M2). This error is exclusive to 'Go to position'; other move commands cannot raise it because their wire formats already limit the displacement.

Possible Causes:

• The requested target position is too many shaft rotations away from the position at the end of the current queue (more than about 655 rotations on M17/M23)
• The current position has accumulated far from zero (for example through long-running velocity moves), so even a modest absolute target is more than the maximum displacement away
• A unit conversion mistake made the target position much larger than intended

Solutions:

• Split very long travels into multiple sequential moves
• Check the current position with 'Get position' before commanding absolute moves after long velocity-mode operation
• Double-check the units of the position value you are sending
• Send the 'System reset' command to clear the fatal error state

Error 47: ERROR_HALL_POSITION_DELTA_TOO_LARGE

Short Description: hall position delta too large

Description: The position derived from the hall sensors jumped further within one control loop cycle than is physically plausible. A single such glitch is tolerated (it can occur at startup); on the second occurrence the MOSFETs are disabled and this fatal error is raised.

Possible Causes:

- Electrical interference or a glitch on the hall sensor signals
- A hall sensor wiring or hardware problem
- The shaft was spun extremely fast by an external force

Solutions:

- Check for sources of electrical interference near the motor and improve shielding/grounding
- Avoid spinning the shaft violently by external means while powered
- Send the 'System reset' command to clear the fatal error state
- If it recurs without an external cause, the device may have a hall sensor hardware problem; contact the manufacturer

Error 48: ERROR_INVALID_FIRST_BYTE

Short Description: invalid first byte format

Description: Not raised as a fatal error by the current firmware. Every packet's first byte must have its least significant bit set to 1 (see the protocol specification). Packets violating this are silently discarded and counted; the count can be read (and optionally reset) with the 'Get communication statistics' command.

Possible Causes:

- The sender is not encoding the packet size byte according to the protocol (LSB must be 1)
- Baud rate mismatch causing bytes to be misread
- Data corruption on the RS485 line
- The device started receiving mid-packet (synchronization loss)

Solutions:

- Check the 'first bit error' counter with 'Get communication statistics' to detect this condition
- Verify the packet encoding in the sender software against the protocol specification
- Verify both ends use 230400 baud
- Check the RS485 line quality

Error 49: ERROR_CAPTURE_BAD_PARAMETERS

Short Description: capture bad parameters

Description: The 'Capture hall sensor data' command was called with one or more invalid parameters.

Possible Causes:

- The capture type was not 1, 2, or 3
- The number of points to capture was 0
- The number of time steps per sample was 0
- The number of samples to sum was 0
- The division factor was 0
- The channel bitmask selected no valid channel: it must have at least one of the low three bits set (values 1 to 7) and no higher bits set

Solutions:

- Use 1, 2, or 3 as the capture type
- Use values of 1 or greater for the number of points, time steps per sample, samples to sum, and division factor
- Use a channel bitmask between 1 and 7 (7 captures all three hall sensor channels; 1, 2, or 4 capture a single channel)
- Send the 'System reset' command to clear the fatal error state

Error 50: ERROR_BAD_ALIAS

Short Description: bad alias

Description: The 'Set device alias' command tried to assign a reserved alias to the device. Aliases 252, 253, and 254 are reserved by the protocol (254 marks extended addressing; 253 and 252 mark responses with and without CRC32) and cannot be assigned to a device.

Possible Causes:

• The requested alias was 252, 253, or 254, which are reserved by the protocol

Solutions:

• Use an alias from 0 to 251, or 255 to leave the device without a specific alias (255 is the broadcast address)
• Send the 'System reset' command to clear the fatal error state

Error 51: ERROR_COMMAND_SIZE_WRONG

Short Description: command size wrong

Description: The payload of a received command did not have the exact size expected for that command's parameters. Every command's payload size is validated: commands with no parameters must have an empty payload, and commands with parameters must have a payload exactly matching the combined size of their parameters.

Possible Causes:

• The command was sent with missing, extra, or wrongly sized parameters
• The packet size byte was encoded incorrectly, so the payload length was misinterpreted
• Data was corrupted during transmission and CRC32 checking was disabled, so the corruption went undetected

Solutions:

• Check the command's exact parameter list and sizes in the API documentation (or motor_commands.json) and send a payload that matches exactly
• Verify the packet size encoding against the protocol specification
• Keep CRC32 enabled so corrupted packets are rejected instead of misinterpreted
• Send the 'System reset' command to clear the fatal error state

Error 52: ERROR_INVALID_FLASH_PAGE

Short Description: invalid flash page

Description: During a firmware upgrade, a 'Firmware upgrade' packet specified a flash page outside the writable firmware region (pages holding the bootloader or the device settings are refused, as are pages beyond the firmware area). This error is raised by the bootloader, which is what runs on the device during a firmware upgrade.

Possible Causes:

• The firmware image being sent is too large for the device's flash
• The upgrade tool computed a wrong page number, or the packet was corrupted

Solutions:

• Use an official .firmware release file and the standard upgrade tool (the upgrade_firmware command, installed by pip install servomotor)
• Verify you are flashing the correct firmware file for this product model
• Power cycle the device and retry the upgrade

Error 53: ERROR_INVALID_TEST_MODE

Short Description: invalid test mode

Description: The 'Test mode' command was called with an unsupported value. Test modes are development and diagnostic features: value 0 clears the test modes (safe as of firmware 0.15.4.0; in older firmware it locks up the device), values 1 to 9 run internal motor tests, 10 to 13 run LED tests, 14 to 73 deliberately trigger fatal error codes (the triggered code is the value minus 14), and 74 to 76 run production hardware tests. Values of 77 and above are invalid and raise this error.

Possible Causes:

- A 'Test mode' value of 77 or higher was sent
- The 'Test mode' command was used without understanding the available test modes

Solutions:

- Avoid the 'Test mode' command during normal operation; it exists for development and production testing
- Warning: 'Test mode' values 10 to 13 lock up the device until it is power cycled (the device stops processing commands, so even 'System reset' cannot recover it); in firmware before 0.15.4.0, value 0 does the same. 'System reset' safely clears all other test modes (motor test modes 1 to 9 and production test modes 74 to 76)
- Send the 'System reset' command to clear the fatal error state

Error 54: ERROR_STREAMING_OVERFLOW

Short Description: streaming overflow

Description: The current-streaming feature produced data faster than it could be transmitted: a streaming buffer became ready to send while the previous buffer's transmission was still in progress (double-buffer overrun). This feature applies to products with current streaming (M23).

Possible Causes:

- The streaming data rate exceeds what the serial link can transmit (the stream is one-way with no flow control, so host behavior cannot cause or prevent this)
- The transmission-complete interrupt was delayed inside the device

Solutions:

- Reduce the streaming data rate
- Stop streaming when it is not needed
- Send the 'System reset' command to clear the fatal error state

Error 55: ERROR_MOSFETS_NOT_ENABLED

Short Description: MOSFETs not enabled

Description: An immediate position target was requested while the output MOSFETs were disabled. Both immediate-target commands require enabled closed-loop position operation. The request is rejected before target mutation; normal fatal-error handling disables outputs and requires a reset.

Possible Causes:

- Sending an immediate target before enabling the motor, or after disabling its output stages

Solutions:

- Reset the motor, establish enabled closed-loop position operation, and retry with an empty trajectory queue